

MOTION IMAGERY STANDARDS PROFILE

Motion Imagery Standards Board



MISP-2025.1: Motion Imagery Handbook

2025-06-11

Table of Contents

Table of Contents	2
List of Figures.....	5
List of Tables	7
Revision History	10
Scope.....	11
Organization.....	11
Chapter 1	13
Terminology and Definitions	13
1.1 Motion Imagery.....	13
1.1.1 Time	13
1.1.2 Frame	13
1.1.3 Image.....	16
1.1.4 Multiple Images.....	17
1.1.5 Full Motion Video (FMV).....	19
1.2 Metadata	19
1.3 Motion Imagery Streaming.....	19
Chapter 2	21
Motion Imagery Functional Model	21
2.1 Introduction.....	21
2.2 Scene	21
2.3 Imager.....	22
2.4 Platform	23
2.5 Control.....	23
2.6 Exploitation.....	24
2.7 Archive	24
2.8 Building Block Functions	24
2.8.1 Compression.....	24
2.8.2 Encodings.....	24
2.8.3 Protocols.....	25
2.8.4 Processing	25
Chapter 3	26
Imagers	26
3.1 Scene Energy	27
3.1.1 Electromagnetic Scene Energy	28

3.2	Energy Adjustments	30
3.2.1	Uncontrolled Energy Adjustments.....	31
3.2.2	Controlled Energy Adjustments	33
3.3	Sensing Process.....	33
3.3.1	Single Element Detection.....	34
3.3.2	Detector Groups.....	38
3.3.3	Sensor Configurations.....	50
3.3.4	Other Sensing Topics	50
3.4	Raw Measurements	50
3.5	Image Creation Process	51
3.6	Image Processing	51
3.7	Image.....	51
3.7.1	Shutter to Scene.....	51
3.7.2	Rolling Shutter.....	51
3.7.3	Interlaced.....	53
Chapter 4		55
Image Color Model		55
Chapter 5		57
Dissemination		57
5.1	Background.....	57
5.1.1	Transmission Methods	58
5.1.2	Internet Protocols.....	59
Chapter 6		62
Time Systems.....		62
6.1	Overview	62
6.2	Time System Elements	62
6.2.1	Timing System Capability Levels.....	65
6.2.2	International Time Systems.....	65
6.2.3	MISP Time System.....	71
6.2.4	Time Systems Summary	72
6.2.5	Time Conversions.....	73
6.2.6	Time Sources	77
6.2.7	Formatting Dates and Times in Text: ISO8601	79
6.3	Timestamp Accuracy and Precision	79
6.4	Time Transfer.....	82

6.4.1	Time Transfer Metadata	90
Chapter 7		94
	Metadata	94
7.1	Metadata Theory	94
7.1.1	Metadata Streams.....	95
7.1.2	Metadata Information.....	96
7.1.3	Metadata Substreams	100
7.1.4	Metadata Operations	103
7.2	Metadata Modeling.....	113
7.3	Metadata Types	114
7.3.1	BER Length.....	115
7.3.2	BER-OID.....	117
7.3.3	Binary.....	118
7.3.4	Boolean	118
7.3.5	Character ISO7	118
7.3.6	Character UTF8	118
7.3.7	Character UTF16	118
7.3.8	Enumeration	119
7.3.9	Floating Point	119
7.3.10	IMAP	119
7.3.11	Integer	119
7.3.12	Unsigned Integer.....	120
7.4	SMPTE – Key, Length, Value (KLV) Metadata Formatting.....	120
7.4.1	KLV Dictionary.....	121
7.4.2	Key.....	121
7.4.3	Length	121
7.4.4	Value.....	121
7.4.5	Parsing Rules.....	124
7.5	MISB KLV Extensions.....	124
7.5.1	MISB KLV Terms and Definitions.....	124
7.5.2	MISB KLV Dictionary.....	125
7.5.3	Value Sizing	126
7.5.4	MISP KLV Groups	126
7.5.5	Float to Integer Mapping.....	128
7.5.6	Data Structures	128

7.5.7	MISP Baseline Packet (Previously MISP Baseline Local Set) Information	129
7.5.8	Implementation of Data Preservation and Data Sharing	132
7.5.9	Report-on-Change.....	132
7.5.10	Metadata Identifiers	137
Appendix A	References	140
Appendix B	Acronyms	142
Appendix C	Reference Algorithms	143
C.1	Reference Software	143
Appendix D	KLV-Key Quick Reference Chart	144

List of Figures

Figure 1: Samples, Pixels, Bands and Frame.....	14
Figure 2: Generation of a Frame	15
Figure 3: Image is a subset of Frame.....	17
Figure 4: Example of Spatial Overlap	17
Figure 5: Relationships: Frame-to-Video and Image-to-Motion Imagery	18
Figure 6: Elements of the Motion Imagery Functional Model.....	21
Figure 7: Motion Imagery from Varieties of Modalities	23
Figure 8: Types of Imagers	26
Figure 9: Imager Processing Model	27
Figure 10: Electromagnetic Spectrum	29
Figure 11: Illustration of Reflection, Refraction and Diffraction.....	31
Figure 12: Uncontrolled Adjustments	32
Figure 13: Transmittance of Energy through the Atmosphere.....	32
Figure 14: Illustration of Timing for a Single Detector.....	35
Figure 15: Illustration of a detector showing photon sensitive and non-sensitive areas.....	36
Figure 16: Illustration of a Detector with a lens to focus most of the incoming photons into the Photon Sensitive area	37
Figure 17: Illustration of blue filter over a single detector	38
Figure 18: Detector Group Patterns.....	39
Figure 19: Illustration of Detector Group, Region and Sub-Region.....	40
Figure 20: Region Readout Orientations across Sub-Regions.....	40
Figure 21: Different Region Orientations in the same Detector Group.....	41

Figure 22: Illustration of a Detector Group with N+1 Sub-Region and a Rolling Shutter Exposure Configuration	42
Figure 23: Illustration of Regions in a Detector Group used to define an Exposure Configuration	43
Figure 24: Example Exposure Pattern	45
Figure 25: Stabilization of Raw Measurements and Detector Group movement.....	49
Figure 26: A Detector Window within a Detector Group (defined by C_1 and C_2) and the resulting image which is a rotation and flip (mirror image) of the projected light onto the Detector Group (<i>Bruges Belgium 2014 Lat: N51 12'18" Lon: E3 13'48"</i>).....	50
Figure 27: Example Motion Effects: Global vs. Rolling Shutter	52
Figure 28: Illustration (simulated) of a rolling shutter image as the imager pans quickly across the scene (<i>Copyright (C) 2014 Her Majesty the Queen in Right of Canada as represented by the Minister of National Defence. All rights reserved.</i>)	53
Figure 29: Illustration (simulated) of an interlaced image as the imager pans quickly across the scene (<i>Copyright (C) 2014 Her Majesty the Queen in Right of Canada as represented by the Minister of National Defence. All rights reserved.</i>)	54
Figure 30: Examples of Formats with Chroma Subsampling	55
Figure 31: Illustration of Clocks	63
Figure 32: Illustration of delay between two clocks count increments	63
Figure 33: Illustration of Clock Relationships	64
Figure 34: Illustration of UT1 LOD and the overlap and gaps that can occur.....	66
Figure 35: Illustration of Leap Seconds Added or Removed from UTC and the associated Date-Text.....	70
Figure 36: Relationships among Time Systems	73
Figure 37: UTC Conversions	75
Figure 38: UTC Conversion with $LD > 0$	75
Figure 39: UTC Conversion with $LD < 0$	76
Figure 40: TAI Leap Second Table	76
Figure 41: A series of time measurements (left) - Errors plotted as a histogram (right).....	80
Figure 42: Example 1 - Poor Accuracy, Good Precision.....	81
Figure 43: Example 2 - Good Accuracy, Poor Precision.....	82
Figure 44: Illustration of GPS time transfer to devices	83
Figure 45: Illustration of Clock Synchronization.....	85
Figure 46: Incorrect method to reacquire lock to Reference Clock.....	89
Figure 47: Correct method to reacquire lock to Reference Clock	89
Figure 48: Time Generator.....	90
Figure 49: Example of losing reference lock twice, with the second loss during a time-convergence	91
Figure 50: Metadata Creation, Life-Cycle and Spawning	95
Figure 51: Metadata Stream.....	96

Figure 52: Relationship between a Metadata Specification and its Instance	97
Figure 53: Ordered List and Unordered List Examples.....	99
Figure 54: Metadata Substreams	102
Figure 55: Metadata Operations.....	103
Figure 56: Example of two Motion Imagery sources with common and unique data	110
Figure 57: BER Short Form Encoding	116
Figure 58: BER Long Form Encoding.....	116
Figure 59: One-byte BER-OID Example.....	117
Figure 60: Multi-byte BER-OID Example	117
Figure 61: Illustration of Key Length Value Item.....	120
Figure 62: Key's Components	121
Figure 63: Illustration of a Local Set.....	123
Figure 64: Illustration of a Variable Length Pack.....	123
Figure 65: Illustration of a Defined Length Pack.....	123
Figure 66: Floating Length Pack Illustration	127
Figure 67: Defined Length Pack Example.....	127
Figure 68: Truncation Pack Example	127
Figure 69: Example of Truncation Pack	128
Figure 70: Example of Truncation Pack with Filler Value	128
Figure 71: Illustration of Local Set CRC Computation.....	130
Figure 72: Method 1 – Check Value for a Defined Length Pack.....	131
Figure 73: Method 1 – Check Value for a Variable Length Pack	131
Figure 74: Method 2 – Check Value for a Defined Length Pack	132
Figure 75: Method 2 – Check Value for a Variable Length Pack	132
Figure 76: Report-on-Change State Diagram	135
Figure 77: Example of metadata lost or Unknown metadata	136
Figure 78: Local Set item vs. Zero-Length Item	137
Figure 79: Motion Imagery Standards Board PiX Home Page	143

List of Tables

Table 1: Tiers of Live and On-Demand Streaming Latency	20
Table 2: Measureable EMR Properties.....	29
Table 3: Electromagnetic Bands - The exact wavelength/frequency ranges are notional	29

Table 4: Exposure Configuration for Figure 22	42
Table 5: Exposure Pattern for Figure 23	44
Table 6: Exposure Metadata for Figure 23 using Exposure Pattern ($S_{R,0,0}$, $E_{R,0,0}$, and $S_{R,1,0}$)	46
Table 7: Single Region Exposure Pattern	46
Table 8: Typical Detector Group Sensor Types	47
Table 9: Window Corner Values	50
Table 10: Pixel Value Range for Various Color Sampling Formats	56
Table 11: Internet Protocols	59
Table 12: UDP Error Types	59
Table 13: MPEG-2 TS Error Types	60
Table 14: Leap Seconds since January 1972	68
Table 15: Leap Second Computation for Dates Ranging from 1961-01-01 to 1972-01-01. Data derived from U.S. Navy Observatory file (ftp://maia.usno.navy.mil/ser7/tai-utc.dat)	69
Table 16: List of Time Systems	72
Table 17: Time System Conversions	74
Table 18: Slow receptor clock reacquiring the reference clock. The yellow highlight indicates the large jump forward in the receptors time.	88
Table 19: Fast receptor clock reacquiring the reference clock and adjusting the reported time incorrectly. The yellow highlight indicates a large jump backwards in the receptors time and the red highlights the discontinuity	88
Table 20: Fast receptor clock reacquiring the reference clock and adjusting the reported time for a slew correction. The receptor's time increments slowly (1 millisecond/tick) while the reference clock catches up. The highlighted yellow shows where the lock is reacquired, and the blue highlight shows where the re-synchronization occurs.	89
Table 21: Points of change in Figure 49	92
Table 22: Example of Ordered List and Unordered List	98
Table 23: Example of a Tree Structure	100
Table 24: Metadata Substream Example	101
Table 25: Example showing the difference of deleting the "Sensor Type" element using the Delete and Enhanced Delete operations for an unordered list	104
Table 26: Example showing the difference of updating the "HAE" element using the Update and Enhanced Update operations for an unordered list	105
Table 27: Example showing an Enhanced Delete-and-Update Amend Metadata as sub-lists of the first-generation metadata	105
Table 28: Possible interpretations of metadata by end user	106
Table 29: Example showing enhanced delete and update Amend Metadata as sub-lists of first-generation metadata	106
Table 30: Possible interpretations of metadata by end user	106

Table 31: Example showing the Amend Metadata with an Enhanced Update operation which includes new metadata	107
Table 32: Example showing two metadata packets each with two Amend sub-lists	107
Table 33: Example showing two metadata packets each with two Amend sub-lists and their respective MSIDs.....	108
Table 34: Example showing two metadata packets each with a hierarchy of Amend sub-lists and their respective MSIDs	109
Table 35: Common Metadata Combined.....	111
Table 36: Options in interpreting the metadata of Figure 56	111
Table 37: Example showing two metadata packets each with two Segment sub-lists	112
Table 38: Example showing two metadata packets each with two Segment sub-lists and their respective MSIDs.....	112
Table 39: Amend Metadata list in a Segment Metadata list	113
Table 40: List of Simple Data Types used by the MISB	115
Table 41: Summary of KLV Groups	122
Table 42: Series of 100 Metadata Packets	133
Table 43: Removing ID Redundancy	133
Table 44: Removing Y Value Redundancy	133
Table 45: Data Reduction with Static Data Updates	134
Table 46: UUID Combination Example	138

Revision History

Version	Approved Date	Change Description
2025.1	2025-06-11	• Revised Appendix C to reflect MISB PiX Site • Editorial revisions throughout the document • Updated references

Scope

The purpose of the Motion Imagery Handbook is to provide:

1. A definition of Motion Imagery.
2. Common terminology for all MISB documentation.
 - a. There is no single authoritative source for technical definitions of terms within the community; therefore, the Motion Imagery Handbook serves as the authoritative source of definitions for the MISB community of practice.
3. Additional detail for topics identified in the Motion Imagery Standards Profile [1].
 - a. The MISP succinctly states requirements, while the Motion Imagery Handbook discusses principles underlying requirements more thoroughly.

Many definitions and terms are used throughout the various commercial groups and vendors however many of these terms are either overloaded with conflicting meanings or there is disagreement about what a term means. The purpose of this document is to provide the MISB view of these definitions when these term definitions arise. The MISB has a “reference, clarify or define” philosophy when using term and definitions:

- When a term is well defined and accepted then we defer the definition to a formal external reference.
- When a term is not well defined due to overloaded use or disagreement then this document will clarify how the MISB will use the term within the MISP documents. The term definition will appear in a formal definition block (see Organization Section).

When a term is non-existent this document will provide the definition as a proper noun. The definition for a term appears in a formal definition block (see Organization Section). Although intended to be educational and informative the Motion Imagery Handbook is not a substitute for available material that addresses the theory of imaging, video/compression fundamentals, and transmission principles.

Organization

The Motion Imagery Handbook is composed of chapters, each emphasizing different topics that support the Motion Imagery Standards Profile (MISP) [1]. Each chapter is intended to be self-contained with references to other chapters where needed. Thus, a reader will be able to quickly locate information without reading preceding chapters. The Motion Imagery Handbook is expected to mature over time to include material considered essential in applying the requirements within the MISP as well as other MISB standards.

Formal Definitions

This document uses highlighted text blocks to emphasize formal definitions. Each definition states the term with bolded and underline emphasis, followed by a colon and the definition. The definition uses underlining to indicate other Handbook defined terms.

For example, the following formal definition block defines: Term, MISB Term and Overloaded Term.

Term: A word or phrase that has a specific meaning within the MISB documents.

MISB Term: A term the MISB defines which is not in other technical literature. MISB Terms may be a common set of words combined to form a precise meaning, in which case the term is a proper noun (E.g., Absolute Time).

Overloaded Term: A term defined by more than one source which does not have a common or clear singular interpretation. The MISB redefines the term from the perspective of how the MISB documents will use the term.

Throughout the remainder of the document, MISB Terms maintain their proper noun capitalization to indicate the term is MISB defined. Overloaded Terms will maintain their normal capitalization form (e.g., lower case, or proper noun, or acronym).

Examples:

- Using the MISB Term “Absolute Time” in a sentence. “The Absolute Time is an offset from a well-known epoch.”
- Using the Overloaded Term “pixel” in a sentence. “A combination of samples forms a pixel.”

Chapter 1

Terminology and Definitions

1.1 Motion Imagery

Many different sensor technologies produce Motion Imagery. To support an imaging workflow in which different sensor data can be utilized by an exploitation system, standards defining common formats and protocols are needed. The standards facilitate interoperable functionality, where different vendor products can readily be inserted within the workflow based on improvement and cost. Such standards need to be developed on a well-defined and integrated system of terminology. This section lays the foundation for this terminology.

1.1.1 Time

Time is fundamental in Motion Imagery. All events whether captured by a camera or artificially created are either formed over a period of time or are displayed over a period of time. In a camera, for instance, the light reflected or emitted from an object is exposed onto the camera's "sensor", which could be some type of imaging array or film. A **Start Time** and an **End Time** bounds the period of exposure. These are important qualifiers that play a significant role in image quality, particularly in capturing motion within the imagery. When exposure times are too long the motion is blurry; when exposure times are too short the motion may not be apparent.

A timeline is the continuous passage of time. An instant is an infinitesimal point on the timeline. Time is a measured offset between an instant and an absolute or relative reference on the timeline. Although these terms have various definitions in the literature, the MISB defines these terms here specific to their application in the MISB community. **Absolute Time** measures are an offset to a known universal source, such as International Atomic Time (TAI). **Relative Time** measures are an offset from some defined event. For example, Relative Time is a basis for overall system timing in the formatting of Motion Imagery, compression, and data transmission. Chapter 6 provides further discussion of time and Time Systems.

Start Time: Instant a process is initiated, measured in either Absolute Time or Relative Time.

End Time: Instant a process is completed, measured in either Absolute Time or Relative Time.

Absolute Time: An instant measured as an offset to a known universal source's (e.g., TAI) starting point, which is called an epoch.

Relative Time: An instant measured as an offset from a starting event.

1.1.2 Frame

The term frame is a common term for describing video and Motion Imagery, for instance, image frame, frame size, frames per second, etc. A **frame** is a two-dimensional array of regularly spaced values, called pixels that represent some type of data – usually visual.

A **pixel** is a *combination* of one or more individual numerical values, where each value is called a **sample**. A **sample** is data that represents a measured phenomenon such as light intensity.

In considering visual information, a frame could be the data representing a monochrome (i.e., greyscale) or color picture. For example, with a monochrome picture, the pixel values are the intensity values of light at each position in the picture. In a color picture, the pixel data is composed of three different intensity values, i.e., red, green, and blue at each position in the picture.

An array of samples where all phenomena are of the same type is called a **band**. For example, a band of samples for the red component of color imagery contains only measurements of light sensitive to the “red” wavelength. For a monochrome picture, one band is sufficient, whereas for color, three bands are needed. A frame can consist of pixels combined from one or more bands. Figure 1 illustrates the relationships of samples and bands to a frame.

A pixel is a combination of a number of samples collectively taken from a number of bands (see Figure 1). Where there is only one band, a pixel is equivalent to a sample. A “color” pixel is a combination of red, green and blue samples from corresponding red, green and blue bands. In Chapter 4 various color models where the relationship between pixels and samples are not one-for-one are discussed.

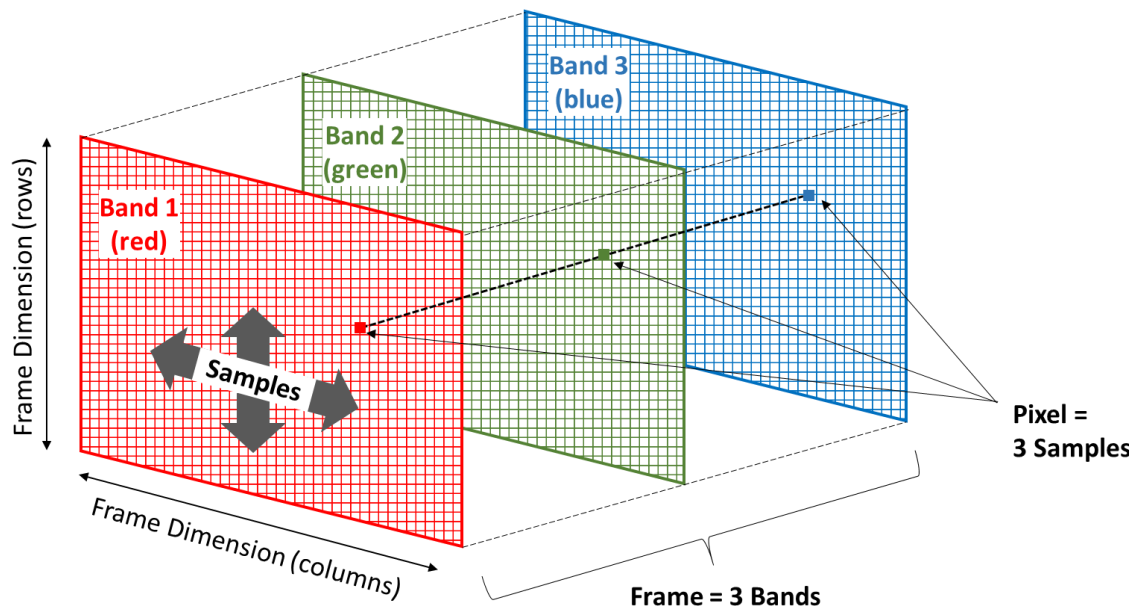


Figure 1: Samples, Pixels, Bands and Frame

The **frame dimension** is the height of the frame measured in pixels per column and the width measured in pixels per row. Pixels within a frame are bounded in their measurement over a period of time; that is, they have a **Start Time** and **End Time**. The Start Time and End Time may be based on Relative Time or Absolute Time. Although all pixels within a frame generally have the same Start Time and End Time, this is not always the case. A frame is bounded by a **Frame Start Time** and **Frame End Time**, which accounts for the extremes of Start Time and End Time for the individual pixels. Subtracting the next frame's Frame Start Time from the current frame's Frame Start Time results in the **frame period**. The inverse of the frame period ($1/\text{frame period}$) is the **frame rate**. **Sample**: a numerical value that represents measured phenomena, such as light intensity along with its Start Time and End Time.

Band: collection of samples where all measured phenomena are of the same type.

Pixel: A combination of a number of samples collectively taken from a number of bands.

Frame: A two-dimensional array of regularly spaced pixels in the shape of a rectangle indexed by rows and columns along with a Start Time and an End Time of each pixel.

Frame Dimension: The height and width of a frame measured in pixels per column and pixels per row, respectively.

Frame Start Time: The minimum time value of all pixel Start Times within a frame.

Frame End Time: The maximum of all pixel End Times within a frame.

Frame Period: The time measured from the Frame Start Time to the next frame's Frame Start Time.

Frame Rate: Inverse of frame period, measured in inverse seconds or Hertz (Hz).

A frame is constructed by processing Source Data into a two-dimensional rectangular array of pixels as illustrated in Figure 2.

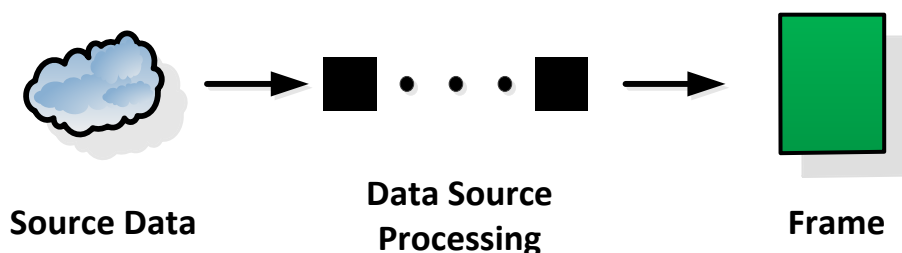


Figure 2: Generation of a Frame

There are two types of **source data**: scene or computer-generated. The space in the physical world imaged by a sensor is called the **scene**. **Scene data** is data sensed by any device that detects scene energy from the scene. **Scene energy** includes Visible, Infrared, and Ultra-Violet light, plus RADAR and LIDAR returns, Acoustical or any type of data radiating from the scene. **Computer-generated data** is data that is not emanated from a scene but is rather manufactured to represent or simulate some type of scene or other information.

Scene: Space in the physical world that is sensed by a sensor and used to form an image.

Scene Energy: Energy in any form that is radiated from the scene.

Scene Data: A representation of the scene energy that is sensed and sampled by any device that detects energy from the scene.

Computer-Generated Data: Manufactured or simulated scene data.

Source Data: Either scene data or computer-generated data.

The Data Source Processing maps the source data to a two-dimensional rectangular output – the frame. The Data Source Processing depends on the type of source data; it can be integral to the sensor or exist as a separate system. Examples of Data Source Processing include Visible Light or Infrared (IR) cameras; the post processing of a 3D LIDAR cloud that supports a viewport into the 3D LIDAR Scene; a simulation of flying over a city; simple text annotation. The Data Source Processing can provide contextual metadata for the frame and how it was formed.

The Data Source Processing may produce near instantaneous frames or frames where the data is integrated over a period a time. Both types of frames are bounded with a Frame Start Time and a Frame End Time (for the near-instantaneous case the Frame Start Time and Frame End Time are considered identical).

1.1.3 Image

Images are special cases of frames. Frames represent content from either scene or computer-generated data; however, **images** are frames created only from scene data.

Image: A frame with pixels derived from scene data.

Image Dimension: The height of an image measured in pixels per column and the width of an image measured in pixels per row.

Image Start Time: The Start Time of an image.

Image End Time: The End Time of an image.

Graphics (e.g., Newscast) and computer animation are based on frames, but because they are not produced from sensor data, they are not images. In contrast, the pictures from an air vehicle sensor, underwater sensor and sonar sensor are all images, because they are formed from sensed data. Image is a subset of frame (depicted as in Figure 3), therefore, images retain all of the attributes of frames (i.e., rectangular array structure and time information).

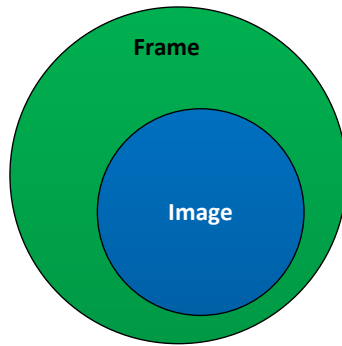


Figure 3: Image is a subset of Frame

1.1.4 Multiple Images

With two or more images, relationships amongst images can be formed both spatially and temporally. When two images contain some portion of the same Scene, there is spatial overlap – these are called **Spatially Related Images**. Figure 4 illustrates spatial overlap, where the red square outlines similar content in each image. Spatially Related Images do not necessarily need to occur within some given time period. For example, the two images in Figure 4 may have been taken within milliseconds, minutes, or hours of one another. Spatially Related Images may be separated by a large difference in time, such as images of a city taken years apart.



Figure 4: Example of Spatial Overlap

Images collected at some regular time interval, where the images form a sequence, the Image Start Time for each is known, and each successive image temporally follows the previous one, are called **Temporally Related Images**. There are no criteria that the content within Temporally Related Images be similar, only that they maintain some known time relationship.

Spatio-temporal data is information relating both space and time. For example, capturing a scene changing over time requires a sequence of images to be captured at a periodic rate. While each image portrays the spatial information of the scene, the sequence of these images portrays the temporal or time-varying information. Images that are both spatially and temporally related are called **Spatio-Temporally Related Images**. These are the type of images found in Motion Imagery.

Spatially Related Images: Images where recognizable content of a first image is contained in a second image.

Temporally Related Images: When the two Image Start Times of two images are known relative to each other; the second image is always temporally after the first.

Spatio-Temporally Related Images: When two images are both Spatially Related Images and Temporally Related Images.

By collecting a series of frames and/or images, **video** and **Motion Imagery** can be defined. The term “video” is not well defined in the literature. The word video is Latin for “I see.” It has become synonymous with standards and technologies offered by the commercial broadcast industry. As such, the term serves a rather narrow segment of the application space served by Motion Imagery.

Video: An ordered series of frames with each frame assigned an increasing Presentation Time, where the Presentation Time is a Relative Time.

Presentation Time: A Relative Time associated with each frame.

This definition of video includes **Presentation Time**, which is an arbitrary relative timeline that is independent of a frame’s Start Time. For example, a video of a glacier ice flow created by taking one picture per day has a Frame Start Time that is 24 hours apart from the next frame; the Presentation Time, however, is set to play each frame at a 1/30 second rate (i.e., video at 30 frames per second).

Motion Imagery: A video consisting of Spatio-Temporally Related Images where each image in the video is spatio-temporally related to the next image.

Video is created from a sequence of frames, whereas Motion Imagery is created from a sequence of Spatio-Temporally Related Images. Just as image is a subset of frame, Motion Imagery is a subset of video. These relationships are depicted in Figure 5.

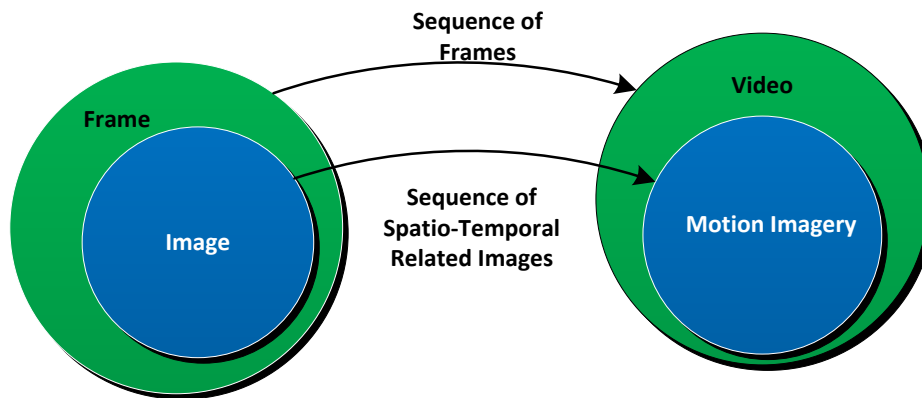


Figure 5: Relationships: Frame-to-Video and Image-to-Motion Imagery

1.1.5 Full Motion Video (FMV)

The term **Full Motion Video** (FMV) loosely characterizes Motion Imagery from Visible Light or Infrared sensors, playback rates typical of video, and frame dimensions typical of those found in the commercial broadcast industry, defined by standards development organizations like SMPTE and ISO.

As with video, the term FMV characterizes a rather narrow subset of Motion Imagery. It is recommended the term FMV not be used, because of its ill-defined and limited applicability across the diverse application space served by Motion Imagery. Moreover, there is no clear definition for FMV available – it is sort of tribal knowledge and varies depending on who is asked. Historically, the term FMV was coined in the 90s by a vendor of video transponders to describe analog video that could be played back at its native frame rate showing all of the “motion” in the video.

The term FMV shall not be used in contractual language.

1.2 Metadata

Motion Imagery is the *visual* information that is exploited; however, in order to evaluate and understand the context of the Motion Imagery and its supporting system additional information called **metadata** is needed. The types of metadata include information about the sensor, the platform, its position, the image space, any transformations to the imagery, time, image quality and archival information. Many MISB standards specifically address the definition of metadata elements and the formatting of the metadata associated with Motion Imagery. Chapter 7 discusses metadata in greater depth.

<u>Metadata:</u> Data that describes and gives information about other data.

1.3 Motion Imagery Streaming

Dissemination is a vital component of any imagery workflow. For Motion Imagery **Streaming** is the continuous flow of the Motion Imagery from a sensor to a client for display or processing. Examples include Motion Imagery transmitted from a sensor via Serial Digital Interface (SDI) to a display device, Motion Imagery compressed and encapsulated as a MPEG-2 Transport Stream within UDP/IP packets distributed over a network to a client device/application, and Adaptive Bitrate Motion Imagery compressed and encapsulated as ISO Based Media File Format (IBMF) segments stored on a server distributed via server/client protocol to a client device/application.

<u>Streaming:</u> A continuous flow of <u>Motion Imagery</u> to a client.
--

There are two dominant streaming CONOPS: Live and On-Demand. **Live Streaming** is streaming without intermediate archival. Live streaming typically is a “push” dissemination where Motion Imagery is broadcast or unicast, and a client accesses the feed.

On-Demand Streaming is streaming from an archive initiated by a client request to begin the streaming process. On-demand streaming typically is a “pull” dissemination where the Motion Imagery is transmitted between a server and a client.

Live Streaming: Streaming without intermediate archival.

On-Demand Streaming: Streaming from an archive.

A critical consideration in all Motion Imagery streaming is **Latency**. Latency is the time delay between the acquisition or request of Motion Imagery to the Motion Imagery being available at a client. Many factors contribute to streaming latency: data representation conversions, compression/decompression, packaging/unpacking, client buffering, client/server response, transmission protocols, length of transmission, etc.

Latency: Time delay between the cause and effect of some physical change in an observed system.

Live Streaming Latency: Latency measured from the acquisition of Motion Imagery to the availability of Motion Imagery at a client display or subsequent process.

On-Demand Streaming Latency: Latency measured from a client request to the availability of Motion Imagery at the client display or subsequent process.

Table 1 categorizes Live and On-Demand streaming latency into tiers useful to frame requirements in application development.

Table 1: Tiers of Live and On-Demand Streaming Latency

Tier	Live Streaming Latency	Tier	On-Demand Streaming Latency
L1	Latency $\leq$ 250ms	D1	Latency $\leq$ 2s
L2	250ms > Latency $\leq$ 2s	D2	2s > Latency $\leq$ 10s
L3	2s > Latency $\leq$ 10s	D3	10s > Latency $\leq$ 25s
L4	10s > Latency	D4	25s > Latency

Chapter 2

Motion Imagery Functional Model

2.1 Introduction

A Motion Imagery Functional Model offers a common narrative across different audiences, such as Program Managers/Procurement officers, Technical Developers and End Users. The Functional Model identifies the elements of systems that generate, manipulate and use Motion Imagery based on the logical data flow from the Scene to the Analyst as shown in Figure 6. In addition, processing functions (denoted in the red blocks of Figure 6) to format and manipulate the Motion Imagery are also included in the Functional Model.

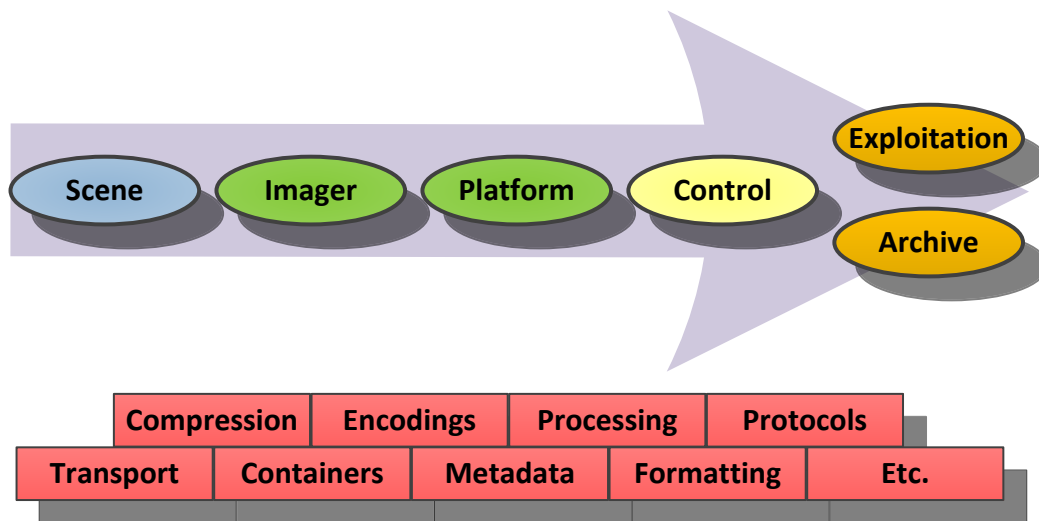


Figure 6: Elements of the Motion Imagery Functional Model

Using the Motion Imagery Functional Model, MISB Standards (ST) and Recommended Practices (RP) that address particular stages in the model are easily relatable. This facilitates ready association to those mandatory standards when specifying system requirements.

The Building Block Functions are core to MISB ST and RP documents; these may be cast in standalone documents, or as guidance provided within the MISP, where the function is defined generically and then referenced within MISB STs and RPs.

2.2 Scene

The scene propagates many different types of energy (scene energy) so different imager types may be used to construct images of a scene. Each scene may produce multiple types of scene data at the same time if multiple imagers are used at the same time. Typical imager types include:

- Electro-Optical - Emitted or reflected energy across the Ultra-Violet/Visible/Infrared portion of the electromagnetic spectrum (Ultraviolet, Visible, near IR, and IR)
 - Ultraviolet – Pictorial representation of Ultraviolet Energy
 - Visible Light – Color or Monochrome
 - Infrared Light – Pictorial representation of thermal Infrared Energy
 - Spectral Imagery – image data captured in discrete frequency spectral bands across the electromagnetic spectrum
 - MSI – Multispectral Imagery - 10's of individual spectral bands
 - HSI – Hyperspectral Imagery - 100's of individual spectral bands
- RADAR – Energy from the radio frequency portion of the electromagnetic spectrum transmitted towards the scene then reflected back and converted into an image representation
- LIDAR – Laser pulses transmitted towards the scene then reflected back providing range information (i.e., point cloud) that is converted into an image

2.3 Imager

The **imager** converts the scene energy into an image and, when possible, provides metadata, such as the imager characteristics or creation time for samples or pixels. The MISP specifies requirements on the format of imagery produced by an imager, such as horizontal and vertical sample/pixel density, temporal rates, and sample/pixel bit depth. These requirements assure common formats and structures are used, thereby facilitating interoperability.

Imager: Process that converts <u>scene energy</u> into an <u>image</u> .

Figure 7 illustrates the varieties of scene energy used to create Motion Imagery. Scene energy is processed into images along with associated metadata. While the methods used to sense and measure the different types of scene energy are unique, a resulting image, along with its metadata is the common result.

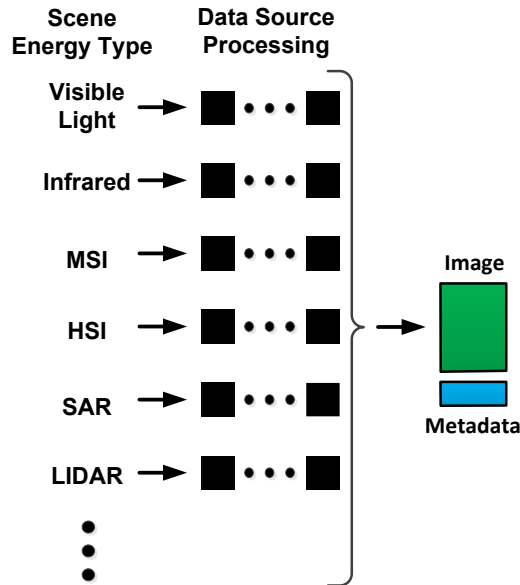


Figure 7: Motion Imagery from Varieties of Modalities

There are many types of imagers depending on the type of scene energy and the phenomena being sensed. An important aspect of the imager is the metadata gathered during the image formation process. The temporal information, orientation and position of the collected image are needed for accurate geographical or relativistic positioning. When this information is not available during the image formation process then it is estimated in later stages of the functional model which may reduce accuracy.

Chapter 3 provides in-depth information about imagers.

2.4 Platform

Any system to which the imager is attached may be considered its platform. A platform may provide information regarding its environment, such as time, place, orientation, condition of the platform, etc. that may be quantified and provided in the form of metadata along with imager essence. The MISP provides numerous metadata elements that serve specific purposes within its suite of Standards documents.

Platform: A static (i.e., non-moving) or mobile system to which an imager is attached.

2.5 Control

Motion Imagery systems generally allow for control over the imager, whether orienting its direction dynamically, or modifying its parameters, such as contrast, brightness, image format, etc. The MISB does not issue guidance for control of a platform; it does, however, prescribe metadata to indicate the state of control variables and actions and to enable image transformations whether at the platform or in later phases of processing.

2.6 Exploitation

Exploitation of Motion Imagery may range from simple situational awareness – the when and where – to in-depth extraction of detected features, measurement, and coordination with other intelligence data. Because the tools used in exploitation operate on the data structures of Motion Imagery, revisions to the MISP are backward compatible as much as possible, so all operational tools may continue to function as new capabilities are made available. While this is a goal, the advance and adoption of new technologies may impact compatibility in some cases.

2.7 Archive

Motion Imagery content is stored for later phases of exploitation, generating reports and historical reference for comparison. An important aspect of storage is file format. Choosing a standardized file format and a means to database/search the Motion Imagery is critical to reuse. The MISP provides guidance on several file containers and continues to evaluate new technologies that may offer greater value in the community.

2.8 Building Block Functions

A Building Block Function is itself a MISB standard that defines a reusable function that supports other higher-level MISB standards.

2.8.1 Compression

Motion Imagery typically is output by an imager as a number of continuous sequential Images, where each image contains a defined number of samples/pixels in the horizontal direction (columns) and a defined number of samples/pixels in the vertical direction (rows). The images are spaced at a fixed time period.

Compression is an algorithmic sequence of operations designed to reduce Motion Imagery sequence data, so the data may be transported within a prescribed bandwidth transmission channel. The tradeoffs in compressing Motion Imagery are transmission data rate, image quality and latency. These must be optimized on a per-application basis. The MISB governs the type of compression and provides guidelines for its proper use. Decompression reverses the process of compression which may result in lossless or lossy data depending upon the compression algorithm.

<u>Compression:</u> Process that reduces the space occupied or bandwidth required for data.
<u>Decompression:</u> The process of restoring compressed data to its approximate original form.

Audio is another “essence” type that may be provided by the platform. It also is typically compressed, and the MISB allows a choice among several industry compression standards.

2.8.2 Encodings

An encoding is the process of putting a sequence of characters (letters, numbers, and certain symbols) into a specialized format for efficient transmission or storage. Encodings such as KLV (Key Length Value) format are designed for low-overhead representations of metadata. While

many MISB Standards assume KLV encodings for metadata, the MISB is investigating other encodings for use in web-enabled environments.

2.8.3 Protocols

Protocols provide the linkage for systems to communicate; they are key to interoperability. Protocols include the interface specifications for data transfer between functions along the Motion Imagery Functional Model. MISB chooses protocols specified by the commercial and international standards development organizations. When appropriate, these protocols are further profiled for specific use in this community, which aids interoperability and conformance.

2.8.4 Processing

Many points along the data flow within the Motion Imagery Functional Model are acted on for conversion, formatting and improvement to the signals passed. Examples include image transformations, data type conversion, and clipping of streams into smaller files. While the MISB does not dictate specific implementations of processing, the Standards are designed to provide flexibility and consistency across implementations.

Chapter 3

Imagers

There are two types of imagers: Direct and Indirect. A Direct Imager transforms the raw source data information into an image. Examples of Direct Imagers include visible light cameras, infrared cameras, and hyperspectral sensors that gather data, perform some processing and generate the image from the same perspective as the sensor. An Indirect Imager transforms the raw source data into an intermediate form, which is then processed into an image via projective methods. A LIDAR sensor is an example of an Indirect Imager that produces images by first building a point cloud; images are then built from “flying” around the point cloud. Figure 8 illustrates the difference between the two types of imagers.

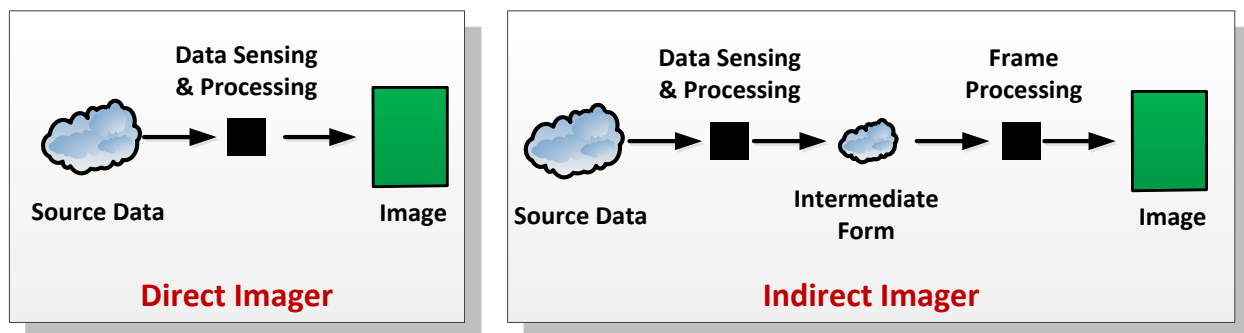


Figure 8: Types of Imagers

The remainder of this chapter focuses on Direct Imagers. All Direct Imagers convert scene energy to a series of images. The process for producing an image is unique for the given type of scene energy that is imaged; however, at a high level there are a common set of steps that define the Imager Processing Model.

Producing an image from scene energy is a multi-step process with each step using the output of one or more previous steps spatially, temporally, or both. When performing precise exploitation of the Motion Imagery, it is important to understand what data manipulations have been performed on the original scene energy. The Image Processing Model shown in Figure 9 provides a consistent method for recording the process.

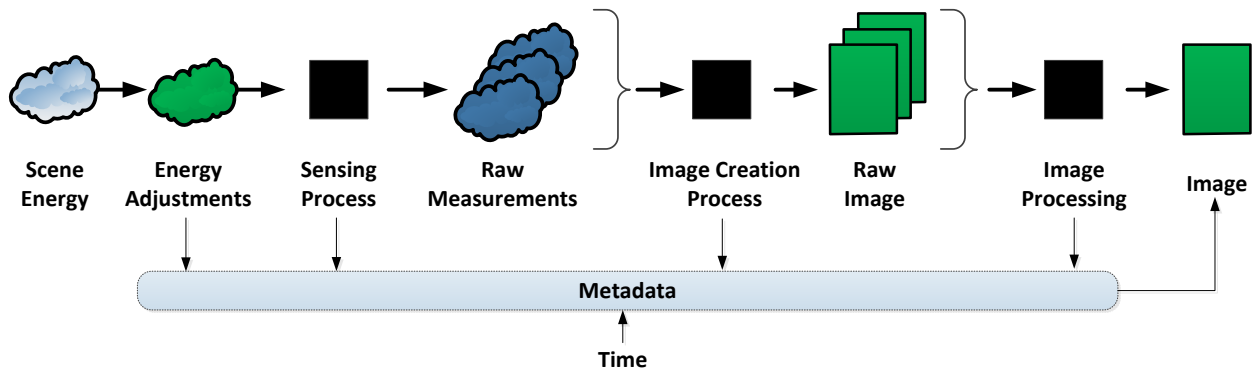


Figure 9: Imager Processing Model

The Image Processing Model defines a number of steps; however, depending on the scene energy type and sensor type not all of the steps are needed (and can be skipped) to produce an image. The only required step is the Sensing Process.

The Imager Processing Model shows the information flow from left to right:

- **Scene Energy:** the energy that emanates from a scene (Section 3.1).
- **Energy Adjustments:** adjustments to the scene energy that occurs before the Sensing Process; for example, atmospheric distortions, optical filters, lens focusing, and distortions (Section 3.2).
- **Sensing Process:** measures the scene energy into a set of digital Raw Measurements (Section 3.3). Examples include CCD camera, CMOS camera, Infrared camera and LIDAR receiver.
- **Raw Measurements:** a two-dimensional array of samples (in any configuration, i.e., shape or spacing) that are measurements from the scene. Each sample has a numeric value, a location relative to the other samples, a Start Time and an End Time (Section 3.4).
- **Image Creation Process:** maps the Raw Measurements to a set of regularly spaced homogenous samples in the shape of a rectangle. The mapping is dependent on the type of scene energy and Raw Measurements. An image can be created from one or more sets of Raw Measurements either temporally or spatially (Section 3.5).
- **Raw Image:** same as definition of image in Section 1.1.3.
- **Image Processing:** either augmentation or manipulation of one or more images (spatially or temporally) for the purpose of formatting or enhancing an image (Section 3.6).
- **Image:** as defined in Section 1.1.3.

3.1 Scene Energy

Scene energy is any energy propagated from scene to the Sensing Process. The energy comes from one or more of the following:

- Generated by the objects in the scene,

- Transmitted through objects in the scene,
- Reflections from objects in the scene from a natural light source,
- Reflections from objects in the scene from a controlled source

Scene energy generated by objects or reflected from natural external sources is **Passive** energy. Scene energy that is a reflection of a controlled source is **Active** energy. There are two forms of scene energy: mechanical and electromagnetic energy. Mechanical energy is energy that propagates through a physical medium such as water or air; examples of systems that measure this energy are sonar and acoustical sensors. Electromagnetic energy is energy that does not require a physical medium and can travel through empty space; examples of systems that measure this energy are visible light cameras, infrared cameras, Synthetic Aperture Radar (SAR) sensors and LIDAR sensors.

Mechanical and electromagnetic energy both propagate the energy using travelling waves. A travelling wave of energy from a scene is represented using Equation 1.

	$\chi(x, t) = A \sin(kx - \omega t + \phi)$	Equation 1
--	---	------------

Where:

$\chi(x, t)$	Energy at a given distance (x) and time (t) from the scene,
A	Amplitude of the energy from the scene,
k	Wave number. The wave number is related to the energy's wave-length, λ by $k = 2\pi/\lambda$,
ω	Angular frequency. The angular frequency is related to the energy's frequency, ν , by $\omega = 2\pi\nu$,
ϕ	Initial phase of the radiator energy.

Equation 1 is similar to Equation 7.5 from [2] but with the addition of the initial phase. This is added to support active energy imagers, which can compute phase changes.

In most cases the scene energy measured for Motion Imagery is electromagnetic, which is discussed in Section 3.1.1. Mechanical energy systems, such as acoustical or sonar, are not documented in the MISP at this time.

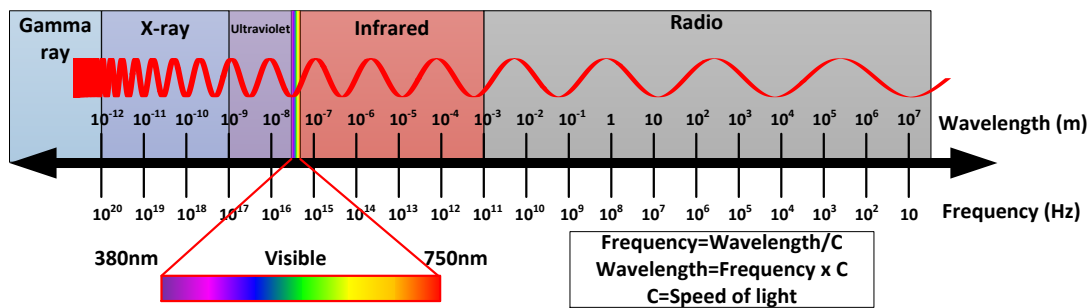
3.1.1 Electromagnetic Scene Energy

Electromagnetic energy, or Electro-Magnetic Radiation (EMR), consists of photons travelling at the speed of light (c). Each photon carries an amount of energy (Q_e), that is related to the frequency of the radiation by $Q_e = h\nu$, where h is Planck's constant. Since EMR waves travel at the speed of light, the frequency (ν) and wavelength (λ) are directly related by $c = \nu\lambda$. EMR waves have an orientation orthogonal to direction of propagation, called polarization, which can also be measured. Table 2 identifies the characteristics of EMR waves (from Equation 1) that can be measured by various sensing devices.

Table 2: Measureable EMR Properties

Name	Symbol	Description
Frequency	ν	Number of oscillations per second. With EMR when the frequency is known the wavelength can be computed.
Wavelength	λ	Distance from the start to the end of one wave cycle. With EMR when the wavelength is known the frequency can be computed.
Photon Energy	Q_e	Energy which relates to the number of photons for a given frequency range.
Phase	ϕ	The angular difference in a wave's starting point from a given reference wave.
Polarization	θ	The orientation of the EMR wave orthogonal to the direction of propagation.

The range of EMR frequencies and wavelengths is described using the Electromagnetic Spectrum as illustrated in Figure 10.

**Figure 10: Electromagnetic Spectrum**

Various ranges of wavelength within the electromagnetic spectrum are grouped into Spectrum Bands. A Spectrum Band is loosely bound by a lower and upper wavelength and given a name; for example, the Visible Band range extends from 380 nanometers (violet) to 750 nanometers (red). Table 3 lists popular Spectral Bands along with their wavelength and frequency values. The Ranges for each band are not well defined and the ranges can overlap between bands

Table 3: Electromagnetic Bands - The exact wavelength/frequency ranges are notional

Spectrum Band	Wavelength		Frequency	
	Start	End	Start	End
Gamma Ray		10 pm		30 EHz
X-Ray	10 pm	10 nm	30 EHz	30 PHz
Ultraviolet	10 nm	380 nm	30 PHz	789 THz
Visible	380 nm	750 nm	789 THz	400 THz
Violet	380 nm	450 nm	789 THz	666 THz
Blue	450 nm	495 nm	666 THz	606 THz
Green	495 nm	570 nm	606 THz	526 THz

Spectrum Band	Wavelength		Frequency	
	Start	End	Start	End
Yellow	570 nm	590 nm	526 THz	508 THz
Orange	590 nm	620 nm	508 THz	484 THz
Red	620 nm	750 nm	484 THz	400 THz
Infrared	750 nm	1 mm	400 THz	300 GHz
Near/Short Wave	750 nm	3 μm	300 GHz	100 THz
Mid-Wave	3 μm	8 μm	100 THz	37 THz
Long-Wave	8 μm	14 μm	37 THz	21 THz
Far-Infrared	14 μm	1 mm	21 THz	300 GHz
Radio	1 mm	100 Mm	300 GHz	3 Hz
Microwave	1 mm	1 m	300 GHz	300 MHz
mm	1 mm	7 mm	300 GHz	40 GHz
W	3 mm	4 mm	110 GHz	75 GHz
V	4 mm	7 mm	75 GHz	40 GHz
Ka	7 mm	12 mm	40 GHz	24 GHz
K	12 mm	17 mm	24 GHz	18 GHz
Ku	17 mm	25 mm	18 GHz	12 GHz
X	25 mm	37 mm	12 GHz	8 GHz
C	37 mm	75 mm	8 GHz	4 GHz
S	75 mm	150 mm	4 GHz	2 GHz
L	150 mm	300 mm	2 GHz	1 GHz
UHF	300 mm	999 mm	1 GHz	300 MHz
VHF	999 mm	10 m	300 MHz	30 MHz
HF	10 m	100 Mm	30 MHz	3 Hz

For a detailed overview of EMR see Chapter 7.1 of [2].

3.2 Energy Adjustments

Energy Adjustments are changes to the scene energy before it is measured. Energy Adjustments can affect any of the EMR properties listed in Table 2.

Waves travel through different mediums as they propagate from the scene to the imager. A Medium is a substance with a specific density. Energy propagates through different mediums such as water, air or glass at different rates. In classical EMR theory, electromagnetic waves can travel through mediums or through a vacuum, which is the standard reference medium for EMR.

There are two types of Energy Adjustments: Absorption and Transitional. Absorption adjustments occur when scene energy is consumed by the medium that the energy is propagating through; for example, color filters absorb certain wavelengths of EMR. Transitional adjustments occur when scene energy in one medium (source) interacts with another medium (destination).

As shown in Figure 11, three changes can occur to wave energy when interacting with the destination medium: Reflection, Refraction and Diffraction.

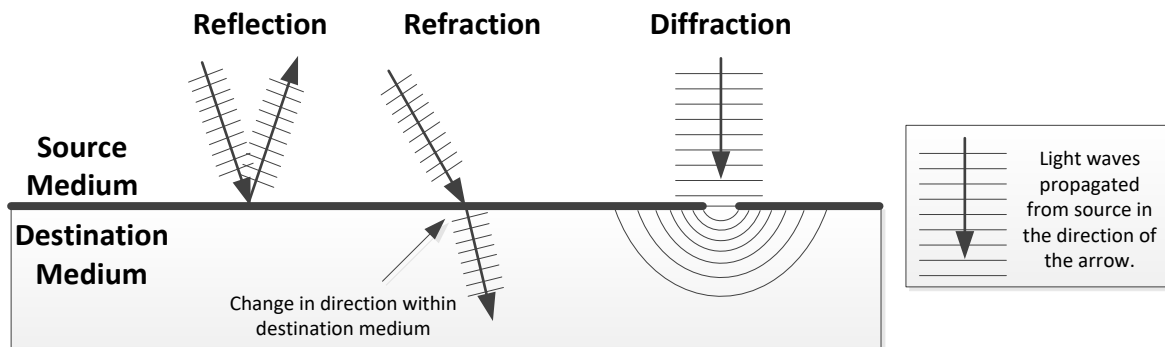


Figure 11: Illustration of Reflection, Refraction and Diffraction

Reflection occurs when a wave's propagation reverses direction away from the destination medium (i.e., bounces backward) and the wave's energy does not enter the destination medium. Refraction occurs when a wave's direction is altered as it enters the destination medium. Diffraction occurs when a wave "bends around" an object or interacts with a slit that is comparable in size to the wavelength. One or more of the three Energy Adjustments can occur simultaneously depending on the medium, the strength of the energy, and the wavelength of the energy.

In the MISP, two classes of Energy Adjustments are considered: uncontrolled and controlled. Uncontrolled energy adjustments are caused by environmental factors and are not directly measurable; however, some uncontrolled energy adjustments can be modelled, estimated and corrected in downstream processing. Controlled energy adjustments are deliberately imposed on the scene energy to enable or improve the energy measurements, for example a lens or a filter.

3.2.1 Uncontrolled Energy Adjustments

As scene energy travels to an imager, it passes through one or more mediums, all of which can distort the energy arriving at the imager in unknown ways. When the imager receives energy there may be additional non-scene energy, changed scene energy, or energy absent from the scene, as shown in in Figure 12.

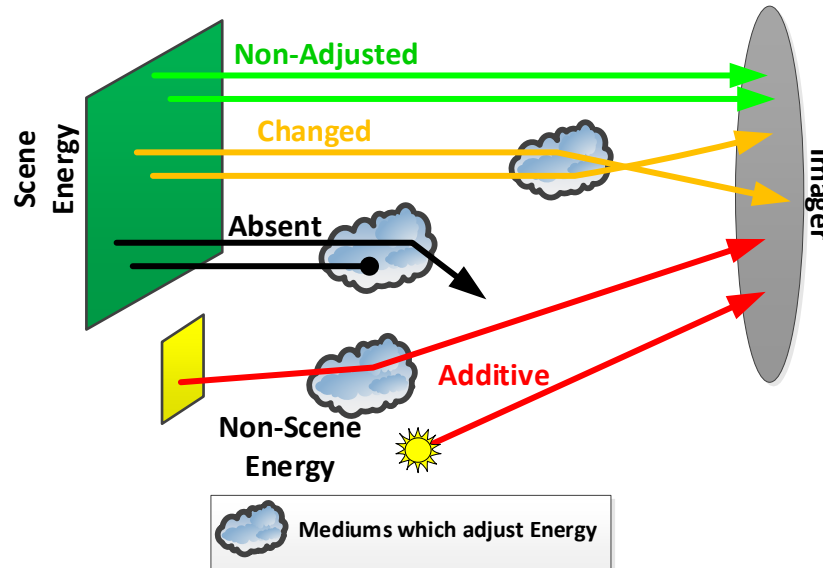


Figure 12: Uncontrolled Adjustments

Additive energy comes from any non-scene source such as refractions and reflections of energy not contributed by the scene. Systems consider additive energy as noise, which can be random, with a pattern or both. An example of added noise is the backscatter of light from particles in the air (e.g., fog). Figure 12 illustrates Additive energy with red lines.

Energy may be absent when the scene energy is either directed away from the sensor (refracted or reflected), or the medium the energy is passing absorbs the energy. For example, the atmosphere absorbs energy of light differently depending on the wavelength composition. Transmittance is the quantity of energy (e.g., light) passing through the atmosphere. Figure 13 shows the transmittance of energy through the atmosphere compared to the wavelength of the energy. The figure shows the atmosphere completely absorbs wavelengths of 5.5 through 7.5 microns. Figure 12 shows scene energy which is Absent at the Imager as black lines.

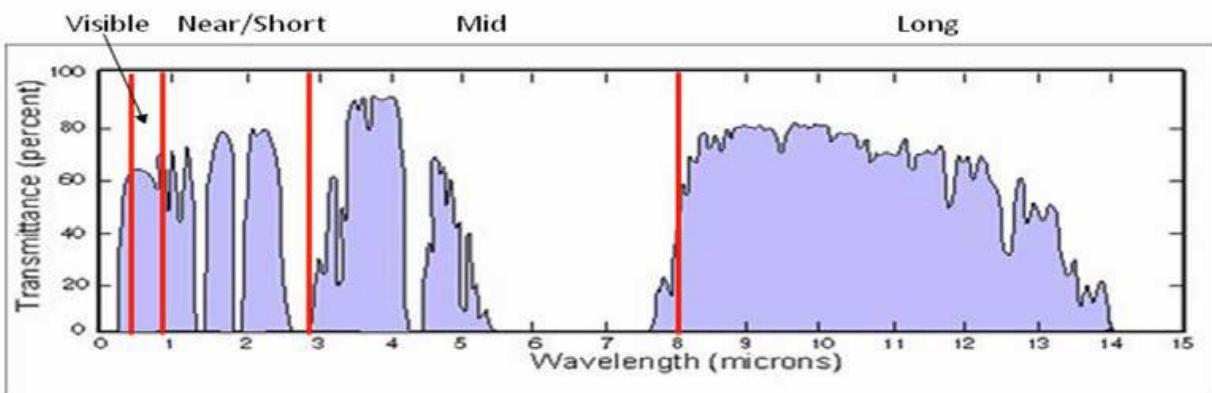


Figure 13: Transmittance of Energy through the Atmosphere

Changed energy comes from refractions and diffraction of the scene energy. For example, energy passing through the atmosphere refracts and diffracts, resulting in atmospheric distortions. The changes in the density of air causes Atmospheric distortions, which refracts the energy in various

directions. Atmospheric distortions cause a set of parallel waves of energy from the scene to become divergent or convergent. Atmospheric distortions result in defocused (blurry) imagery. In Motion Imagery, atmospheric distortions will cause apparent movement or distortion in each successive image of a static scene. Figure 12 illustrates Changed energy as orange lines.

Uncontrolled energy adjustments do not include distortions from imager technology such as lenses, mirrors and glass; these are Controlled Energy Adjustments and are discussed in Section 3.2.2.

Systems can correct for uncontrolled energy when given sufficient information about the environment, and knowledge of the imaging system. Motion Imagery metadata can include such information about the environment and imaging system. Currently the MISB does not define this data.

3.2.2 Controlled Energy Adjustments

Controlled energy adjustments are deliberately imposed energy adjustments on the scene energy to enable various types of energy measurements. Controlled energy adjustments include the application of reflection, refraction, diffraction and absorption of energy using devices such as mirrors, lenses, diffraction gratings and filters, respectively. Since the devices' behavior is known, metadata can be collected to assist in characterizing the collected data and correcting any adverse effects from such energy adjustments.

Typically, a multitude of controlled energy adjustments are combined either serially or in parallel, providing a wide range of functionality which includes focusing, zoom, color imagery, hyper-spectral imagery and others.

Chapter 4 of [2] provides a detailed description of controlled energy adjustments (e.g., lenses, mirrors and filters) used by imager systems.

3.3 Sensing Process

The Sensing Process converts Adjusted Energy into a set of digital Raw Measurements over a specified time period. In the MISB Image Processing Model, the Sensing Process requires a device that accepts scene energy and produces digital Raw Measurements of some aspect of the energy. These aspects include Intensity, Frequency, Phase and Propagation Time.

Intensity is the amount of Energy incident on a given area for a given time period. Intensity measurements range from counting a single photon (e.g., Geiger Mode LIDAR) to determining the total amount of energy captured over a detector's area during a defined period of time. Filters and prisms provide the means to measure the Intensity of different wavelengths at the same time, which can be further processed into color, multi- and hyperspectral imagery.

Energy Frequency, Phase and Propagation Time Raw Measurements measure energy information to/from the scene; this is used by LIDAR and RADAR systems. Imagers that measure Frequency, Phase and Propagation Time are not discussed in the MISB at this time.

Imagers are constructed from one or more individual sensing elements that collect energy collaboratively. There are many configurations and types of sensing elements but they all measure energy, which can be indirectly equated to counting photons.

3.3.1 Single Element Detection

Sensing intensity is achieved by using electro-mechanical devices, called **detectors**, which collect energy (photons) over a period of time and report the results as a digital value called a sample (see Section 1.1.2). As discussed in [3], detectors are divided into two classes: Thermal and Photon (or Quantum). Thermal detectors absorb EMR producing a change in the temperature of the detector relating to the intensity of the source EMR. At this time, Thermal detectors are not a subject within the MISP.

Quantum detectors interact with the incoming EMR resulting in changes to four possible electrical characteristics of the given detector material: EMR can be converted directly into an electrical charge, EMR can produce a photocurrent, EMR can cause a change in resistance (photoconductivity), or EMR can generate a voltage across a junction (photovoltaic).

Detector: Electro-mechanical device which senses EMR intensity.

Generally, all detectors include two primary stages: exposure and readout. **Exposure** accounts for the detector interacting with the EMR, where the device is either thermally or quantumly changed. **Exposure Duration** is the period of time spent in this stage. After the exposure Duration, additional EMR is prevented from being included in the measurement until the start of the next Exposure Duration. The detector is “shuttered” during the non-exposure time. Historically, a shutter was a physical device that blocked light from exposing film. With digital sensors, a shutter can be either a physical device or an electronic means of preventing EMR from affecting a detector’s measurement.

Exposure: When EMR is interacting with a detector for the purposes of measuring changes in the characteristics of the detector.

Exposure Start Time: The Start Time of exposure for a detector.

Exposure End Time: The End Time of exposure for a detector.

Exposure Duration: The time period when the detector is exposed to EMR measured as the difference between Exposure End Time and Exposure Start Time.

Readout is the process where the changes in the detector are converted into electrical signals and transferred out of the detector as a data value. The period of time for this process is called the Readout Duration.

Readout: When the changes of a detector (after exposure) are converted to either analog or digital values.

Readout Start Time: The Start Time of readout for a detector.

Readout End Time: The End Time of readout for a detector.

Readout Duration: The time period when readout is occurring measured as the difference between Readout End Time and Readout Start Time.

In addition to exposure and readout other processes for clearing charges, shutter drains and other detector “maintenance” are performed. Between the exposure End Time and the Readout Start Time other operations may be performed that clear the detector and other electronic components. Figure 14 illustrates an example of timing for a single detector. First, there is a Clear operation, which is followed by an exposure (with Exposure Start Time and Exposure End Time noted),

then a Clear-Register operation, which is followed by a readout operation (with Readout Start Time and Readout End Time noted). These four steps complete one operating cycle of the detector. This cycle then repeats. The number of cycles, or exposures, per second is called the Exposure Rate. Image frame rate is equal to the Exposure Rate when temporal processing is not performed in later steps (i.e., Image Creation and Image Processing) of the Imager Processing Model.

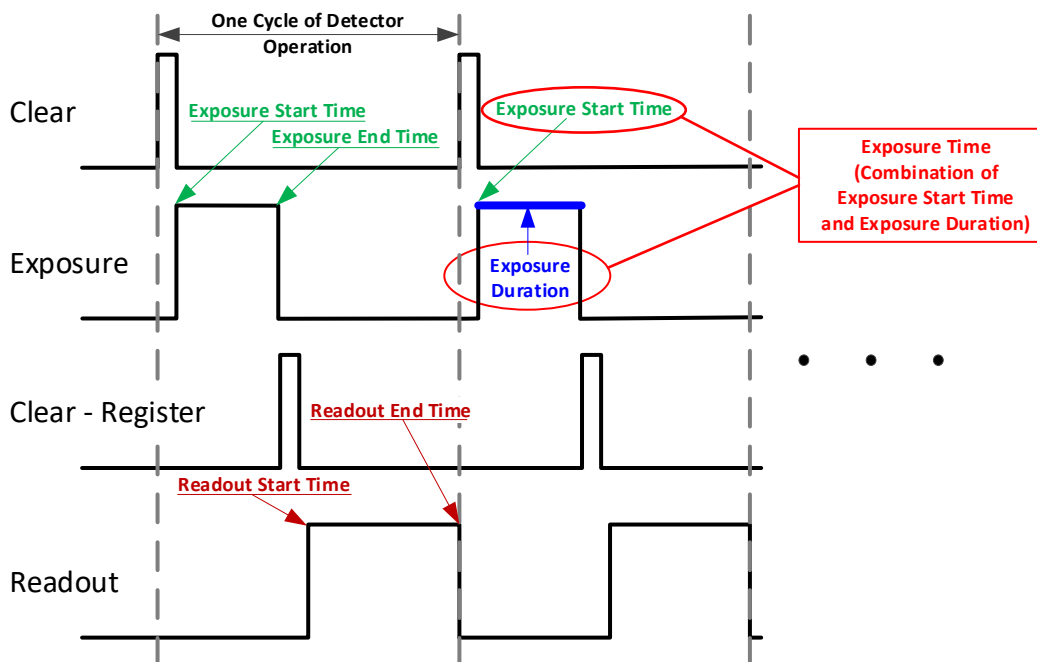


Figure 14: Illustration of Timing for a Single Detector

Exposure Rate: The number of exposures processed by a detector in one second.

In this handbook, Exposure Time is comprised of two measurements: Exposure Start Time and Exposure Duration, which is an offset to the Exposure Start Time. Equivalently the Exposure Time can be defined as the Exposure Start Time and Exposure End Time. By providing both the Exposure Start Time and the Exposure Duration (or Exposure End Time), any time within the Exposure Duration can be computed as a reference time for the image.

Exposure Time: The pairwise combination of the Exposure Start Time and the Exposure Duration.

Exposure Time (alternate): The pairwise combination of the Exposure Start Time and the Exposure End Time.

Detectors have a physical shape or surface where they gather EMR. The detector shape is typically rectangular or square, but other shapes, such as hexagons, are sometimes used for compact detector spacing (see Detector Groups in Section 3.3.2).

Along with a detector's measurements, related information – if determined and recorded – provides great value; these include Exposure Time, the surface area of the detector, and the physical location of the detector. In the MISP and MISB-related standards, the Exposure Time is

specified by the Precision Time Stamp or the Nano Precision Time Stamp (see Section 6.2.3), and the physical location of the detector is included in the photogrammetric metadata.

3.3.1.1 Physical Description of Single Detector and Supporting Infrastructure

Single detectors are devices that collect photons over some surface area of the device. To support photon collection, the surface area may be partially covered with non-photon collecting supporting material; this is shown in Figure 15, where the photon sensitive area covers a portion of the total detector surface.

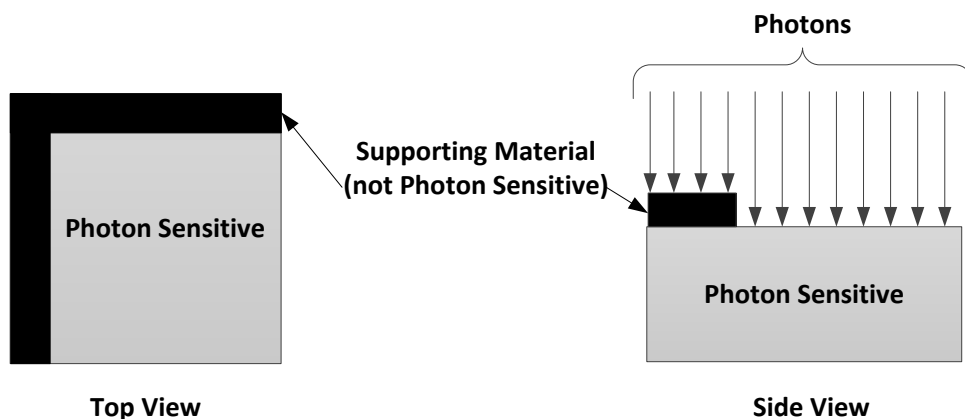


Figure 15: Illustration of a detector showing photon sensitive and non-sensitive areas

A side view of the detector shows some incoming photons blocked by the devices supporting material. There are several methods for dealing with this issue. One method is to ignore the loss of photons and attempt to keep the ratio of photosensitive-to-support area as high as possible during design and manufacturing. Other methods include using lenses (see Section 3.2.2) or other devices to direct the light above the detector onto the photon-sensitive area, as shown in Figure 16. The lens focuses the light only onto the photon sensitive portion of the detector. An array of these lenses, used with a group of detectors, is called a lenslets or microlens array.

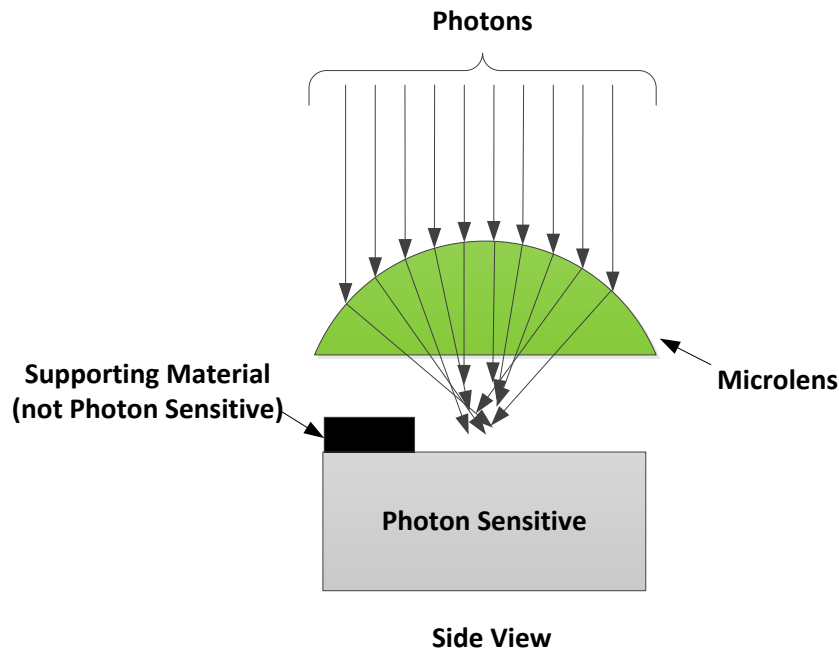


Figure 16: Illustration of a Detector with a lens to focus most of the incoming photons into the Photon Sensitive area

Some detector systems include a filter intended to pass only photons meeting a specific criterion. For color imagers, filters allow photons with certain wavelengths to pass through to the detector. For polarimetric imagers, filters allow only photons with certain wave orientations to pass through to the detector. When detectors are used in a Detector Group (Section 3.3.2), the filter types may not be the same for each detector. For example, two adjacent detectors may have two different color filters, such as a blue and green color filter. There are many different filter patterns used within Detector Groups (Section 3.3.2) based on manufacturer and purpose. Two examples of filter patterns are the Bayer pattern, used to detect red, green and blue EMR, and Polarimetric filters, used to detect different polarization orientations of EMR. Figure 17 illustrates a blue filter, which prevents all colors except blue from reaching the detector. The filter can be above or below the micro lens.

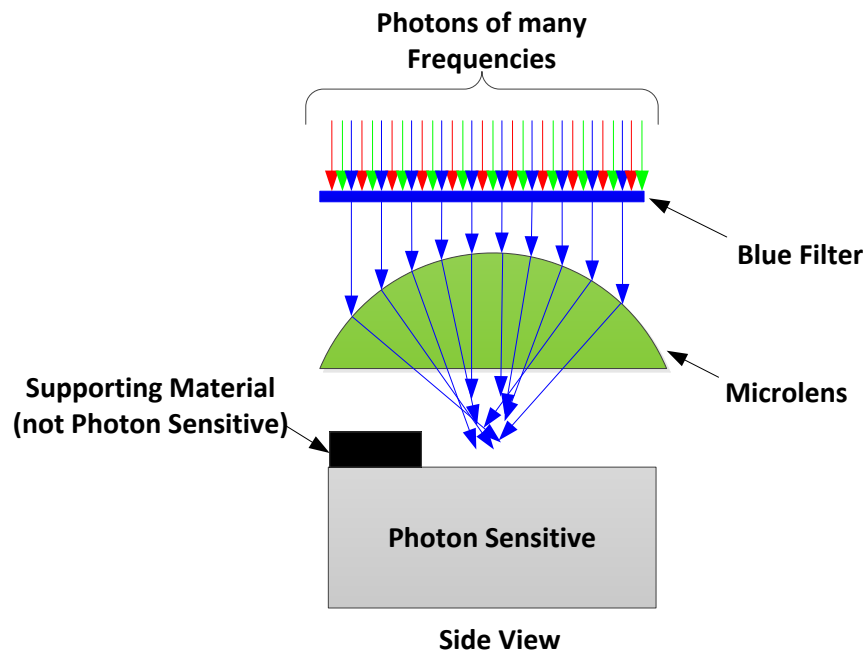


Figure 17: Illustration of blue filter over a single detector

3.3.1.2 Noise

Several types of noise may affect the output of a detector, including shot noise, dark current noise and circuit noise. Shot noise is attributed to the statistical nature of gathering EMR within a detector. Dark current noise (aka thermal noise) is caused by temperature variations within the detector over time. Circuit noise comes from supporting electronics used to amplify and measure the photons within the detector. For more information on noise sources see Chapter 7 of [2].

3.3.2 Detector Groups

Detectors are configured and operated together in Detector Groups. A **Detector Group** combines multiple individual detectors spatially, temporally or both, producing a set of Raw Measurements (i.e., samples) used to create an image. A Detector Group can have any physical configuration or geometric shape. The most primitive configuration of a Detector Group is a single line (linear) of two or more detectors. A more common Detector Group configuration is a rectangular array of detectors, where the rows and columns of detectors may lie in a grid pattern, offset grid pattern or other patterns. Figure 18 illustrates linear, regular grid, offset grid and hexagonal detector patterns.

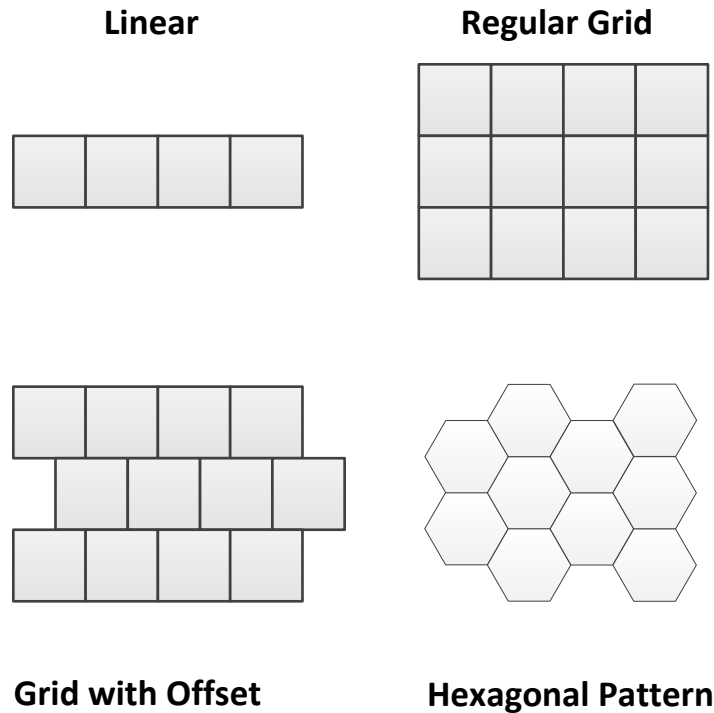


Figure 18: Detector Group Patterns

Both the physical layout of the Detector Group and the area of EMR being imaged by each detector are important metadata for photogrammetry. With most layout patterns, the center location of each detector can be computed if the pattern, spacing and detector sizes are known.

Detector Group: A collection of detectors operating together with a physical layout (detector count and dimension, detector pattern, and pitch (distance) between detectors).

The exposure and readout of each detector (Section 3.3.1) in a Detector Group may occur at different times for a single set of raw measurements from the Detector Group. Examples include: (1) exposure of all detectors simultaneously within the Detector Group (i.e., Global Shutter), or (2) exposure of one line of a Detector Group at a time sequentially through the image (i.e., Rolling Shutter), or (3) random exposure of each detector in the Detector Group. In all cases, the Exposure Time(s) are important metadata for Motion Imagery sensors to report; this information is called the **Exposure Configuration**. An Exposure Configuration is a list of all Exposure Times for a single set of samples collected from all detectors in a Detector Group. The Exposure Configuration may change for each set of samples measured depending on adjustments needed for proper exposure (i.e., auto exposure adjustments).

Exposure Configuration: A list of all Exposure Times for a single set of raw measurements from a detector group.

Specifying the Exposure Configuration for every detector produces a large quantity of metadata. For some Exposure Configurations, the quantity of metadata can be reduced. Identifying the constant values and repetitions within an Exposure Configuration can help to minimize this metadata.

To organize the exposure and readout of a Detector Group, each Detector Group is divided into Regions, where each Region is composed of Sub-Regions. Figure 19 illustrates a Detector Group (the composite rectangle) with two Regions (A and B), where each Region consists of a number of Sub-Regions (detectors common to a Sub-Region are shaded similarly).

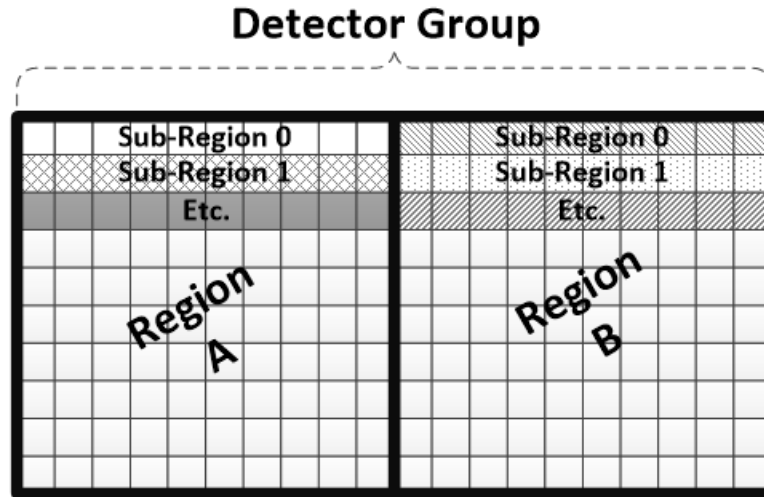


Figure 19: Illustration of Detector Group, Region and Sub-Region

3.3.2.1 Regions

A **Region** is created by dividing a Detector Group into rectangular Sub-Regions that share the same exposure orientation. Orientation is the direction across the Region that two or more Sub-Regions are exposed.

Region: A collection of Sub-Regions with the same exposure orientation.

Possible Region readout orientations are shown in Figure 20; these include Top-Down, Bottom-Up, Left-Right and Right-Left. In this illustration each Region consists of five Sub-Regions, where each row or column is a Sub-Region. In the Top-Down case, for example, each Sub-Region beginning with the top row then downward as numbered is exposed and read out sequentially.

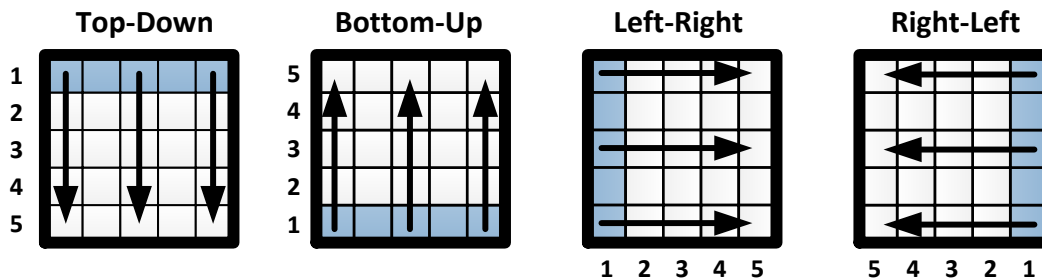


Figure 20: Region Readout Orientations across Sub-Regions

The different orientations of Figure 20 can be found within the same Detector Group. For example, Figure 21 (left figure) shows a Left-Right Region (Region 1) together with a Right-Left

Region (Region 2) organized as an Outward-In configuration, and a Right-Left Region (Region 1) together with a Left-Right Region (Region 2) in an Inward-Out configuration (right figure).

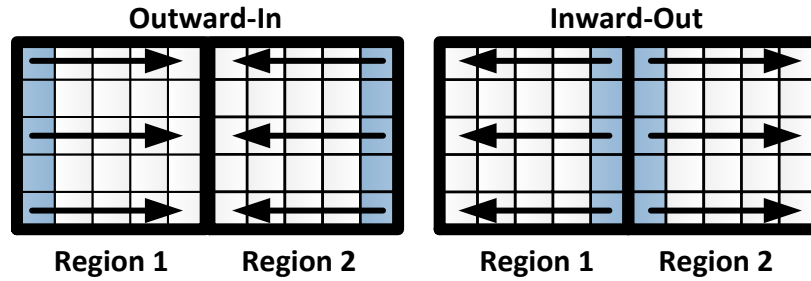


Figure 21: Different Region Orientations in the same Detector Group

A Sub-Region area is the spatial size – in rows and columns of detectors – that each Sub-Region occupies within a Region. In Figure 20 the area of the Sub-Region in the Top-Down example is a one by five (i.e., one row by five columns of detectors).

3.3.2.2 Sub-Regions

A **Sub-Region** is a line (vertical or horizontal) of detectors that operate with a similar Exposure Start Time and an identical Exposure Duration. The Exposure Start Time for all detectors in a Sub-Region has a linear relationship with the Exposure Start Time of the first detector.

Let $\tilde{s}_i$ denote the Exposure Start Time for the i^{th} detector in a Sub-Region, where $\tilde{s}_0$ denotes the Exposure Start Time for the first detector. Also, let Δ indicate a linear offset from the Exposure Start Time of the first detector in the Sub-Region. Then, the Exposure Start Time for successive detectors in the Sub-Region is computed as $\tilde{s}_i = (i * \Delta) + \tilde{s}_0$. When all detectors in a Sub-Region have the same Exposure Start Time (i.e., the offset $\Delta=0$) the Sub-Region is called *Homogeneous*.

Now, let δ_e denote the Exposure Duration, and E_i the Exposure End Time. If the Exposure Duration is constant for all detectors in a Sub-Region, the Exposure End Time for a given detector is $E_i = \tilde{s}_i + \delta_e$ (see Figure 22 for an example).

Sub-Region: A collection of detectors in which each detector has similar Exposure Start Time and the same Exposure Duration. The Exposure Start Time of a given detector in the Sub-Region can be computed as a linear offset from the first detector of the Sub-Region.

3.3.2.3 Exposure Configuration Examples

The following two examples illustrate how Regions and Sub-Regions are used to define the timing of a Detector Group.

3.3.2.3.1 Exposure Configuration Example 1

In Figure 22 a basic rolling shutter Exposure Configuration is shown. The Detector Group is represented as one Region with Sub-Region's 0 through N. Each Sub-Region contains M detectors within a row. The Exposure Start Time for the j^{th} Sub-Region is denoted as S_j . Within each Sub-Region the Detector Start Time is the same; thus, the Sub-Regions are homogeneous (i.e., $\Delta = 0$) and $S_j = \tilde{s}_{j,0} = \tilde{s}_{j,1} = \tilde{s}_{j,2} \dots$ etc., where $\tilde{s}_{j,i}$ is the i^{th} detector in the j^{th} Sub-Region.

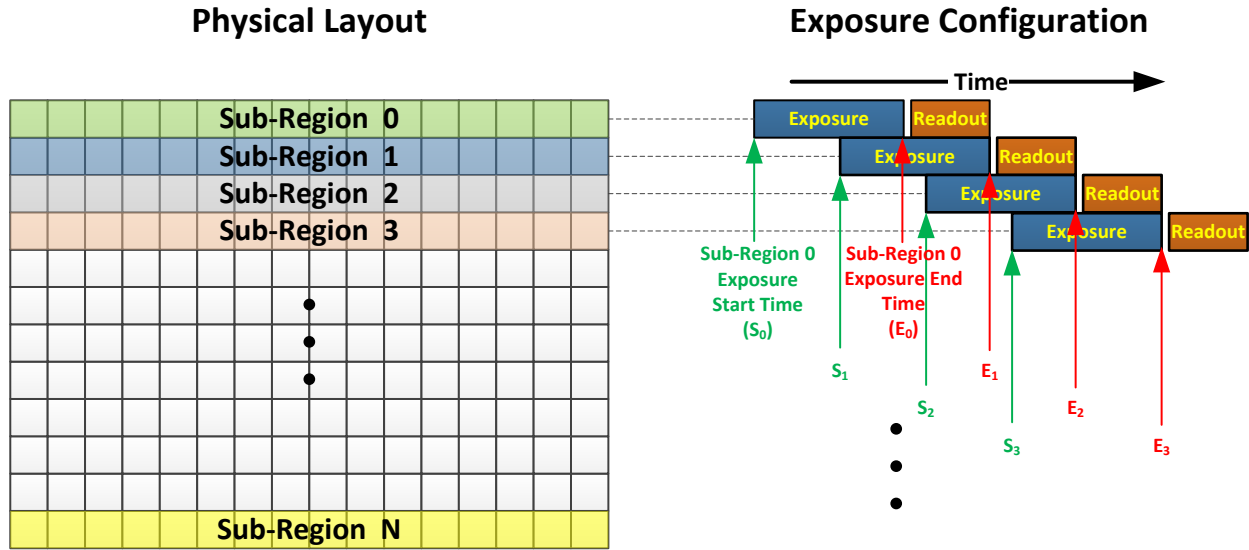


Figure 22: Illustration of a Detector Group with N+1 Sub-Region and a Rolling Shutter Exposure Configuration

In Figure 22 the exposure and readout process begin with the exposure of the detectors in Sub-Region 0 (Exposure Start Time = S_0) followed by the readout of the samples from Sub-Region 0. A short time after S_0 , there is exposure of the detectors in Sub-Region 1 (Exposure Start Time = S_1) followed by its readout, which begins after the readout of Sub-Region 0 is completed. This pattern continues for the entire Detector Group. For simplicity, other operations for clearing the detector or registers are not shown. This process continues producing N+1 Raw Measurements (one for each row of detectors) to form a complete image. The resulting Exposure Configuration for Figure 22 is listed in Table 4.

Table 4: Exposure Configuration for Figure 22

Sub-Region Number	Exposure Start Time	Exposure End Time	From		To	
			Row	Column	Row	Column
0	S_0	E_0	0	0	0	M
1	S_1	E_1	1	0	1	M
2	S_2	E_2	2	0	2	M
3	S_3	E_3	3	0	3	M
Etc.						
N	S_N	E_N	N	0	N	M

Using the information from Table 4, the Exposure Time (Exposure Start Time and Exposure Duration) for each detector can be determined. The total time to read out the Detector Group is then $T_{DG} = E_N - S_0$.

If the example is adjusted for non-homogeneous Sub-Regions, then an additional offset value Δ must be accounted for in each Sub-Region.

3.3.2.3.2 Exposure Configuration Example 2

Example 2 demonstrates a more complicated Exposure Configuration, where the Detector Group employs a layout of Regions with different orientations and Exposure's. The Regions are physically separate but can perform readout operations at the same time as other Regions. Figure 23 illustrates a Detector Group with sixteen 3x4 detector Regions, each with the orientation shown by the arrows. Each Region is composed of 3x1 detector Sub-Regions (the first Sub-Region is shaded). Each arrow color represents a Region with the same timing. In this example Regions 1A, 1B, 1C and 1D are all exposed with the same timing. Each Sub-Region is homogeneous.

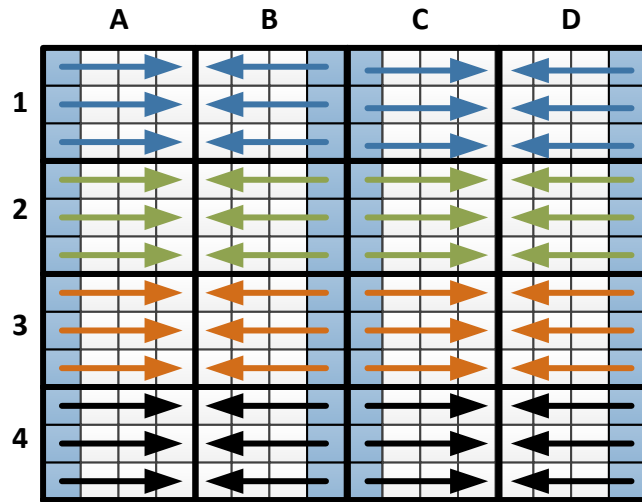


Figure 23: Illustration of Regions in a Detector Group used to define an Exposure Configuration

To determine the Exposure Configuration for a complete Detector Group, each Region needs to have its orientation, area and timing defined for every Sub-Region as shown in Table 5. The Exposure Start Time and Exposure End Time are denoted as $S_{R,j}$, $E_{R,j}$ where R is the Region identifier (i.e., 1A, 1B, ...) and j is the jth Sub-Region within Region R. In this example, all Sub-Regions that start with 1 operate at the same time in parallel, so their Exposure Start Time is based on S_{1A} , likewise for Regions that start with 2, etc.

Table 5: Exposure Pattern for Figure 23

Region	Sub-Region	Exposure		Orientation	From		To	
		Start Time	End Time		Row	Column	Row	Column
1A	0	$S_{1A,0}$	$E_{1A,0}$	→	0	0	2	0
1A	1	$S_{1A,1}$	$E_{1A,1}$	→	0	1	2	1
1A	2	$S_{1A,2}$	$E_{1A,2}$	→	0	2	2	2
1A	3	$S_{1A,3}$	$E_{1A,3}$	→	0	3	2	3
1B	0	$S_{1A,0}$	$E_{1A,0}$	←	0	7	2	7
1B	1	$S_{1A,1}$	$E_{1A,1}$	←	0	6	2	6
1B	2	$S_{1A,2}$	$E_{1A,2}$	←	0	5	2	5
1B	3	$S_{1A,3}$	$E_{1A,3}$	←	0	4	2	4
Etc. for Regions 1C & 1D								
2A	0	$S_{2A,0}$	$E_{2A,0}$	→	3	0	5	0
2A	1	$S_{2A,1}$	$E_{2A,1}$	→	3	1	5	1
2A	2	$S_{2A,2}$	$E_{2A,2}$	→	3	2	5	2
2A	3	$S_{2A,3}$	$E_{2A,3}$	→	3	3	5	3
2B	1	$S_{2A,0}$	$E_{2A,0}$	←	3	7	5	7
2B	1	$S_{2A,1}$	$E_{2A,1}$	←	3	6	5	6
2B	2	$S_{2A,2}$	$E_{2A,2}$	←	3	5	5	5
2B	3	$S_{2A,3}$	$E_{2A,3}$	←	3	4	5	4
Etc. for remaining Regions								

This table defines the timing and readout orientation of every Sub-Region. The total time to read out the Detector Group is then $T_{DG} = E_{4A,3} - S_{1A,0}$.

3.3.2.4 Exposure Patterns

As indicated earlier, the quantity of metadata to describe an Exposure Configuration can be large. An Exposure Pattern captures the constant values and repetitions within an Exposure Configuration to help minimize this metadata. A global Exposure Pattern includes both the *Inter-Region* patterns, which are the layout of Regions of the Detector Group, and the *Intra-Region* patterns, which are the timing within the Region (i.e., Sub-Region timing).

Exposure Pattern: A constant or repetitive method of exposure in an Exposure Configuration.

3.3.2.4.1 Inter-Region Exposure Patterns

To describe an Exposure Configuration for any Detector Group, an Exposure Pattern is needed for each Region. As discussed in Section 3.3.2.1, a Region is defined by its position, area and orientation. The position and area are specified using two opposite (diagonal) corners of the Region. Let the first corner -- the Region Starting Corner be denoted D_0 , and the second corner -- the Region Ending Corner be denoted D_3 . D_0 is the location of the first detector of the first Sub-

Region, while D_3 is the last detector of the last Sub-Region in the Region. To establish the orientation, the first detector of the second Sub-Region is specified -- denoted D_2 . As a convenience for the Intra-Region timing, a fourth detector is specified that indicates the last detector of the first Sub-Region -- denoted D_1 .

Figure 24 illustrates the Exposure Pattern of the Detector Group from Figure 23. Each Region in this example is a single vertical line of three detectors.

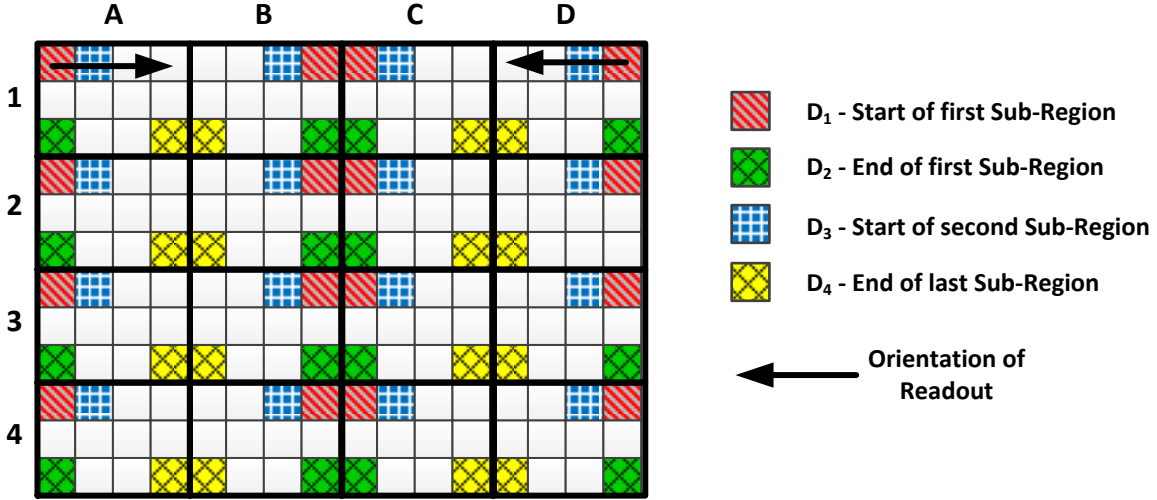


Figure 24: Example Exposure Pattern

As shown in Figure 24, once all of a Detector Group Region's patterns are defined, the start and end of each Region can be determined. To complete the Exposure Configuration, the timing information associated with each Sub-Region is defined as described in Section 3.3.2.4.2.

3.3.2.4.2 Intra-Region Exposure Pattern

Intra-Region Exposure Patterns are defined using the starting and ending timing of specific detectors within Sub-Regions. The notation used to specify starting or ending time of a specific detector is $S_{R,j,i}$ or $E_{R,j,i}$ respectively for the R^{th} Region, j^{th} Sub-Region and i^{th} detector within the Sub-Region.

Within a single *homogeneous* Region, an Exposure Pattern is defined that describes all of the Sub-Region timing information using only three values: the first Sub-Region's Exposure Start Time and Exposure End Time ($S_{R,0,0}$, $E_{R,0,0}$) and the Exposure Start Time of the second Sub-Region ($S_{R,1,0}$). These values are used to compute two values, the Exposure Duration (λ) and Sub-Region Delay (γ), which are assumed constant for the entire Sub-Region. This assumption may reduce the accuracy of the timing, but the overall effect is minor. As defined in Section 3.3.1, the Exposure Duration is the difference between the Exposure End Time and Exposure Start Time; that is $\lambda = E_{R,0,0} - S_{R,0,0}$. The Sub-Region delay is the time between the Exposure Start Time of each Sub-Region, $\gamma = S_{R,1,0} - S_{R,0,0}$.

Within a Region R , knowing $S_{R,0,0}$, $E_{R,0,0}$, $S_{R,1,0}$ and the number of Sub-Regions, the complete Region's Exposure Configuration can be computed. Each $S_{R,j,i}$ can be computed using $S_{R,j,i} = S_{R,0,0} + j * \gamma$, and $E_{R,j,i}$ can be computed by adding the duration (λ) to $S_{R,j,i}$.

To illustrate the reduction of data possible using a Configuration Pattern for a Region's Exposure Configuration, Table 6 shows how each Exposure Start Time and Exposure End Time from Table 4 are computed. For these time values, Table 4 would produce $2*N$ values to report the Exposure Start Time and Exposure End Time for this example, while Table 6 only produces three values (i.e., $S_{R,0,0}$, $E_{R,0,0}$, $S_{R,1,0}$) to define all the times.

Table 6: Exposure Metadata for Figure 23 using Exposure Pattern ($S_{R,0,0}$, $E_{R,0,0}$, and $S_{R,1,0}$)

Sub-Region Number	Exposure Start Time	Exposure End Time
0	$S_{R,0,0}$	$E_{R,0,0} = S_{R,0,0} + \lambda$
1	$S_{R,1,i} = S_{R,0,0} + 1 * \gamma$	$E_{R,1,i} = S_{R,1,i} + \lambda$
2	$S_{R,2,i} = S_{R,0,0} + 2 * \gamma$	$E_{R,2,i} = S_{R,2,i} + \lambda$
3	$S_{R,3,i} = S_{R,0,0} + 3 * \gamma$	$E_{R,3,i} = S_{R,3,i} + \lambda$
Etc.		
N	$S_{R,N,i} = S_{R,0,0} + N * \gamma$	$E_{R,N,i} = S_{R,N,i} + \lambda$

When the Sub-Regions are not homogeneous an additional value is needed to determine the offset (Δ) between the Exposure Start Time for each detector within the Sub-Region; this value is assumed to be constant for all detectors within a Sub-Region. The offset can be computed by reporting any Exposure Start Time after the first Exposure Start Time within the first Sub-Region. The chosen value to use is the last Exposure Start Time of the first Sub-Region ($S_{R,1,M}$); with this value the offset is $\Delta = \frac{S_{R,0,M} - S_{R,0,0}}{M+1}$. With a homogeneous Sub-Region $S_{R,0,M} = S_{R,0,0}$ so $\Delta = 0$. For non-homogeneous Sub-Regions, the equations for the Exposure Start Time in Table 6 are updated to $S_{R,j,i} = S_{R,0,0} + j * \gamma + i * \Delta$.

3.3.2.4.3 Complete Exposure Pattern

Combining the information from both the Inter and Intra Exposure Patterns a Detector Group's Exposure Pattern can be identified. Table 7 lists the information needed to define the timing for each Region, R, in a Detector Group. The first column lists the value identifiers defined in Sections 3.3.2.4.1 and 3.3.2.4.2; the second column defines the values that are used; the third column is the count of values. The byte size is dependent on the method of transferring the values.

Table 7: Single Region Exposure Pattern

Value Identifiers	Values	Value Count
$D_{R,0}$, $S_{R,0,0}$, $E_{R,0,0}$	row, col, time offset, time offset	4
$D_{R,1}$, $S_{R,1,M}$	row, col, time offset	3
$D_{R,2}$, $S_{R,1,0}$	row, col, time offset	3
$D_{R,3}$	row, col	2
Total		12

To reduce the number of values (and bandwidth) there are situations where some of the values can be eliminated:

1. When all Sub-Regions within the Region share the same Exposure Start Time, then $(D_{R,2}, S_{R,2})$ is not required. If this value is omitted then $S_{R,2}$ is equal to $S_{R,0}$.
2. When all detectors within the same Sub-Region share the same Exposure Start Time, then $(D_{R,1}, S_{R,1})$ is not required. If this value is omitted then $S_{R,1}$ is equal to $S_{R,0}$.

3.3.2.5 Exposure Pattern Types

There are many different Exposure Patterns that can be defined with the techniques defined in Section 3.3.2.4. There are four types of Exposure Patterns: Global Shutter, Type 1 Rolling Shutter, Type 2 Rolling Shutter and Multi-Group. The first three are more common than the last.

Table 8 lists the Exposure Pattern Types and further details are provided in the following sections. In some cases, the rules from Section 3.3.2.4.3 are applied to minimize the timing data needed for describing the Exposure Pattern.

Table 8: Typical Detector Group Sensor Types

Type	Description (Section)	Value Identifiers	Value Count
Global Shutter	3.3.2.5.1	$(D_{R,0}, S_{R,0,0}, E_{R,0,0})$ $(D_{R,3})$	6
Type 1 Rolling Shutter	3.3.2.5.2	$(D_{R,0}, S_{R,0,0}, E_{R,0,0})$ $(D_{R,2}, S_{R,1,0})$ $(D_{R,3})$	9
Type 2 Rolling Shutter	3.3.2.5.3	$(D_{R,0}, S_{R,0,0}, E_{R,0,0})$ $(D_{R,1}, S_{R,0,M})$ $(D_{R,2}, S_{R,1,0})$ $(D_{R,3})$	12
Multi-Region	3.3.2.5.4	see below	see below

3.3.2.5.1 Global Shutter

With a Global Shutter sensor, all detectors within the Detector Group are exposed simultaneously, i.e., all detectors in the Detector Group use the same Exposure Time. The only information needed to describe a Global Shutter sensor is the Exposure Start Time and Exposure End Time for one detector, which in turn describes the timing for the whole group. With Global Shutter sensors the complete image is "frozen" in time (provided the exposure time is short enough such that there is no change in the scene during the exposure time). The advantage of a Global Shutter sensor is superior motion capture capability.

3.3.2.5.2 Type 1 Rolling Shutter

A Type 1 Rolling Shutter sensor has one Region for the whole Detector Group, and all detectors within each Sub-Region share the same Exposure Start Time. In a Type 1 Rolling Shutter sensor,

each Sub-Region of the image is measured separately, thus introducing image effects. These image effects are discussed in Section 3.7.2.

3.3.2.5.3 Type 2 Rolling Shutter

A Type 2 Rolling Shutter sensor has one Region for the Detector Group, but each detector in within a Sub-Region does not share the same Exposure Start Time. In a Type 2 Rolling Shutter sensor, each row (or column) of the image is measured separately, thus introducing image effects. These image effects are discussed in Section 3.7.2.

3.3.2.5.4 Multi-Region Shutter

A Multi-Region Shutter sensor can expose different areas of the sensor at different times and in different orientations. Multi-Region shutter sensors will have image effects which are similar to Rolling Shutter effects but may be more chaotic in nature.

3.3.2.5.4.1 Interlaced

An Interlaced Detector Group is a Multi-Region Shutter system that has $N/2$ Regions with each Region containing only two Sub-Regions (or rows); where N is equal to the number of rows in the Detector Group. Each Region shares the same timing. The first Sub-Region is exposed, and then the second Sub-Region is exposed a short time later. This is equivalent to the odd rows of the Detector Group being exposed simultaneously and the even rows are exposed after the odd rows have been exposed. This disjoint capture in time of image data introduces image effects. These image effects are discussed in Section 3.7.3.

3.3.2.6 Detector Group Readout and Stabilization

Some detector systems can perform stabilization within the sensing unit by moving the area sampled to compensate for small movements of the Detector Group. The area of the Detector Group sampled is called a Detector Window. Figure 25 illustrates the effect of the stabilization, and the samples measured from the Detector Group.

In Figure 25 illustration (1), a set of samples is read from the Detector Group in the red area defined from detector (2, 2) to (9, 12). In the subsequent illustration, the sensor, and therefore Detector Group, moves (because of some disturbance), so the stabilization algorithm compensates to maintain the image on the same area of the scene; this causes the Detector Window to shift. In illustration (2), a set of samples is read from the Detector Group in the Detector Window defined from detector (1, 4) to (8, 14). In the last illustration (3), another movement causes a shift, and a set of samples is read from the Detector Group in the Detector Window defined from detector (3, 1) to (10, 11). In these three illustrations, the Detector Window does not change size; only the location within the Detector Group changes. However, it is possible for a Detector Window to change size if necessary.

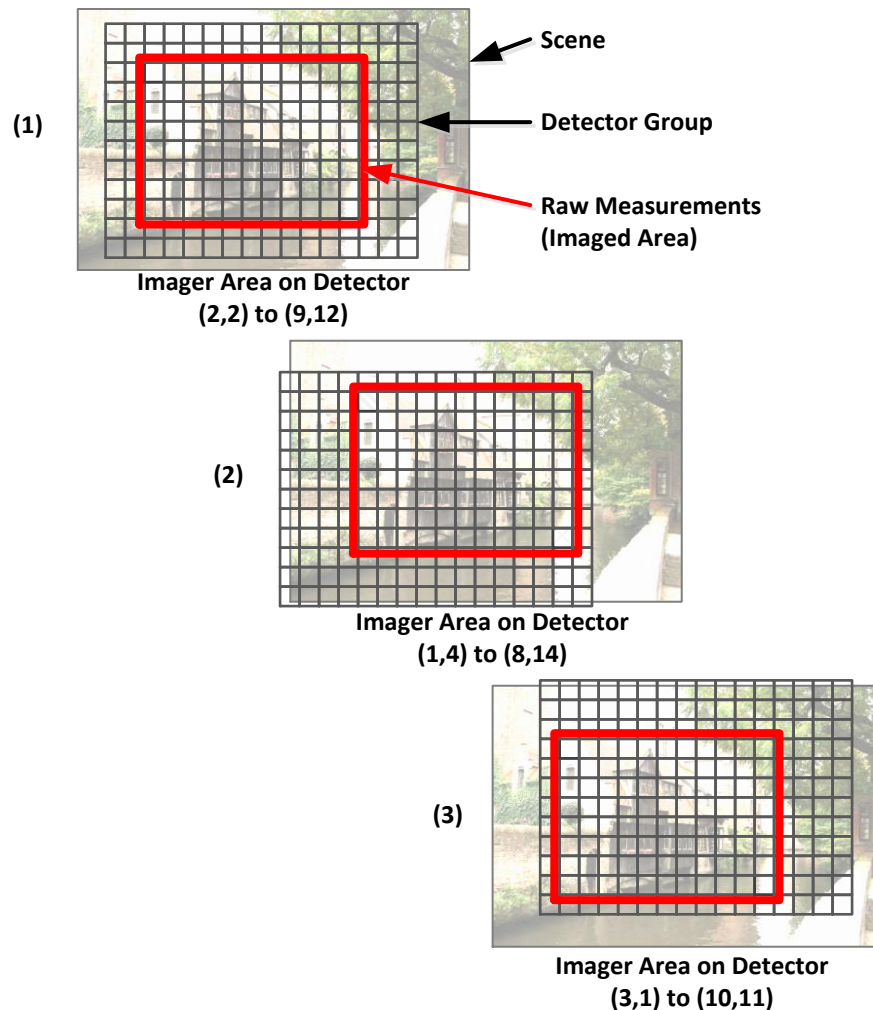


Figure 25: Stabilization of Raw Measurements and Detector Group movement
(Bruges Belgium 2014 Lat: N51 12'18" Lon: E3 13'48")

3.3.2.6.1 Detector Window

An image sampled from a Detector Group can be geometrically smaller than the Detector Group for various reasons, such as dimensional requirements and the stabilization process. Although Detector Groups can be constructed to any density, standards specify dimensional requirements for practical reasons. For example, a Detector Group might contain 2048x2048 detectors, but the application may require a 1920x1080 image; in this case, a match of detectors to image – or window of detectors – is needed. Window information thus helps to define the two bounding corner detectors used in a windowed process.

Additionally, the Detector Group and the image will have different coordinate systems and orientation. The sensor can have any orientation relative to the scene, for example the light from the scene may be projected onto the Detector Group upside down or as a mirror image. Creating the image from the Detector Group so that it is right side up requires the Window corners to have any orientation.

The necessary window information consists of two detector corner coordinates C_1 and C_2 , with each coordinate specifying a row and column from the Detector Group as listed in Table 9 and

illustrated in Figure 26. The corners can define any area within the Detector Group and can have any orientation. The resulting image is mapped from the Detector Window with C_1 being the upper right corner of the image and C_2 being the lower left corner of the image.

Table 9: Window Corner Values

Corner	Values	Description
C_1	row, column	First corner of the Detector Window. This corner becomes the upper left corner of the image.
C_2	row, column	Second corner of the Detector Window, which is diagonally opposite from the first. This corner becomes the lower right corner of the image.

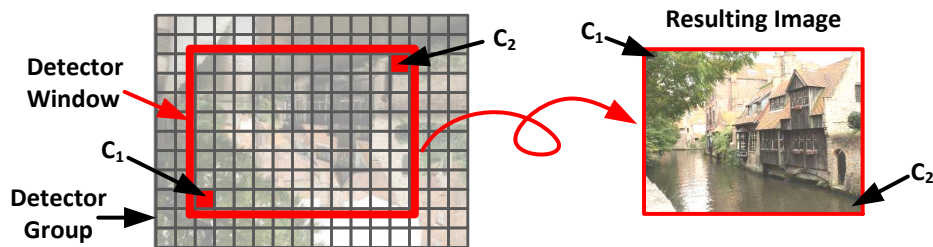


Figure 26: A Detector Window within a Detector Group (defined by C_1 and C_2) and the resulting image which is a rotation and flip (mirror image) of the projected light onto the Detector Group (Bruges Belgium 2014 Lat: N51 12'18" Lon: E3 13'48")

3.3.3 Sensor Configurations

There are a wide variety of sensors, lenses, prisms, and filter combinations, which enable sensing a variety of phenomenology types; examples include multi-spectral, hyper-spectral imagery and plenoptic systems. With multi-spectral and hyper-spectral imagery, prisms separate different bands of light, which sensors can detect at the same time. Plenoptic systems can use lenslets to send light to groups of individual detectors at the same time. The resulting imagery is lower in resolution but can be post processed to enable post capture focusing on objects at different locations in the scene.

3.3.4 Other Sensing Topics

There are many methods used to improve the resulting detected image, including auto-exposure and binning. Auto-exposure is a process built into the sensor to adjust the exposure time based on the amount of energy being detected. Auto exposure changes the detectors timing without external controls to ensure that an unsaturated image is detected. Binning is a process where multiple detectors values are combined during readout to reduce noise and improve the signal-to-noise ratio.

3.4 Raw Measurements

The result of the Sensing Process is one or more Raw Measurement datasets. Each Raw Measurement dataset is the detector data recorded for a single exposure time for some Detector Subgroup. The Raw Measurements datasets contain a measured value and location for each

detector in the subgroup. The Detector Subgroup's shape may not be a regular gridded shape, so the Raw Measurement rows and columns may not represent adjacent data (e.g., hexagonal Detector Group). Alternatively, depending on the shape of the Detector Subgroup, the data position in an array represents the physical location.

3.5 Image Creation Process

The Image Creation Process converts Raw Measurements into a set of regularly spaced homogeneous samples in the shape of a rectangle. During this processing detector values may be averaged or removed either spatially, temporally or both. Data from different exposure times may be adjacent to each other in the final result. The end result of this process is an image as defined in Section 1.1.3.

For color imagery, a common step during the Image Creation Processing is computing the color bands from the detector data's Bayer or color pattern. Other processing that can occur during the image creation process includes algorithms for digital zoom and stabilization.

3.6 Image Processing

The final step before an image leaves a camera is Image Processing. Image Processing is performed in-part or on the whole image; for example, color smoothing/correction or averaging multiple temporal images into a single image to match a desired temporal rate. The result of the Image Processing step is an image as defined in Section 1.1.3.

3.7 Image

The output of the Image Processing Model is a series of images. These images may show undesirable effects depending on the detector settings and the intended application, such as Shutter to Scene, Rolling Shutter, and Interlaced effects.

3.7.1 Shutter to Scene

When designing Motion Imagery Systems, the expected action within the scene and the frame rate of the imager needs to be considered to prevent *temporal aliasing*. Temporal aliasing occurs when the frame rate is slower than the motion in a scene; for example, the movement of a propeller may look like it is rotating slowly while in fact it is rotating very fast. This optical illusion is called the wagon-wheel effect and may cause users to incorrectly interpret what is happening within the scene.

3.7.2 Rolling Shutter

In a rolling shutter imager, as defined in Section 3.3.2.5.2 and shown in Figure 22, the rows of detectors are not exposed at the same time. For example, in a "top down" rolling shutter imager the first row is exposed, then a short time later the second row, then the third, etc. until the whole Detector Group's lines are exposed and readout. The Exposure Times for all of the detectors in one row are the same, and the time difference between rows is constant; however, the Exposure Duration for each row can change resulting in partial exposure effects (see Section 3.7.2.2).

The advantage of the rolling shutter is the imager can continue to gather energy during the acquisition process, thus increasing sensitivity. They are also low cost as compared to other types of sensors. The disadvantages of rolling shutter include distortion of fast-moving objects (see Section 3.7.2.1) and partial exposure effects. Most rolling shutter technology is found in the consumer market i.e., cell phones. Since the exposure process moves *through* the image over some length of time, users should keep in mind the following issues when working with rolling shutter sensors.

3.7.2.1 Motion Blur and Wobble

Motion blur may occur depending on the speed of motion within the scene or motion of the whole scene. The motion blur can severely distort the imagery to a point where objects no longer appear natural and uniform. For example, the blades of a fan may take on irregular shape (see Figure 27). When the imager is subject to vibration, the image may appear to wobble unnaturally, sometimes called the “jello-effect.”

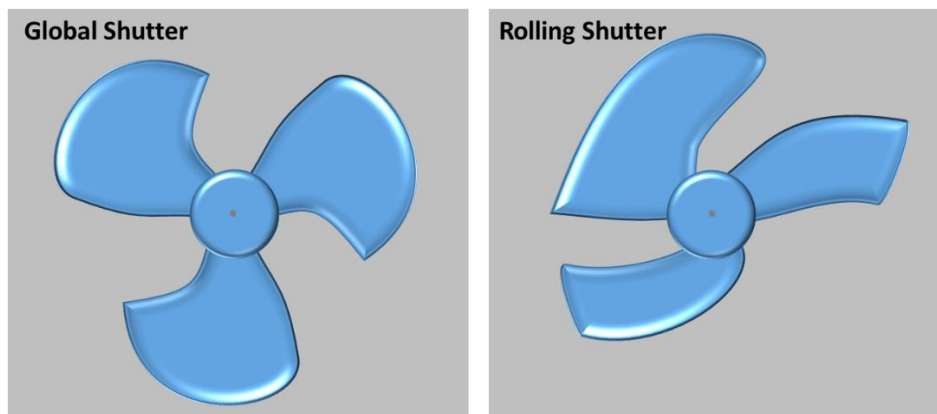


Figure 27: Example Motion Effects: Global vs. Rolling Shutter

When panning a rolling shutter imager, objects in the scene may appear to lean away from the direction of motion as illustrated in Figure 28.

Original**Rolling Shutter**

Figure 28: Illustration (simulated) of a rolling shutter image as the imager pans quickly across the scene (Copyright (C) 2014 Her Majesty the Queen in Right of Canada as represented by the Minister of National Defence. All rights reserved.)

3.7.2.2 Strobe Light and Partial Exposure

A rolling shutter imager is not well-suited for capturing short-pulse light sources, such as strobe light or flash. Unless the light source remains on for the duration of exposure, there is no guarantee of adequately capturing the light source. This will result in an image with varying levels of illumination across the scene. As this effect manifests differently in successive images of a Motion Imagery sequence, the imagery may appear to “breathe” with some content possibly washed out completely.

3.7.3 Interlaced

In an interlaced scan imager, the image is “imaged” in two passes: the odd-numbered rows of the image are captured during the first pass, and then the even-numbered rows in the next pass. Thus, two complete passes (or scans) are required to capture a complete image. One main drawback of interlaced scanning is that images tend to flicker, and motion – especially vertical motion – appears jerky. Another drawback is image detail, such as object edges can be “torn” demonstrating a stair-step “jagged” effect along an object edge, as shown in Figure 29. As the motion increases stair-stepping can become quite pronounced greatly distorting image features essential to exploitation tasks. This distortion is further compounded when compressing an image. Because the stair-stepping artifact introduces higher frequency detail, coding efficiency is reduced as the coder attempts to spend its allocated bits representing these artifacts. With higher compression ratios, these artifacts are even further degraded.

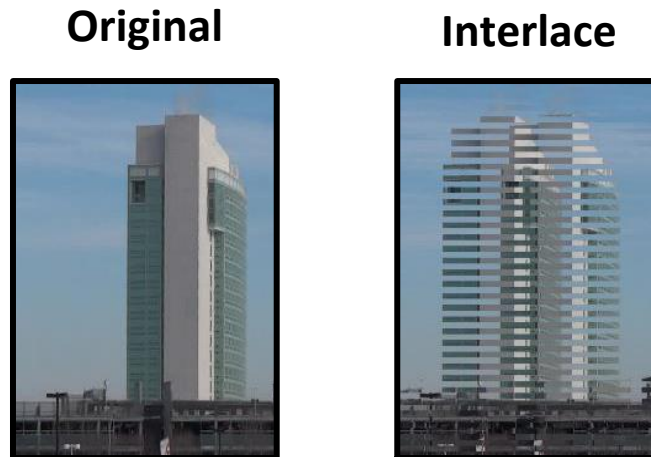


Figure 29: Illustration (simulated) of an interlaced image as the imager pans quickly across the scene *(Copyright (C) 2014 Her Majesty the Queen in Right of Canada as represented by the Minister of National Defence. All rights reserved.)*

Interlaced scan is an older technology developed to deliver analog television within limited bandwidth criteria. Because of its time in the marketplace, it is inexpensive, which makes it attractive. However, it is a poor choice for surveillance applications, where poor edge definition and compression greatly reduces motion fidelity.

Chapter 4

Image Color Model

Color images are generally represented using three bands comprised of a number of samples per band interpreted as coordinates in some color space. A color space is a mathematical representation of a set of colors. Several popular color spaces include RGB (Red-Green-Blue), Y'U'V' and Y'C_bC_r' (where Y' represents the luma, (brightness information), and U'V' or C_bC_r' (the chroma or color difference information). The “prime” notation indicates that the values represent the gamma-corrected versions of the corresponding signals. Gamma-correction describes the total of all transfer function manipulations, such as corrections for any nonlinearities in the capture process (see SMPTE EG 28 [4] for definitions).

Color spaces, such as Y'U'V' and Y'C_bC_r', which are derived as linear combinations of the R'G'B' values, are efficient representations to express color (e.g., the color difference signals require less bandwidth than luma or the primary R'G'B' signals). As such, the color difference signals can be represented using fewer samples. Nomenclatures of 4:4:4, 4:2:2 and 4:2:0 denote spatial sampling of the color bands. The graphic in Figure 30 helps to explain color sampling for these common encodings.

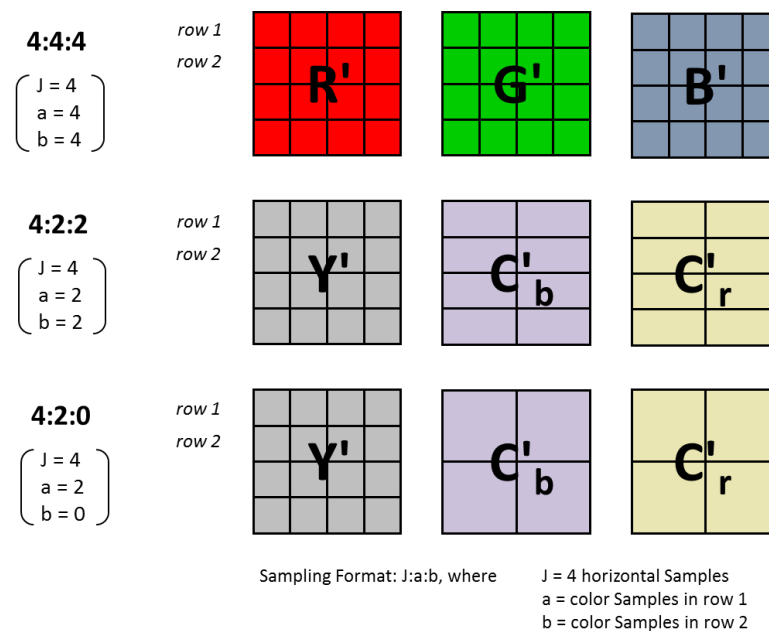


Figure 30: Examples of Formats with Chroma Subsampling

At the top of the figure, a set of 4x4 sample arrays represent three color bands, one each for Red, Green and Blue. Likewise, the middle and bottom show three sample arrays that represent the color: one band of luma (Y') and two bands of chroma (C_bC_r'). A sampling ratio with notation J:a:b is used: “J” is the dimension of the array horizontally, in this case J = 4; “a” indicates the number of samples in row 1, and “b” the number of samples in row 2.

For example, in 4:4:4 (top of Figure 30), each of the three bands have the same spatial sampling; that is, each band (R'G'B' in the example) has a sample that represents primary color information in each pixel location. A 4:4:4 model contains the maximum number of samples, which is 48 samples ($16Y' + 16C'_b + 16C'_r$).

In 4:2:2 (middle of Figure 30), every two samples in row 1 share a chroma sample ($a=2$); likewise, every two samples row 2 share a chroma sample ($b=2$). For 4:2:2 when forming a pixel, a single chroma sample is reused by two pixels (the pixel's row-wise neighbor); this reduces the number of samples by one-third to 32 samples ($16Y' + 8C'_b + 8C'_r$).

In 4:2:0 (bottom of Figure 30), every two samples in row 1 share a chroma sample ($a=2$); row 2 shares its chroma sample with the top row. For 4:2:0 when forming a pixel, a single chroma sample is reused by four pixels (the pixel's row-wise and column-wise neighbors); this reduces the number of samples by one-half to 24 samples ($16Y' + 4C'_b + 4C'_r$).

Often a pixel, such as “24-bit color pixel” or “16-bit color pixel”, describes a 3-band set of sample values. Determining the pixel value for three bands where each has the same spatial sampling is straightforward, i.e., pixel value range = $3B$, where B = bits per sample for one band. In the case of color sampling, an *equivalent* pixel value range can be computed in reference to the pixel arrangement shown in Figure 30. Note that in the color sampling for 4:2:2 and 4:2:0 the chroma bands have fewer samples than the luma band.

In Table 10, the pixel value range in bits per pixel for the three-color samplings are listed for several sample value ranges of 8, 10 and 12 bits per sample. The pixel value range is based on the number of possibly unique samples within the sample array. For instance, 4:4:4 has equal sample spacing in each band, so there is one sample in each band, i.e., full sample density. In 4:2:2 for every sample in band 1 there are 0.5 samples in band 2 and 0.5 samples in band 3. Likewise, in 4:2:0 for every sample in band 1 there are 0.25 samples in band 2 and 0.25 samples in band 3. Together the samples across bands represent one pixel. The pixel value range is then computed by multiplying the average number of samples per pixel by the sample value range.

Table 10: Pixel Value Range for Various Color Sampling Formats

Color Sampling Format	3-Band color (Average Samples/Band)			Average Samples/ Pixel	Sample Value Range (bits/Sample)		
	Band 1	Band 2	Band 3		8	10	12
					Pixel Value Range (bits/Pixel)		
4:4:4	1	1	1	3	24	30	36
4:2:2	1	0.5	0.5	2	16	20	24
4:2:0	1	0.25	0.25	1.5	12	15	18

Chapter 5

Dissemination

Motion Imagery Data is often produced some distance away from where it is controlled and/or exploited. The action of transmitting Motion Imagery Data from a source (i.e., imager, platform or control station) to one or more users is called **dissemination**. Transmitting Motion Imagery Data can affect end users in two ways: Quality and Latency. Motion Imagery quality is impacted by the compression applied to the Motion Imagery and data losses during transmission. Similarly, metadata can also be impacted by data losses.

Latency is a measure of amount of the time it takes to move data from one point to another in a Motion Imagery System. Latency is impacted by the compression of the Motion Imagery and the transmission path taken. Total Latency is the elapsed time from an occurrence in the scene to when that occurrence is viewed in the Motion Imagery at its destination. When Total Latency is significant, a platform pilot may not be able to accurately control the imager(s), and an end user may not be able to coordinate with other users or Intel sources in real time. Therefore, minimizing Total Latency is an overarching design goal, especially for systems used for real time applications. There is always a balance between quality and latency as both are difficult to optimize at one time.

While the subject of transmission can be extensive, in this section common methods endorsed by the MISP for dissemination for Motion Imagery Data are discussed.

5.1 Background

Although the MISP does not levy requirements on the transmission of Motion Imagery Data, the MISP does levy requirements on the quality of Motion Imagery. The transmission of Motion Imagery can impact the quality of the Motion Imagery; understanding some basic methods for transmission is beneficial.

Systems may transmit data by breaking the data into a sequential series of data groups and transmitting each data group in order. Each data group is a **data packet** (or just packet), and the sequential series of packets is a **stream**. Packet definition (i.e., what goes in a packet) is based on their context (e.g., an MPEG TS packet or UDP packet) and packets themselves are decomposable to smaller packets for different purposes. The concept of a stream is also decomposable into substreams, for example an MPEG Transport Stream may contain a video substream and multiple metadata substreams; each of these substreams is further decomposable into more substreams/packets, and so on. A stream of data implies a series of data packets in a specific order. A data type (e.g., MPEG TS, KLV) or protocol (e.g., UDP, TCP/IP) will define the logical contents of a packet.

Packet: A logical grouping of data for a specific data type or protocol.

Stream: A series of packets in a defined order.

The health of a delivery path from imager through exploitation depends on many factors and begins with the method of transmission.

5.1.1 Transmission Methods

There are three transmission methods typically used in MISP applications: Wireless, Wired and Mixed Use.

5.1.1.1 Wireless

Wireless transmission generally assumes a radio link, such as from an airborne platform to a ground station. Although wireless technologies are designed to support varied applications and have different performance criteria, they are susceptible to interference from other communications signals. Interference introduces distortion into the transmitted signal, which can cause data errors. Because errors in wireless transmission are anticipated, methods to detect and repair errors often are provided; for example, Forward Error Correction is one popular method used in a digital link. Such processes add additional overhead to the data transmitted, and they are limited to correcting certain types of errors.

5.1.1.2 Wired

Wired transmission can be divided into circuit-switched and packet-switched technologies. In circuit-switching a dedicated channel is established for the duration of the transmission; for example, a Serial Digital Interface (SDI) connection between a sensor and an encoder. Packet-switching, on the other hand, divides messages into packets and sends each packet individually with an accompanying destination address. Internet Protocol (IP) is based on packet-switching.

5.1.1.3 Mixed Use

In a network infrastructure, a mix of wireless and wired transmission methods is usually present. For example, Motion Imagery Data from an airborne platform might be transmitted wirelessly to a satellite, relayed from the satellite to a ground receiver, and then transmitted over a wired IP network. Each method of transmission has its own susceptibility to errors that must be understood by developers when implementing a Motion Imagery System, and by users who receive and use the data.

5.1.1.4 Bandwidth

Wired transmission, in general, offers greater bandwidth capacity than wireless; this has important implications in the dissemination of Motion Imagery Data. Because of the large data characteristics of Motion Imagery, compression is needed when delivering Motion Imagery over the more bandwidth-constrained wireless link. Compression and subsequent encoding increase the complexity of the data, which makes it susceptible to errors introduced in transmission.

5.1.2 Internet Protocols

Internet Protocols represent a family of protocols used in an Internet packet-switching network to transmit data from one system to another. Table 11 provides information about the Internet Protocol family.

Table 11: Internet Protocols

Protocol Name	Description
Internet Protocol (IP)	The principal communications protocol for relaying packets across networks. IP data packets (datagrams) are sent from a transmitting to a receiving system using switches and routers. IP is a low-level protocol that does not guarantee delivery, or the data will be correct (i.e., it could be corrupted) when it arrives at the destination.
User Data Protocol (UDP/IP)	UDP [5] uses a simple transport layer protocol based on IP. It does not guarantee data delivery or that data packets arrive in order. UDP specifies a network Port that enables multiple data sources from one system to be transmitted to multiple receiving systems. Data sent from one system to multiple systems is called multicasting. UDP provides one of the fastest methods of transmitting data to a receiver, which makes it suitable for time-sensitive applications (low latency). UDP multicasting is used in delivering Motion Imagery Data to multiple systems at once, which reduces overall network bandwidth.
Transmission Control Protocol (TCP/IP)	TCP [6] is a transport layer protocol providing reliable guaranteed delivery of data; however, TCP does not guarantee <u>time-sensitive</u> delivery of data. Systems use TCP when transferring non-time-sensitive data, such as Motion Imagery Data files.

When UDP/IP is used, there are several types of packet errors that can occur as shown in Table 12. These errors can affect any protocol or data that uses UDP/IP (i.e., RTP).

Table 12: UDP Error Types

Error Type	Description
Packet Loss	Packets can be lost in a number of ways, such as network routers/switches being overwhelmed, or network devices physically disconnected. When routers/switches are overwhelmed, they will discard packets, which are then forever lost to all downstream devices. Other causes of packet loss include poor wiring and faulty equipment; these can cause intermittent packet loss and be hard to detect.
Packet Corrupted	Packets can be corrupted during the transmission from one device to another. Corruption can be caused by faulty equipment, poor wiring or from interference. Interference is primarily an issue with wireless technologies, although crosstalk in wired technologies can also be problematic. When routers/switches receive a packet and UDP error checking determines that the packet is corrupted, the packet is dropped and lost to a receiving system (see Packet Loss). If a corrupted packet passes its UDP error check, the corrupted packet could go undetected unless further error detection methods are used.

Packet Out of Order	Networks usually contain more than one router/switch, and typically there is more than one path for transmitting an IP packet from a source to a destination. Packets that take different paths may arrive at a destination out of the order they were transmitted. This condition is not detectable by UDP error checks, so other means for detecting and possibly reordering the packets need to come from additional information supplied within the transmitted data.
---------------------	---

5.1.2.1 MPEG-2 TS Packets

The MPEG-2 Transport Stream (MPEG-2 TS [7]) is a widely used Container for disseminating Motion Imagery Data. For example, Motion Imagery Data transmitted from an airborne platform, as well as to points along a network that supports exploitation, is typically in a MPEG-2 TS Container. Developed originally for wireless transmission of television signals, MPEG-2 TS is organized as successive 188-byte data packets with each packet including a 4-bit continuity count. This count can be used to detect whether a packet is either missing or received out of order; however, because of the small size of the continuity counter it only detects a small percentage of the possible discontinuities. MPEG-2 TS is commonly used in delivering Motion Imagery Data over IP as well. The MISP has standardized how to insert MPEG-2 TS packets into UDP packets in MISB ST 1402 [8]. Table 13 describes the effects of UDP errors on the MPEG-2 TS.

Table 13: MPEG-2 TS Error Types

Error Type	Description
Packet Loss	There are several types of packet loss in MPEG-2 TS. The first occurs when one (or more) UDP packet(s) are discarded. Up to seven MPEG-2 TS packets can be encapsulated into one UDP packet. Loss of one UDP packet can mean the loss of up to seven MPEG-2 TS packets. Such a loss can be detrimental to the decompression of the Motion Imagery, and the effects range from a decoder that may stop working to intermittent losses of imagery. This significantly impacts exploitation. A second type of packet loss is more localized to an individual MPEG-2 TS packet. Here, the internal information within a packet may be incorrect; this could result from a malfunctioning system component, or corruption in transmission. A packet could be discarded by receiving equipment if the error is an ill-formed packet. Depending on the contents of a discarded packet the effect could be major (i.e., timing or important decode information) or minor (i.e., loss of a portion of the imagery). In both types of packet loss, when a packet contains metadata, the information is likely unrecoverable.
Packet Corrupted	A MPEG-2 TS packet may be corrupted by a system issue, such as encoder or Transport Stream multiplexer malfunction or information within a packet can become corrupted in transit. The packet may appear to be properly formed – and therefore not discarded, but the data contained inside is not meaningful. Issues like these are not easily diagnosed.
Packet Out of Order	As discussed in Table 12, out-of-order packets generally result from network device operation and varied network paths data may take. The 4-bit continuity count in each MPEG-2 TS packet provides a limited indication of packet sequence order;

	however, without information in advance on how many MPEG-2 TS packets are in a UDP packet it may be difficult to determine the actual MPEG-2 TS packet order.
--	---

5.1.2.2 Real Time Protocol (RTP)

RTP [9] is designed for end-to-end, real-time transfer of data. RTP was specifically designed for delivery of A/V (audio/video) services over IP. Each data type (i.e., Motion Imagery, metadata, and Audio) is delivered as an independent data stream. Relational timing information for synchronizing the individual data streams at a receiver is published in Real Time Control Protocol (RTCP) – a companion protocol. RTP is encapsulated in UDP/IP and includes a timestamp for synchronization and sequence numbers that aide in packet loss and reordered packet detection. RTP/RTCP is typically considered for bandwidth-constrained environments, where a choice among the supported data types can be made.

There are also advantages to encapsulating a MPEG-2 TS into RTP; in fact, this method is widely used by the commercial industry in long-haul delivery of video over IP. A receiver can use the RTP timestamp to measure inter-packet jitter to estimate the stress occurring in a network. Such information also indicates potential decoder buffer over/under flows, which could cause decoder failure. The RTP sequence number is 16-bits; much larger than the MPEG-2 TS 4-bit packet count, which enables a wider detection range for lost and reordered packets.

Delivery of Motion Imagery Data using RTP is subject to the errors similarly found in MPEG-2 TS (see Table 13). In RTP, however, data is organized into larger packets, which can be as large as the limits (called the Maximum Transmission Unit, or MTU) of UDP for a medium. A lost packet of RTP may have a greater effect than a lost packet of MPEG-2 TS as it contains more data; again, the impact to decompression and exploitation would depend on the data contained within the packet.

Chapter 6

Time Systems

6.1 Overview

Time is fundamental within Motion Imagery Systems (MIS): it coordinates subsystems and components internally, as well as for coordination among external systems; it also enables people to share and operate on data collaboratively. A **Temporal Event** is when time, in some form, is associated with an event. The action of assigning a time to an event is called **Timestamping**; the time value assigned is called a **Timestamp**. Measurements within a MIS while imaging a scene (i.e., “frame-time”), or actions within a scene being exploited by an analyst are examples of temporal events. Temporal events can be instantaneous or have finite duration.

Temporal Event: When time is associated with an event. The time can be a duration, defining a Start Time and End Time, or instantaneous, defining a Start Time (Note: Start Time equals End Time, in this case).

Timestamp: The time value(s) associated with a Temporal Event.

Timestamping: The action of assigning time to a Temporal Event.

Correlation of temporal events is critical for proper interpretation of activities within a scene. Analysis is enhanced when information and metadata across multiple systems are synchronized temporally. Although many systems for tracking time have been developed, the Motion Imagery Standard Profile defines its Time System based upon the international Time Systems.

6.2 Time System Elements

Timestamps depend on three elements of a Time System: **Clock**, **Epoch** and **Adjustment Criteria**.

Within this handbook, a **Clock** is a function that counts *units* of time, for example a second, minute, hour, etc. The Clock *value* is the number of units counted, which can include fractional units. The *clock-period* is the amount of time between two successive unit counts, measured in Standard International (SI) Seconds [10] (i.e., SI Second).

Depending on the type of Clock, the clock-period can be constant or vary. A Clock with a constant clock-period is called a *linear-clock*. A linear-clock with a clock-period equal to the SI Second is called a SI-clock. A *varying-clock* has a non-constant clock-period.

The **Epoch** is the reference point for the Clock, which includes the beginning date and time. Clock values represent a count of units since some defined Epoch.

Figure 31 illustrates a Clock using a number line. In the left diagram, each tick mark represents the Clock value as the count of seconds since the Epoch and the clock-period is indicated as the space between each tick mark. The middle and right diagram illustrates the difference between a linear-clock and varying-clock.

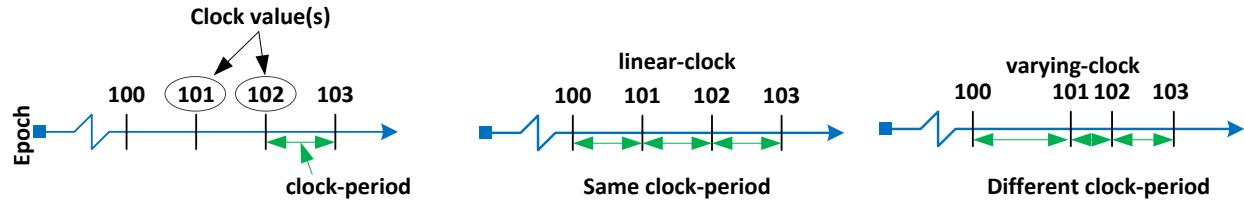


Figure 31: Illustration of Clocks

The **Adjustment Criteria** are processes or rules used to change a Time System's Clock value to match some given criteria. An example Adjustment Criteria is to change a Clock's value daily to match the time it takes for the earth to rotate (i.e., UT1 time system in Section 6.2.2.2).

The relationship between two Clocks can be described in terms of the Clock's clock-period and differences in when each clock increments its count. The smallest difference in time between one Clocks count incrementing, and a second Clocks count incrementing is called *delay*. Figure 32 illustrates the four primary cases of delay between two Clocks. Case (1) shows two linear-clocks with the same period, and the delay, d_0 , is constant. Case (2) shows two linear-clocks with different periods; in this case the two delays (d_1 and d_2) are not the same. Case (3) shows two delays (d_3 and d_4) –of many possible different delays – between a linear-clock and varying-clock. Case (4) shows two delays (d_5 and d_6) of many possible different delays between two varying-clocks.

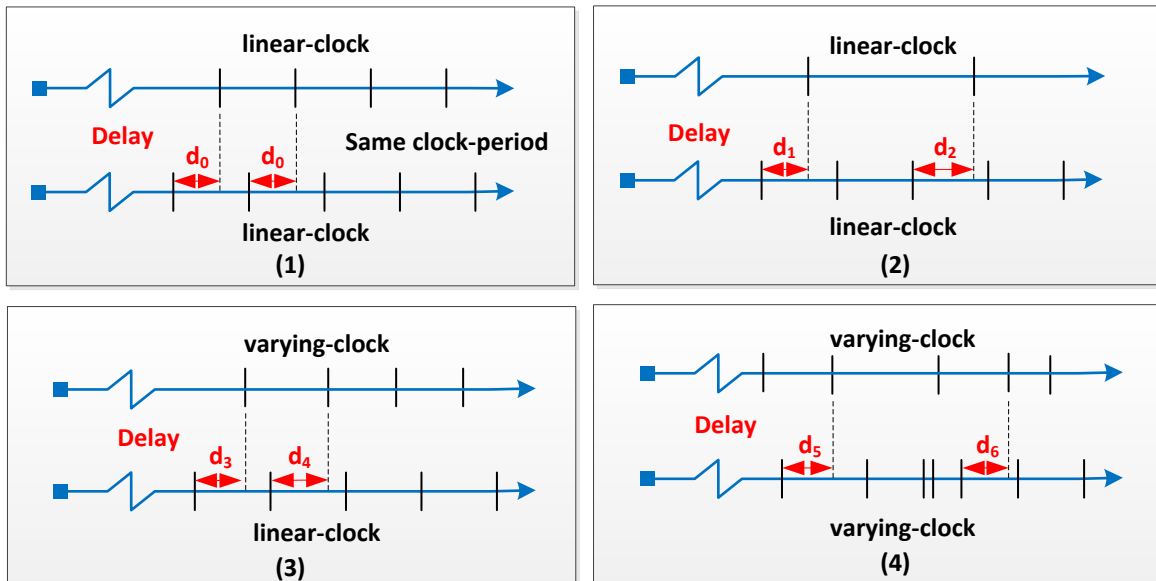


Figure 32: Illustration of delay between two clocks count increments

Two Clocks that have the same clock-period and the delay is zero are called *locked*. Two Clocks that have the same clock-period and the delay is not zero are called *lock-delayed*. In the lock-delayed case the delay is a constant value, see Figure 32, case (1).

Clocks that are locked increment their count of units at the exact same instant of time. Such an example is a wall clock which increments its second hand at the same instant as the second hand

on a wristwatch (i.e., they “tick” at the same time). In this example, when the wristwatch ticks at the same clock-period but is slightly behind, the two clocks are lock-delayed.

Locked, however, does not guarantee that the time reported on each is identical; the watch’s second hand may be pointing to the 10th second, while the wall clock is pointing to the 49th second. The difference in reported time between any two Clocks at the same instant is the *offset*. Two Clocks are *synchronized* when they are locked and have no offset between them; these Clocks then report the same time for all time values. Figure 33 illustrates these relationships using linear clocks.

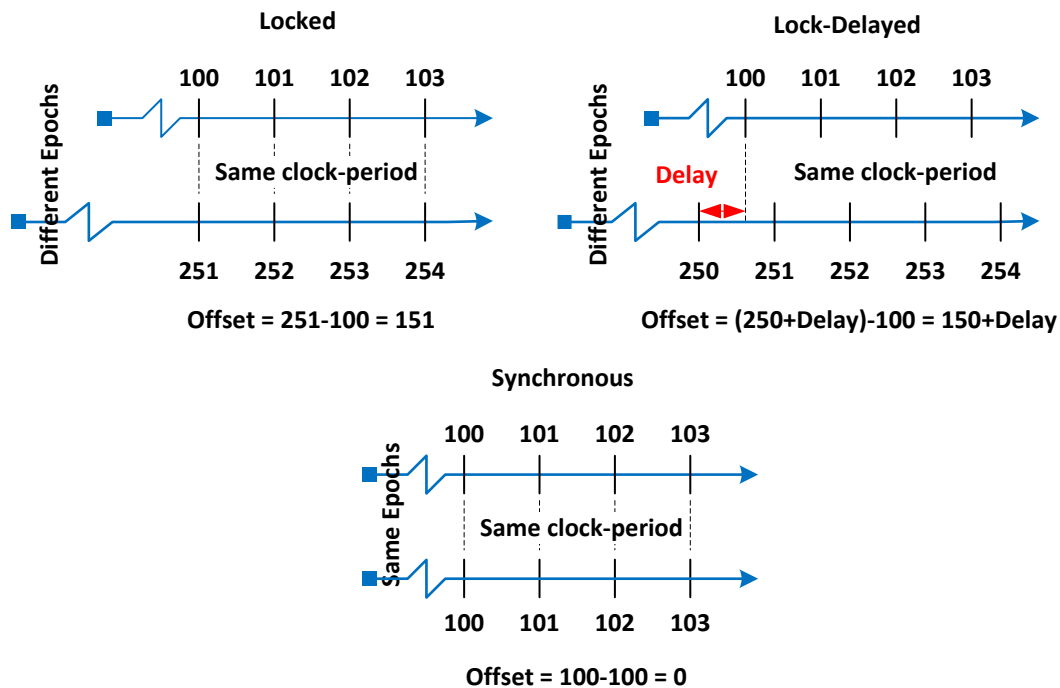


Figure 33: Illustration of Clock Relationships

Clock-period: The amount of time between each successive count of a single Clock, measured in SI Seconds.

Delay: The smallest difference in time between one clock’s count incrementing and a second clock’s count incrementing.

Locked: When two Clocks have the same clock-period and zero delay.

Lock-delayed: When two Clocks have the same clock-period and non-zero delay.

Offset: The difference in Clock values between any two clocks at the same instant.

Synchronized: When two Clocks are locked and have zero offset.

Note, in the following discussion a Clock’s “unit of time” is called a “second,” which is not necessarily a fixed-length SI second.

Two methods are used to represent a Timestamp: Second-Count and Date-Text.

Second-Count: A count of seconds (and fractions of seconds) from an Epoch.

Date-Text: A date/time text value (year, month, day, hours, minutes, seconds, and fractions of seconds). Date-Text is a measure of time since the beginning of the Common Era (CE); that is, the Epoch of Date-Text is midnight January 1, year zero (0000-01-01T00:00:00.0Z).

Although there were several different calendars and methods of time keeping established throughout history, this document's equations, and algorithms compute time as if the current method and calendar has been in place since the year zero Epoch.

6.2.1 Timing System Capability Levels

Three hierarchical capability levels for timestamping temporal events are defined: **Level 1 - Total Ordering**, **Level 2 - Relative Differencing** and **Level 3 - Absolute Time**. A fourth level defines an absence of the other capabilities, **Level 0 – Inconsistent Timing**.

Level 0 – Inconsistent Timing, a below baseline timing system that does not provide Level 1, 2 or 3 capabilities. With Inconsistent Timing, random temporal jumps, either forward or backward, prevent Total Ordering.

Level 1 - Total Ordering, a baseline capability of a timing system provides Total Ordering of Temporal Events. Total ordering means the timestamp for every Temporal Event provides the order in which the events occur. As an example, using a different Clock for metadata timestamping than that for the imager may not produce Total Ordering of the events; in this case it may not be possible to know if metadata came before, during or after the image – Total Ordering is not guaranteed. Total Ordering only determines the order of Temporal Events; it does not imply that the timestamp is related to any specific Absolute Time. Total Ordering also enables the indexing (the ability to quickly find Temporal Events in a list) of Temporal Events.

Level 2 - Relative Differencing capability builds upon Level 1 by adding the ability to compute the difference in time, using SI Seconds, between two timestamps. Accurately determining differences between Temporal Events enables durations to be computed for events which span any amount of time.

Level 3 - Absolute Time capability builds upon Level 2 by generating timestamps in relation to a known universal Absolute Time reference. This enables Temporal Events outside of the MIS to be coordinated with information produced from the MIS.

6.2.2 International Time Systems

Many standard time systems are in use worldwide. Five of these time systems are discussed to aid understanding the time system mandated by the MISP: TAI, UT1, UTC, GPS and POSIX Time. All but POSIX Time use the SI-clock. POSIX is included in this section only for historical reasons.

6.2.2.1 International Atomic Time (TAI)

International Atomic Time employs a number of atomic clocks to denote the passage of time by counting the number of SI Seconds from the TAI Epoch, 1958-01-01T00:00:00.0Z¹. TAI is a SI-clock system where the duration of minutes, hours, and days does not change from 60, 3600 and 86,400 SI Seconds, respectively. TAI is a Level 3 time system as defined in Section 6.2. TAI does not have any Adjustment Criteria.

TAI can be represented using Second-Count of SI seconds since the Epoch, or with Date-Text. When converting to/from Date-Text leap days must be added (or subtracted) as needed (see Section 6.2.5).

As is discussed in Section 6.2.2.2, the length of a mean solar day is not exactly 86,400 SI seconds; therefore, the start of a TAI “day” drifts away from the mean solar day. When a resulting Date-Text is computed from a TAI Second-Count there will be a difference between the TAI Date-Text and UT1 or UTC Date-Text.

6.2.2.2 Universal Time 1 (UT1)

Universal Time 1 measures time by observing the length of a mean solar day in SI Seconds. A mean solar day is the period of time for the sun to move from noon-to-noon between two days. Adjustment Criteria are performed daily based on the Length of Day (LOD). The LOD can change based on planetary wobble and other effects (tidal and lunar). With UT1, the LOD is $86,400 \pm \Delta$ SI Seconds, where Δ is a small number in the milliseconds range. UT1 is represented *only* using Date-Text. At the end of each day, regardless of the Δ value, the Clock is reset to zero hundred hours (00:00:00.0). At the reset time, there is either a potential overlap between days, or a gap in time between the two days, in terms of the Date-Text. Figure 34 illustrates a timeline with Day 1 LOD 0.5 second *longer* than 86,400 seconds (shown as 23:59:60.5). Day 2 then effectively begins with an overlap of Day 1 by 0.5 second; the Date-Text for Day 1 that is greater than 23:59:60 “overlaps” with Day 2. If Day 2 LOD is 0.5 second *shorter* than 86,400 seconds (i.e., 23:59:59.5), there will be a gap between Day 2 and Day 3.

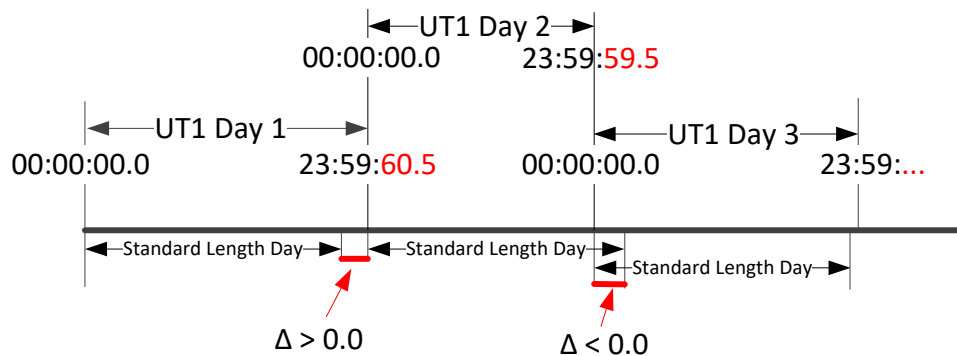


Figure 34: Illustration of UT1 LOD and the overlap and gaps that can occur

¹ At the time TAI was developed UTC did not exist as a standard, so the Epoch is measured relative to Universal Time.

The LOD difference implies that UT1 and TAI are not locked even though both systems count SI Seconds. The Epoch for UT1 is year zero, since UT1 is only represented in Date-Text.

Since the LOD is constantly changing, UT1 is a Level 1 time system. Total Ordering is possible when comparing the UT1 Date-Text; however, Relative Differencing between times from two different days requires knowing the LOD of all days between the differenced times. TAI time can slowly diverge from the UT1 time because of LOD difference between TAI and UT1. The advantage of UT1 is that it is locked with the mean solar day.

6.2.2.3 Coordinated Universal Time (UTC)

UTC is the international time standard used extensively within international communities. Coordinated Universal Time combines the constancy of TAI with the UT1 LOD information. UTC has an Epoch of 1972-01-01T00:00:00.0Z, and it is Locked with TAI, but not synchronized. UTC's Adjustment Criterion is to add or remove "leap seconds" to keep alignment to UT1 to within ± 0.9 SI Seconds. A leap second is a SI Second added or subtracted from UTC on a designated month as determined by U.S. and international standards bodies; since the UTC Epoch there have been a number of leap seconds added to UTC. Table 14 lists the start and end dates/times for when the given leap seconds are in effect. The Start Date/Time values are inclusive in the time range; however, the End Date/Time values are exclusive in the time range.

Table 14: Leap Seconds since January 1972

Start Date/Time (Inclusive)	End Date/Time (Exclusive)	Leap Seconds Offset	Comments
1972-01-01T00:00:00	1972-07-01T00:00:00	10	Original offset from TAI
1972-07-01T00:00:00	1973-01-01T00:00:00	11	1 leap second added
1973-01-01T00:00:00	1974-01-01T00:00:00	12	1 leap second added
1974-01-01T00:00:00	1975-01-01T00:00:00	13	1 leap second added
1975-01-01T00:00:00	1976-01-01T00:00:00	14	1 leap second added
1976-01-01T00:00:00	1977-01-01T00:00:00	15	1 leap second added
1977-01-01T00:00:00	1978-01-01T00:00:00	16	1 leap second added
1978-01-01T00:00:00	1979-01-01T00:00:00	17	1 leap second added
1979-01-01T00:00:00	1980-01-01T00:00:00	18	1 leap second added
1980-01-01T00:00:00	1981-07-01T00:00:00	19	1 leap second added
1981-07-01T00:00:00	1982-07-01T00:00:00	20	1 leap second added
1982-07-01T00:00:00	1983-07-01T00:00:00	21	1 leap second added
1983-07-01T00:00:00	1985-07-01T00:00:00	22	1 leap second added
1985-07-01T00:00:00	1988-01-01T00:00:00	23	1 leap second added
1988-01-01T00:00:00	1990-01-01T00:00:00	24	1 leap second added
1990-01-01T00:00:00	1991-01-01T00:00:00	25	1 leap second added
1991-01-01T00:00:00	1992-07-01T00:00:00	26	1 leap second added
1992-07-01T00:00:00	1993-07-01T00:00:00	27	1 leap second added
1993-07-01T00:00:00	1994-07-01T00:00:00	28	1 leap second added
1994-07-01T00:00:00	1996-01-01T00:00:00	29	1 leap second added
1996-01-01T00:00:00	1997-07-01T00:00:00	30	1 leap second added
1997-07-01T00:00:00	1999-01-01T00:00:00	31	1 leap second added
1999-01-01T00:00:00	2006-01-01T00:00:00	32	1 leap second added
2006-01-01T00:00:00	2009-01-01T00:00:00	33	1 leap second added
2009-01-01T00:00:00	2012-07-01T00:00:00	34	1 leap second added
2012-07-01T00:00:00	2015-07-01T00:00:00	35	1 leap second added
2015-07-01T00:00:00	2017-01-01T00:00:00	36	1 leap second added
2017-01-01T00:00:00	Unknown	37	1 leap second added

From 1961-01-01 to 1972-01-01, UTC used the equivalent of fractional leap seconds that would be added daily. Equation 2 is used to compute the leap second offset (TAI-UTC) for a given day D. D is used to look up values B, S and R from Table 15.

	$Leap\ Second = TAI - UTC = B + (D - S) * R$	Equation 2
--	--	------------

Where:

<i>B</i>	Offset in seconds. A lookup value from Table 15 based on D
<i>D</i>	Day for which leap seconds are accounted in Modified Julian Days (MJD).
<i>S</i>	TAI Reference Date in MJD. A lookup value from Table 15 based on D.
<i>R</i>	Rate factor. A lookup value from Table 15 based on D.

For MJD conversions there are many internet web tools.

Table 15: Leap Second Computation for Dates Ranging from 1961-01-01 to 1972-01-01.
Data derived from U.S. Navy Observatory file (<ftp://maia.usno.navy.mil/ser7/tai-utc.dat>).

Start Date (Inclusive)	End Date (Exclusive)	B	S	R
1961-01-01	1961-08-01	1.4228180	1961-01-01	0.001296
1961-08-01	1962-01-01	1.3728180	1961-01-01	0.001296
1962-01-01	1963-11-01	1.8458580	1962-01-01	0.0011232
1963-11-01	1964-01-01	1.9458580	1962-01-01	0.0011232
1964-01-01	1964-04-01	3.2401300	1965-01-01	0.001296
1964-04-01	1964-09-01	3.3401300	1965-01-01	0.001296
1964-09-01	1965-01-01	3.4401300	1965-01-01	0.001296
1965-01-01	1965-03-01	3.5401300	1965-01-01	0.001296
1965-03-01	1965-07-01	3.6401300	1965-01-01	0.001296
1965-07-01	1965-09-01	3.7401300	1965-01-01	0.001296
1965-09-01	1966-01-01	3.8401300	1965-01-01	0.001296
1966-01-01	1968-02-01	4.3131700	1966-01-01	0.002592
1968-02-01	1972-01-01	4.2131700	1966-01-01	0.002592

Example 1: Computation of Leap Second for 1970-01-01T00:00:00.0

$$D = \text{MJD}(1970-01-01\text{T}00:00:00.0) = 40587.0$$

From last row in Table 15:

$$B = 4.2131700$$

$$S = \text{MJD}(1966-01-01\text{T}00:00:00.0) = 39126.0$$

$$R = 0.002592$$

$$Leap\ Second = TAI - UTC = B + (D - S) * R = 4.21317 + (40587 - 39126) * 0.002592$$

$$Leap\ Second = 8.000082$$

From 1972-01-01 and on, leap second additions and subtractions to UTC Date-Text are described in ITU-R TF.460-6 [11]:

2.1 - A positive or negative leap-second should be the last second of a UTC month, but first preference should be given to the end of December and June, and second preference to the end of March and September.

2.2 - A positive leap-second begins at 23h 59m 60s and ends at 0h 0m 0s of the first day of the following month. In the case of a negative leap-second, 23h 59m 58s will be followed one second later by 0h 0m 0s of the first day of the following month.

Figure 35 illustrates the addition and subtraction of leap seconds. The top illustration shows the addition of a leap second by adding a 61st second to the last minute of the last day of a UTC month; the time increments from 59 to 60 seconds and then to 00 seconds. The bottom illustration shows the removal of a second from the last minute of the last day of a UTC month; the time increments from 58 seconds to 00 seconds (of the next day), the 59th second is skipped.

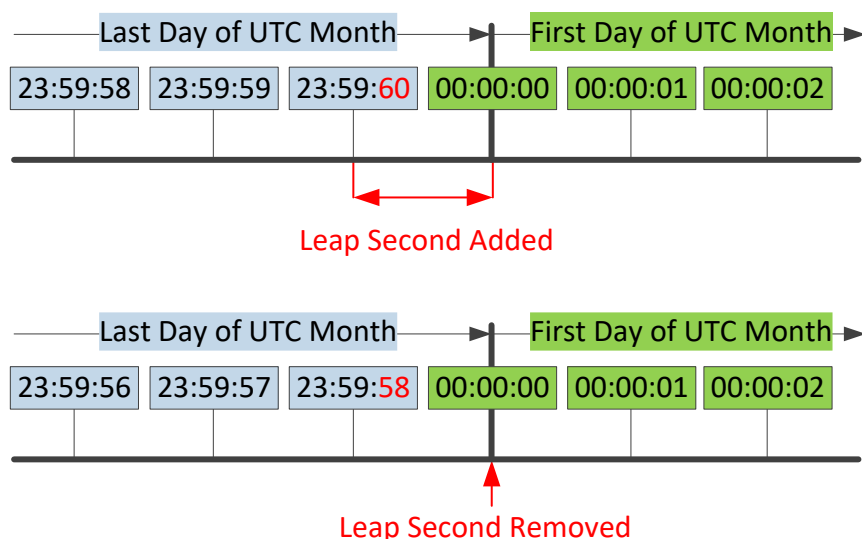


Figure 35: Illustration of Leap Seconds Added or Removed from UTC and the associated Date-Text

Although UTC is represented only using Date-Text, it can be converted to a Second-Count from any given Epoch occurring after the UTC Epoch – provided the number of leaps seconds are known and included for the specific date and Epoch.

Since leap seconds are added or removed UTC does not meet any of the capability levels as defined in Section 6.2; Total Ordering is not possible beyond the day. Since there is the possibility of added leap seconds, Relative Differencing between times from two different days requires knowing the leap seconds between the differenced times.

UTC is offset from TAI with a defined integer number of SI Seconds plus leap seconds.

6.2.2.4 Global Positioning Time (GPS)

The GPS time system has its Clock locked with the TAI Clock with an Epoch of 1980-01-06T00:00:00.0Z (UTC). For Date-Text, GPS is equivalent to TAI with an offset of 19 SI Seconds. For Second-Count, GPS is equal to TAI with an offset of the GPS Epoch Second-Count

plus the 19 SI Seconds (see Table 17). GPS time does not have any Adjustment Criterion. GPS is a capability Level 3 time system as defined in Section 6.2.

The GPS system relies on a collection of Atomic time clocks and provides position information enabling geo-location anywhere on or above the earth. The GPS system consists of a constellation of satellites orbiting the earth, where each satellite provides location and time information to GPS receivers. Many GPS receivers produce multiple time systems including UTC time. GPS supplies a message, which contains UTC corrections that allows a receiver to convert GPS Time into UTC or any time zone. This message includes the time difference in whole seconds between GPS time and UTC.

6.2.2.5 POSIX Time

Historically, POSIX time was the basis of the time systems used in older versions of the MISP and supporting documents. However, because of vagueness in the POSIX standard about if and how to adjust for UTC leap-second changes, the POSIX time system is no longer used as the basis for the MISP. The lack of clarity led to different interpretations and implementations. One interpretation included UTC leap second corrections, while the other (originally intended) interpretation did not include UTC leap second corrections. The information in this section is included for historical and reference only.

POSIX Time is a time system developed in POSIX IEEE Std. 1003.1 [12]. POSIX Time is defined as the Second-Count (along with a nanosecond counter) since the Epoch of 1970-01-01T00:00:00.0Z (UTC²) and does not include leap seconds. The duration of its second is not specified, but it is nominally equal to a SI Second; therefore, POSIX is not locked to TAI. POSIX Time does not have any Adjustment Criterion. The POSIX standard includes an algorithm for converting the Second-Count to Date-Text (called “broken-down time”) as UTC, but this conversion does not include leap seconds, so the relationship between UTC and Second-Count is noted as unspecified in the standard. Because the value of the second is not mandated to be an SI second, the POSIX Time system is not locked with TAI; thus, it is a Level 1 time system as defined in Section 6.2.

POSIX-compliant systems provide methods for operating with POSIX Time and converting to/from broken-down time. If the appropriate values are added for leap seconds, POSIX-based systems provide a toolset for converting between UTC and POSIX Time.

6.2.3 MISP Time System

The MISP Time System clock counts SI-Seconds since the Epoch of 1970-01-01T00:00:00.0Z (UTC³). The MISP Time System is locked with TAI with a Leap Second offset of 8.000082 seconds (see Example 1 for computation). The MISP Time System does not include UTC leap-seconds. The Epoch date is based on UNIX time (also known as POSIX time). The MISP Time System is essentially equivalent to the UNIX or POSIX time (without UTC leap second corrections) documented within prior versions of MISP standards, recommended practices and engineering guidelines. Historically, the fractional 82 microseconds were not included in the offset computation; existing systems did not need this degree of accuracy. However, in future

² Leap second offset is computed using Equation 2

³ Leap second offset is computed using Equation 2

systems, where more precise timing may be required, the 82-microsecond offset may be important when comparing measurements against TAI.

The MISP time system defines the **Precision Time Stamp** to represent the value of the count of the number of microseconds (based on the SI Second) since the UNIX Epoch time of 1970-01-01T00:00:00.0Z (UTC⁴). The MISP time system is a Level 3 time system as defined in Section 6.2.

The MISP time system also defines the **Nano Precision Time Stamp** to represent the value of the count of the number of nanoseconds (based on the SI Second) since the UNIX Epoch time of 1970-01-01T00:00:00.0Z (UTC⁵).

The MISP time system is a Level 3 time system as defined in Section 6.2.

6.2.4 Time Systems Summary

Table 16 lists the time systems and their properties.

Table 16: List of Time Systems

Time System	Clock Type	Epoch	Adjustment Criteria
TAI	SI-clock	1958-01-01T00:00:00.0Z	None
UT1	SI-clock	0000-01-01T00:00:00.0Z	Daily Length of Day Adjustments
UTC	SI-clock	1972-01-01T00:00:00.0Z	Potential Monthly Leap Second Adjustments ⁶
GPS	SI-clock	1980-01-06T00:00:00.0Z	None
POSIX	varying-clock	1970-01-01T00:00:00.0Z	None
MISP	SI-clock	1970-01-01T00:00:00.0Z	None

Figure 36 shows the relationship of the various time systems. This figure does not include UT1, because Motion Imagery applications do not directly use it.

⁴ Use Equation 2 to compute the Leap second offset.

⁵ Use Equation 2 to compute the Leap second offset.

⁶ While it is possible to adjust UTC with the addition or subtraction of one leap second at the end of each month, June and December are the preferred months to make this change. Historically, there has never been a need to make an adjustment outside of these two months.

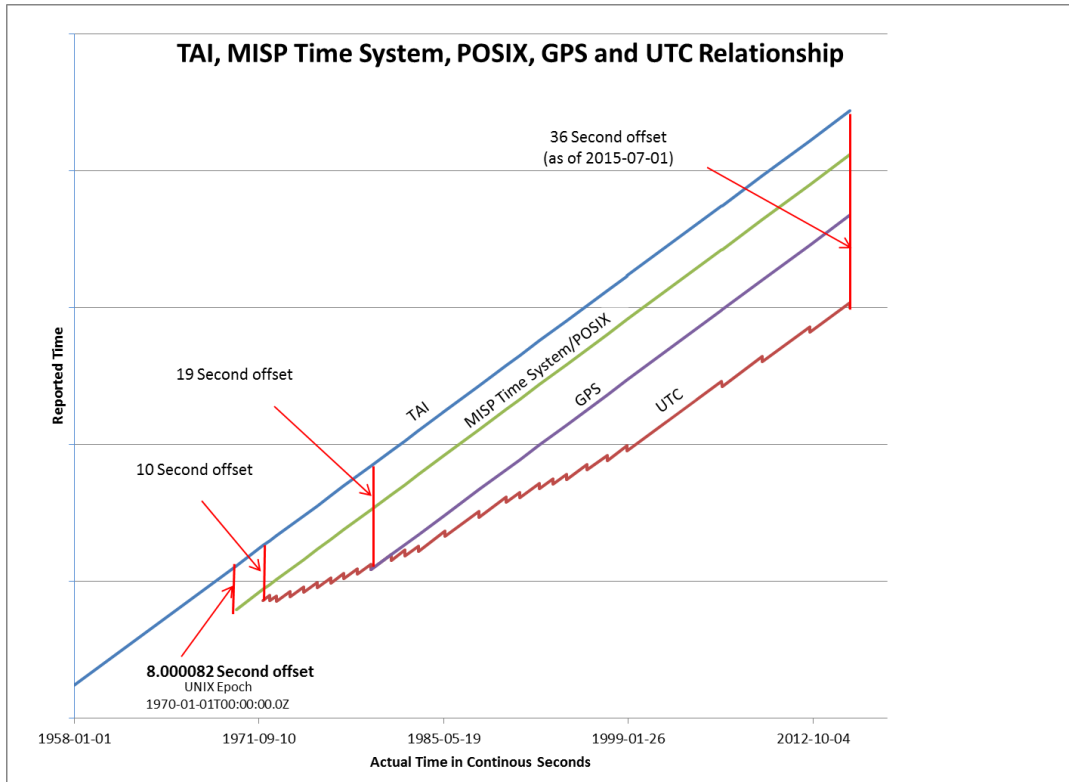


Figure 36: Relationships among Time Systems

Figure 36 shows TAI, POSIX (without UTC leap second corrections), MISP and GPS are all linear time systems with constant offsets from each other; this enables straightforward time conversions between these systems. UTC is piecewise linear; thus, the conversion of UTC to/from any other time systems requires a table of Offsets for each given time period where the Offset was in effect.

6.2.5 Time Conversions

Table 17 lists the equations to perform the conversion from one time system to another. The values denoting the various time systems T, G and M are in Second-Counts from their respective epoch (see Table 16) and U is in Date-Text. Converting to/from UTC requires converting Date-Text into Second-Counts and vice versa, where the resulting Second-Counts are offsets from the TAI epoch.

Table 17: Time System Conversions

		To			
		TAI	UTC	GPS	MISP
From	TAI (T)		ComputeUTC(T)	$T - G_{\text{Epoch}}$	$T - M_{\text{Epoch}}$
	UTC (U)	ComputeTAI(U)		$\text{ComputeTAI(U)} - G_{\text{Epoch}}$	$\text{ComputeTAI(U)} - M_{\text{Epoch}}$
	GPS (G)	$G + G_{\text{Epoch}}$	$\text{ComputeUTC}(G + G_{\text{Epoch}})$		$G + G_{\text{Epoch}} - M_{\text{Epoch}}$
	MISP (M)	$M + M_{\text{Epoch}}$	$\text{ComputeUTC}(M + M_{\text{Epoch}})$	$M + M_{\text{Epoch}} - G_{\text{Epoch}}$	
Where M_{Epoch} and G_{Epoch} are offsets to the TAI Epoch: $M_{\text{Epoch}} = \text{MISPEpoch} + 8.000082$ $G_{\text{Epoch}} = \text{GPSEpoch} + 19$					

This table does not include UT1 because Motion Imagery applications do not use UT1 directly.

6.2.5.1 Compute TAI

Converting from UTC to TAI requires the subtraction of the current leap second value from the UTC time.

ComputeTAI (from UTC):

- 1) Lookup UTC time in Table 14 to get leap second offset, L
- 2) Convert UTC time Date-Text to seconds-count
- 3) $\text{TAI} = \text{UTC} + L$

See Appendix C for how to access reference code.

6.2.5.2 Compute UTC

Converting TAI time to UTC requires several conditional checks to determine the correct leap second addition. The first step in the conversion is to change the TAI time representation from a seconds-count to Date-Text. A TAI time converts to a Date-Text format (TAI_{Date}) using the method from Section 6.2.2.5, which produces an uncorrected TAI_{Date} . To correct TAI_{Date} to UTC the conversion algorithm needs to determine the correct leap-second offset.

Given the TAI_{Date} the first step is to define three Leap-Second Zones from the Leap-Second table, Table 14: During, Before, and After. Each zone is inclusive on the begin date (left) and exclusive on the end date (right). The During Leap-Second Zone contains the uncorrected TAI_{Date} and the zone has leap-second value of L_D , i.e., to compute the TAI time for each UTC time in this zone, add L_D seconds. The Before Leap-Second Zone has a leap-second value of L_B and the After Leap-Second Zone has a leap-second value of L_A .

Figure 37 illustrates a timeline with TAI time below the timeline (blue), UTC time above the timeline (orange), and the three Leap-Second Zones shown above UTC time.

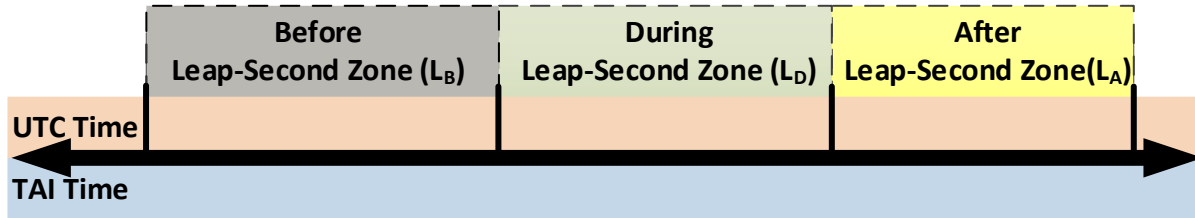
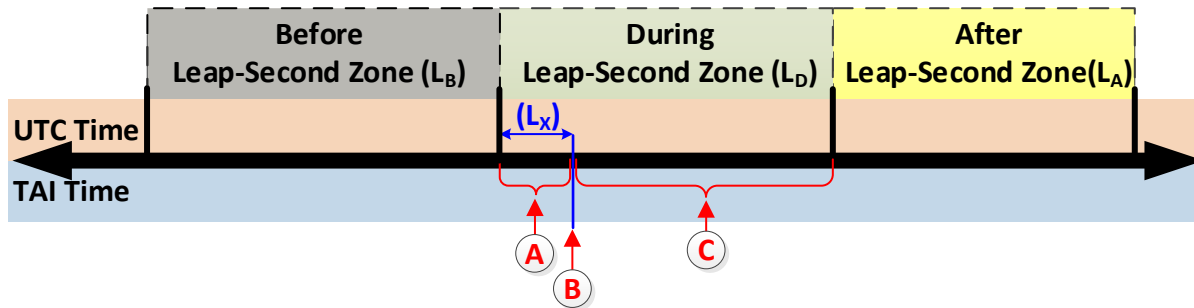


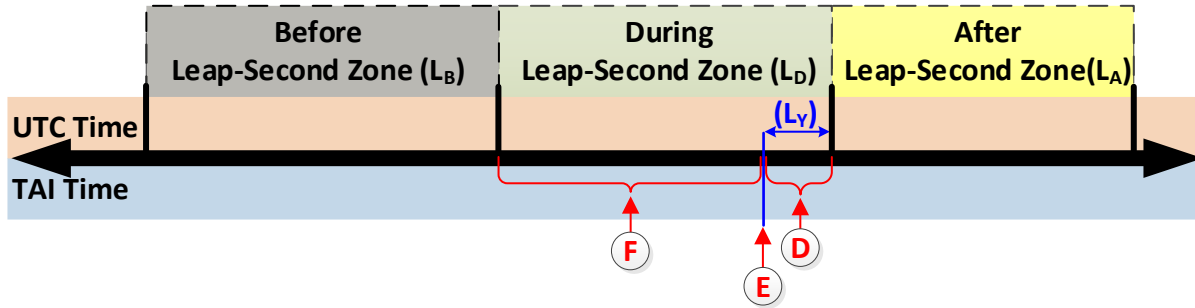
Figure 37: UTC Conversions

There are three correction cases shown in Figure 38 when L_D is positive or negative. When L_D is positive the TAI_{Date} is in one of three regions: A, B, or C. Region A ranges from the During Leap-Second Zone start date (on the left of the zone) to the start date plus the value of L_X , exclusively. Region A uses the L_B leap second value to correct the TAI_{Date} . The value of L_X depends on the relationship of L_D and L_B , discussed in the next paragraph. Region B is a transition region with a duration of one SI-second. In this region the conversion must either include or exclude the UTC leap second (i.e., where the last second of the last minute of the day is either a value of 58 or 60 seconds – see 6.2.2.3). Region C ranges from the end of transition region to the beginning of the After Leap-Second Zone, exclusively. Region C uses the L_D leap second value to correct the TAI_{Date} .

Figure 38: UTC Conversion with $L_D > 0$

As mentioned, the value of L_X depends on the relationship between the Before and During Leap-Second Zone's leap second values. If L_D is greater than L_B the During Leap-Second Zone has an addition leap second so $L_X = L_B$ and Region B's seconds value is 60. If L_D is less than L_B then the During Leap-Second Zone has the removal of a leap second and $L_X = L_B - 1$ and Region B's seconds value is 58.

When L_D is negative the TAI_{Date} is in one of three regions D, E, or F as shown in Figure 39. Going right to left, region D ranges from the After Leap-Second Zone start date (on the right of the zone) to the start date minus the value of L_Y , exclusively. Region D uses the L_A leap second value to correct the TAI_{Date} . The value of L_Y depends on the relationship of L_D and L_A , discussed in the next paragraph. Region E is a transition region, with a duration of one SI-second. In this region the conversion must either include or exclude the UTC leap second (i.e., where the last second of the last minute of the day is either a value of 58 or 60 seconds – see 6.2.2.3). Region F ranges from the end of transition region to the beginning of the During Leap-Second Zone, exclusively. Region F uses the L_D leap second value to correct the TAI_{Date} .

Figure 39: UTC Conversion with $L_D < 0$

As mentioned, the value of L_Y depends on the relationship between the During and After Leap-Second Zone's leap second values. If L_A is greater than L_D the During Leap-Second Zone has an addition leap second so $L_Y = L_A$ and Region E's seconds value is 60. If L_A is less than L_D then the During Leap-Second Zone has the removal of a leap second and $L_Y = L_A - 1$ and Region E's seconds value is 58.

A simpler approach (less conditions) is to use an alternate leap second table where the boundaries between leap second times are in TAI time. Compute each TAI leap second start time with $TAI_i = UTC_i + Leap_i$. Figure 40 illustrates a TAI Leap second table below the timeline for positive leap seconds. When converting a TAI time, the TAI table provides the correct leap second value for the given TAI time; however, the boundary values still need further processing to either include or exclude the UTC leap second (i.e., where the last second of the last minute of the day is either a value of 58 or 60 seconds – see 6.2.2.3) in the final UTC time.

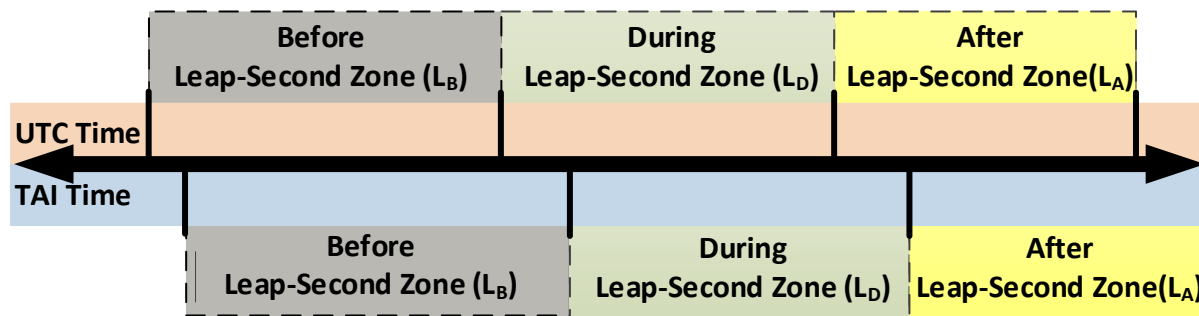


Figure 40: TAI Leap Second Table

See Appendix C for how to access reference code.

6.2.5.3 Date-Text to Seconds-Count

Computing the Seconds-Count from Date-Text requires converting and combining all the Date-Text components into nanoseconds. Given Date-Text with *Year*, *Month*, *Day*, *Hour*, *Minute*, *Second*, and *Nanoseconds*, the general algorithm for computing the Seconds-Count relative to the TAI epoch is:

- 1) Days = days in year from **Month** and **Day** in month (include leap day as necessary).

- 2) Leaps = number of leap years (not including this year) since TAI Epoch. This is a count of leap days.
- 3) DaySeconds = (Days + Leaps) * (seconds in a day)
- 4) YearSeconds = (**Year** – EpochYear) * (seconds in a standard 365-day year)
- 5) HourSeconds = **Hour** * (seconds in hour)
- 6) MinuteSeconds = **Minute** * (seconds in minute)
- 7) TotalSeconds = YearSeconds + DaySeconds + HourSeconds + MinuteSeconds + **Second**
- 8) **Seconds-Count** = TotalSeconds * (nanos per second) + Nanoseconds.

Note: EpochYear = 1958

Note: (nanos per second) = 10^9

The resulting Seconds-Count is the offset from the TAI epoch.

See Appendix C for how to access reference code.

6.2.5.4 Seconds-Count to Date-Text

Computing the Date-Text from Seconds-Count requires dividing out the various Date-Text components (i.e., **Year**, **Month**, **Day**, **Hour**, **Minute**, **Second**, and **Nanoseconds**) from the Seconds-Count. Given Seconds-Count relative to the TAI epoch as S_C , the general algorithm for computing the Date-Text is:

- 1) TotalSeconds = Floor (S_C /(nanos per second))
- 2) **NanoSeconds** = (TotalSecond - S_C) * (nanos per second)
- 3) Days = TotalSeconds/(seconds per day)
- 4) **Year** = Floor(days/(days in year average)) + EpochYear
 - a. Year may need adjustment depending on edge case from leap year (see reference code)
- 5) Leaps = number of leap years (not including this year) since TAI Epoch. This is a count of leap days.
- 6) YearFirstDay = (Year – EpochYear) *365 + Leaps
- 7) DayInYear = Days – YearFirstDay
- 8) Compute **Month** and **Day** from DayInYear using table lookup
- 9) DaySeconds = TotalSeconds – Days * (seconds in day)
- 10) **Hour** = Floor((DaySeconds) / (seconds in hour))
- 11) HourSeconds = DaySeconds – Hour*(seconds in hour)
- 12) **Minute** = Floor(HourSeconds/(seconds in minute))
- 13) **Second** = Floor(HourSeconds – min*(seconds in minute))
- 14) **Date-Text** combines the **Year**, **Month**, **Day**, **Hour**, **Minute**, **Second**, and **Nanoseconds**

Note: EpochYear = 1958

See Appendix C for how to access reference code.

6.2.6 Time Sources

There are many different time sources, and each provides some degree of precision and accuracy. Time sources can be categorized into three types: **Atomic**, **Independent** and **Hybrid**.

Atomic Time Sources (ATS) are second-by-second synchronized (or better) with TAI time; however, the Epoch of the time source may differ from TAI. An example of an ATS is time

output by a GPS receiver. An **Independent** time source is not synchronized with TAI, which may or may not count seconds at a constant rate (i.e., a varying-clock). An example of an independent time source is a non-networked computer using an internal clock as its time reference. A **Hybrid** time source operates as an independent source but is periodically synchronized to an Atomic time source. In the period of time until resynchronization, a Hybrid time source is said to be *freewheeling*. An example Hybrid time source is a GPS receiver connected to a computer; the GPS provides a periodic time signal update, which the computer synchronizes to, and then free-wheels until the next GPS update. During the freewheeling period the clock may change its absolute relation to its reference source; this is one form of what is called *clock drift*.

6.2.6.1 GPS System

GPS provides an Atomic time source and position information enabling geo-location anywhere on or above the earth. The GPS system consists of a constellation of satellites orbiting the earth, where each satellite provides location and time information to GPS receivers.

Many GPS receivers output time information along with a one pulse per second (1PPS) synchronization signal. This time information may be in a variety of formats (UTC, GPS, etc.) and is usually accurate only to the SI Second. The 1PPS synchronization signal enables Hybrid time sources to be built for Motion Imagery Systems. A 1 PPS signal enables sub-second finer gradations of time (i.e., microseconds) to be derived by phase locking a high frequency (e.g., 1 MHz) clock to the signal. Some GPS receivers output an Inter-Range Instrumentation Group (IRIG) Standard 200 time signal from which times to the SI Second and time to the sub-second can be derived.

6.2.6.1.1 - GPS Time to UTC Conversion

Some receivers provide only GPS Week and GPS Seconds parameters. The offset of GPS Seconds is defined relative to the beginning of the current GPS week. GPS time is referenced to a UTC zero-time point originally defined as midnight (00:00 UTC) before the morning of 1980-01-06. The GPS Week parameter is 10 bits, which is modulo 1024, so the GPS week cycle is 1024 weeks (7168 days, or 19+ years). The latest zero-time point was 1999-08-22 00:00 GPS time (more modern GPS navigation systems use a 13-bit field that repeats every 8,192 weeks.) The following algorithm provides for calculation of the date and time to within one second (further precision may require provisions such as a local clock reference synchronized to the GPS signal):

Formula:

UTC = GPS – leap seconds			
Since,	GPS	=	GPS Week + GPS Seconds + 1999-08-22 00:00
Then,	UTC	=	(GPS Week + GPS Seconds) + 1999-08-22 00:00 – leap seconds
		=	GPS Week + (GPS Seconds – leap seconds) + 1999-08-22 00:00

Algorithm:

Compute UTC Date-Text (Year-Month-Day-Hours:Minutes:Seconds, seconds includes fractions of seconds) from GPS.**(Note: Notional and untested)**

```
// If (GPS Seconds - leap seconds) < 0, add in one week to the GPS Seconds
// count and subtract one week from GPS Week count (avoids negative time)
If (gpsSeconds - Leap_Seconds) < 0
    gpsSeconds = gpsSeconds + (7 * 24 * 60 * 60) /* add week */
    gpsWeek = gpsWeek - 1 /* subtract week */
End If
tmpBeginning_of_current_week = (7 * gpsWeek) + 1999-08-22 00:00
tmpDay_of_week = floor((gpsSeconds - Leap_Seconds) / (24 * 60 * 60))
tmpSeconds_from_midnight = (gpsSeconds - Leap_Seconds) % (24*60*60)
utcCurrent_date = tmpBeginning_of_current_week + tmpDay_of_week
utcHours = floor(tmpSeconds_from_midnight / (60*60))
utcMinutes = floor((tmpSeconds_from_midnight % (60*60)) / 60)
utcSeconds = tmpSeconds_from_midnight % 60
```

6.2.7 Formatting Dates and Times in Text: ISO8601

All dates and times within the Handbook use the ISO 8601 [13] standard formatting of:

CCYY-MM-DDThh:mm:ss.sZ

Where *CCYY* is a four digit year; *MM* is the month number; *DD* is the day within the given month; **T** is a placeholder to signify a separation between the Date and Time; *hh* is the hour number ranging from 0 to 23; *mm* is the number of minutes in an hour; *ss.s* is the number of seconds along with fractions of a second which can be more than one digit; *Z* is a single letter that signifies the time zone of the time, for this document all times are in the Zulu, or Z, time zone.

6.3 Timestamp Accuracy and Precision

Quantities that are measured always have some tolerance and disturbances that introduce uncertainty. Measurement uncertainties are composed of both **systematic** and **random** errors. **Systematic** error is caused by abnormalities in one or more system components and tends to shift all measurements in a systematic way, so that during a number of measurements the average value is constantly displaced or varies in a predictable way. For example, a sampling of a system's reference clock may be offset by some fixed amount from the delay of processing. **Random** errors may be caused by noise, lack of sensitivity, and other reasons; these errors vary in an unpredictable way.

Accuracy indicates how close a measured value is to its actual value. In timestamping, accuracy is the average difference between each timestamp (the measured time) and the actual time of the event (reference time). A measured time value relative to the time reference will either be exact or more likely, not exact. In a time system, each measured value represents a new instance of time. As time increases, each new measured value is compared to the actual reference time and the differences between pair are collected to provide a "picture" of error – i.e., a histogram showing a distribution of the differences between a measured time and its reference.

In Figure 41 (left), measured values (red diamonds) of a time reference show deviation from the reference time; some measured values are near identical to the reference, while others are either

greater (above the green dashed line), or less (below the green dashed line) than the reference time. Taking the difference between a measured value and the reference over a collection of many time measurements yields a graph similar to the graph on the right in Figure 41.

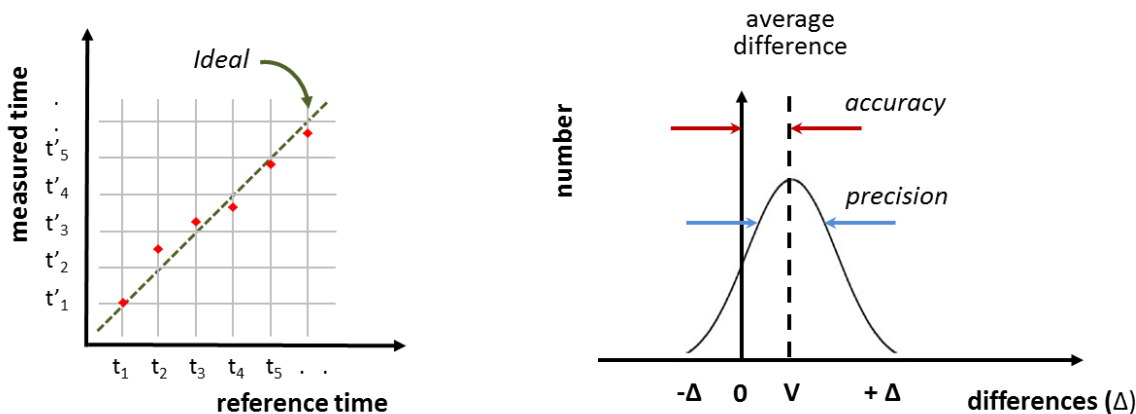


Figure 41: A series of time measurements (left) - Errors plotted as a histogram (right)

The “average difference” represents the average of all the differences between each measured time value and its corresponding reference time. Ideally, this line would be centered at zero; the offset from zero (shown at “V”) represents the accuracy of the data.

The presence of a consistent offset from zero indicates that the accuracy has a *bias* to it. Bias is equivalent to the total systematic error in the measurement, and a correction to negate the systematic error can be made by adjusting for the bias. An example of bias is processing latency, where the capture process of the measurement takes a certain quantity of time to produce the result. System implementers need to understand the source of bias – quantify the bias – and provide correction if necessary. Random error may also move the average difference from zero; this, however, will likely be a much smaller error component if bias is present.

Precision is the ability of a measurement to be consistently reproduced. In timestamping, precision is the variation of the difference between each timestamp and the actual time of the event (reference time). Precision and accuracy are different statistics based on the same set of values. The dispersion of difference values (indicated as $\pm\Delta$ in Figure 41) about the average difference provides a measure of precision. Systems with measured values that tend to be nearer the average in error are said to have “good” precision, i.e., they are repeatable within an acceptable range of confidence. The standard deviation is the metric normally used to gauge this range of confidence. One standard deviation (1σ) about the average will contain 68.3% of the same measured errors.

Figure 42 illustrates an example where 500 samples of a time reference are plotted as a function of the error between the time reference and a sampling (measurement) of the time reference.

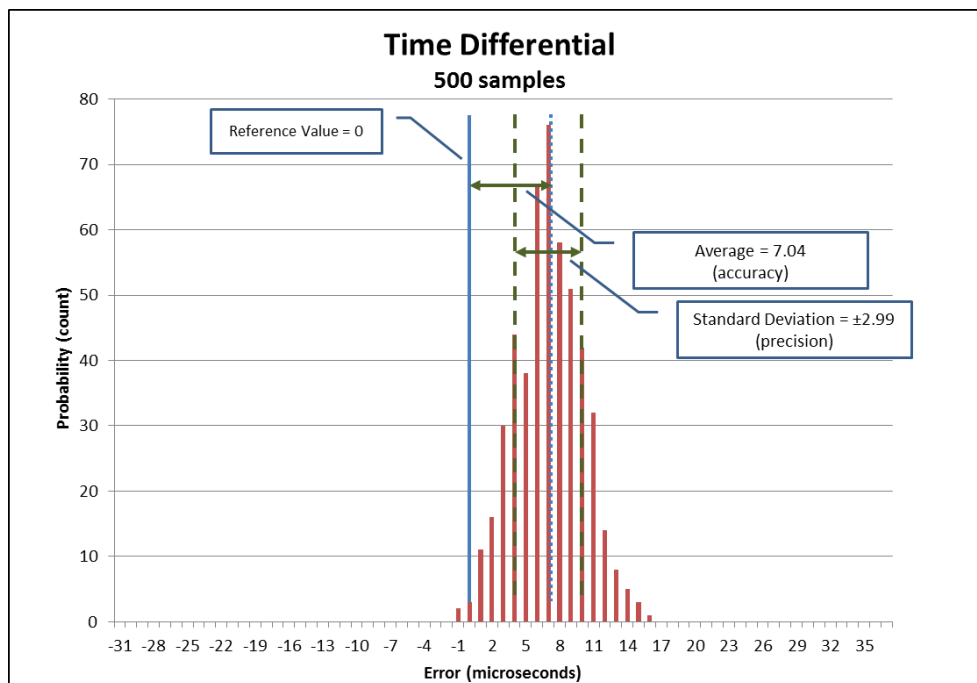


Figure 42: Example 1 - Poor Accuracy, Good Precision

A plot with no errors would produce all zero values, and thus a single line at the value 0 microseconds. The line called “Reference Value = 0” is the ideal case of no error. Accuracy is a measure of the distance from this reference value to the average value of all sampling; in this case 7.04 microseconds. This histogram of values shows that most errors tend towards a value of 7.04 microseconds the average of all 500 samples produces an average of 7.04 microseconds. Whether an average error in sampling of 7.04 microseconds is acceptable is a systems implementation decision.

In Figure 42, the standard deviation is ± 2.99 microseconds. Thus, the system in Figure 42, example 1, has an average accuracy of 7.04 microsecond’s ± 2.99 – so the timestamp of the time reference can be found to be within 4.05 to 10.03 microseconds 68.3% of the time.

In Figure 43, example 2, the accuracy is better, i.e., 1.46 microseconds, but the precision is worse i.e., ± 9.78 microseconds than the case in example 1. The timestamp is more accurate than in example 1 but shows a wider dispersion of error. Thus, the timestamp can be -8.32 to 11.24 microseconds 68.3% of the time with respect to the time reference. This example is considered to have good accuracy, but poor precision as compared to example 1. Again, no judgement is made on whether this is acceptable for a systems implementation.

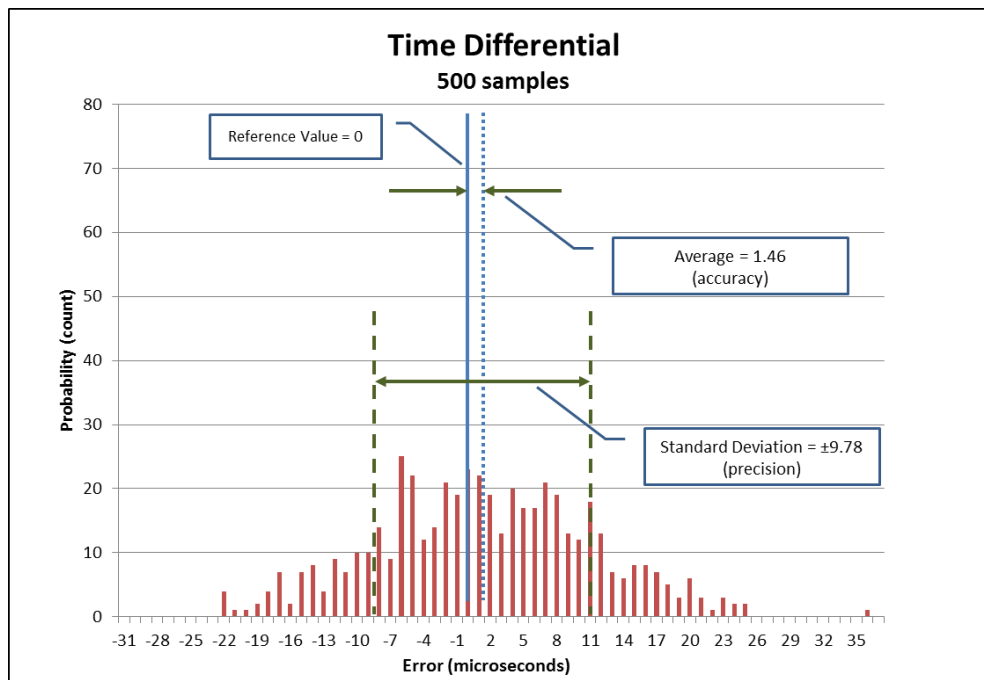


Figure 43: Example 2 - Good Accuracy, Poor Precision

6.4 Time Transfer

Time transfer is the process of continuously *synchronizing*⁷ (see Section 6.2) two or more clocks producing a *hybrid* time source. When performing time transfer one clock is a reference clock and the other clock(s) are receptor clocks. With Motion Imagery Systems time transfer shares the reference time of a well-known accurate clock (e.g., GPS) with a local receptor clock (e.g., GPS receiver). Time transfer is a continuous process where each receptor clock is constantly comparing and adjusting its time to match the reference clock.

Figure 44 illustrates a two-level time transfer system with two reference/receptor pairs. The first reference/receptor pair, called the primary reference/receptor pair, is a GPS time transfer system between the GPS satellite and GPS receiver. Within this time transfer system, the GPS satellite broadcasts a reference time, which after a brief time later (due to propagation time), the receiver's antenna receives. The GPS receiver in turn synchronizes the primary receptor time to an internal secondary reference time adjusting for various effects, such as the propagation time to the antenna and relativistic clock rates. This introduces a secondary reference/receptor time system, wherein the GPS receiver produces a time reference (secondary reference) for listening receptor (secondary receptor) devices.

Because the following discussion applies to reference/receptor pairs in general, there is no need for the qualifiers "primary" and "secondary".

⁷ Italicized terms used in this section are the terms defined in Section 6.2

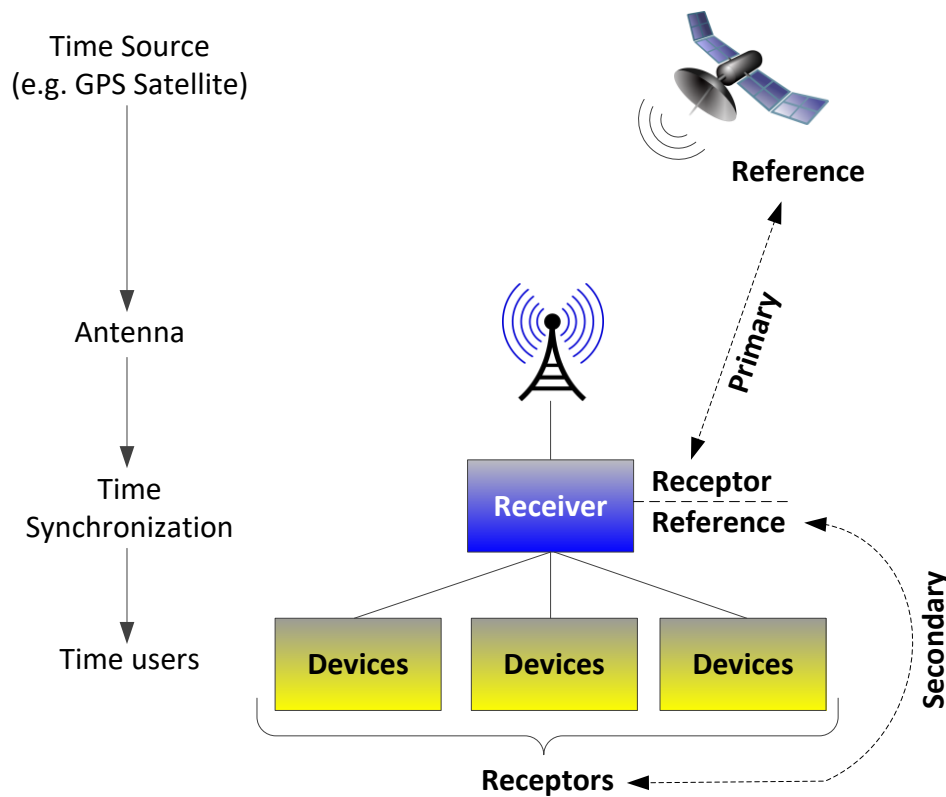


Figure 44: Illustration of GPS time transfer to devices

In typical operation, the reference time *synchronizes* with a receptor, meaning the two clocks are in the *locked* state with almost zero *offset* between them. The receptor clock may show slight error because of latency and other errors, but systems can characterize these errors statistically with a mean and standard deviation.

In cases of signal loss, interference, or other issues, the receptor clock may temporarily disconnect, or unlock, from the reference clock for a period. Clocks have internal time systems which allow them to continue counting time regardless of external input from a time reference. Such a clock is a *freewheeling* clock, as it is independently counting time based on its self-contained timing mechanism. When a receptor clock unlocks and is freewheeling, the clock will drift in time from the reference clock. The difference between the reference time and receptor time at a given instant is *clock-drift*. Typically, *clock-drift* is linear as a function of time, based on the drift-rate.

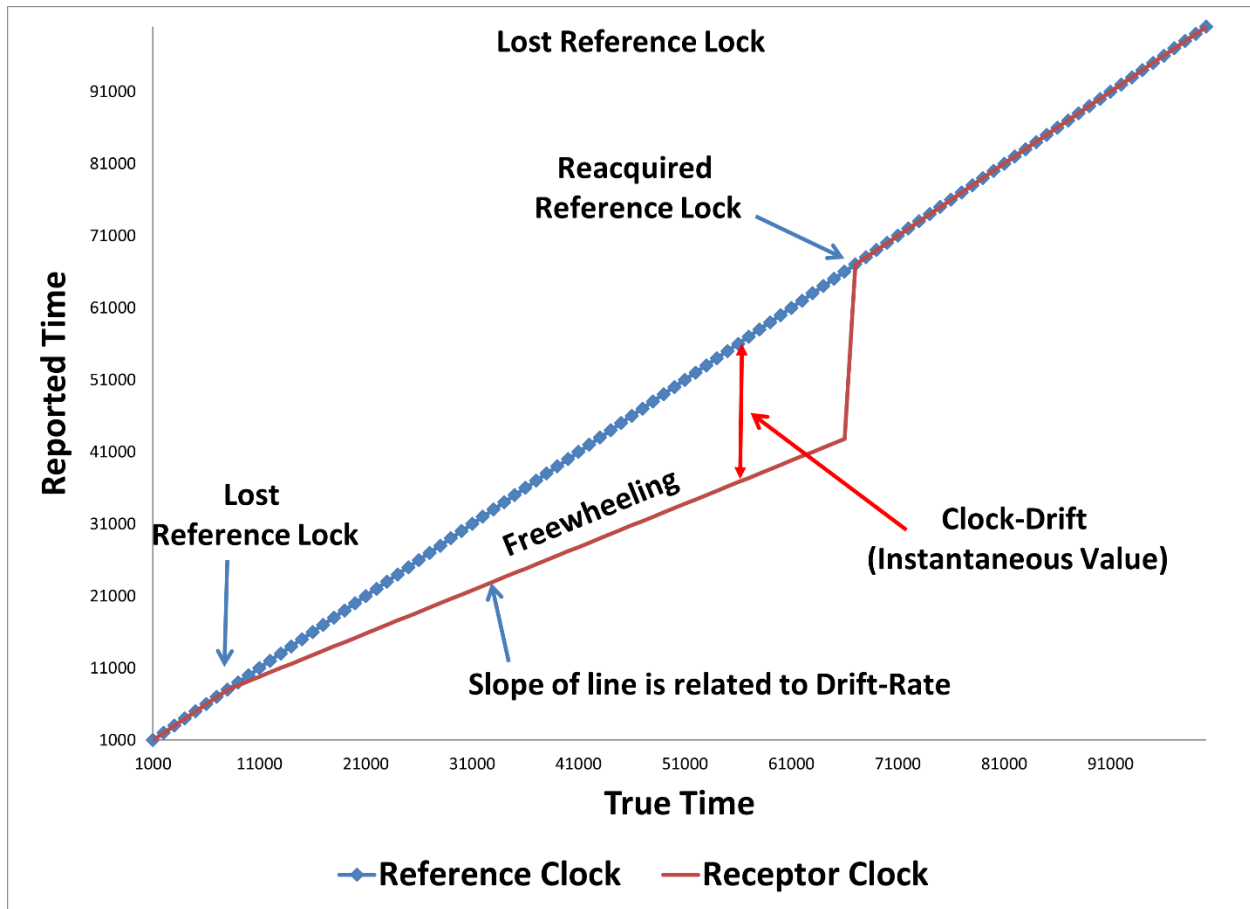


Figure 45 illustrates two clocks, a reference clock in blue and a receptor clock in red, measuring time in milliseconds. The bottom axis is true time (i.e., reference time), while the left axis reports time from the two clocks. At the origin, (1,000 milliseconds or one second) the reference clock is synchronized with the receptor clock. The receptor clock loses lock to the reference time at 9,000 milliseconds. At this point, the receptor clock is freewheeling and drifts away from the reference clock with a constant drift-rate until lock is reacquired at 66,000 milliseconds.

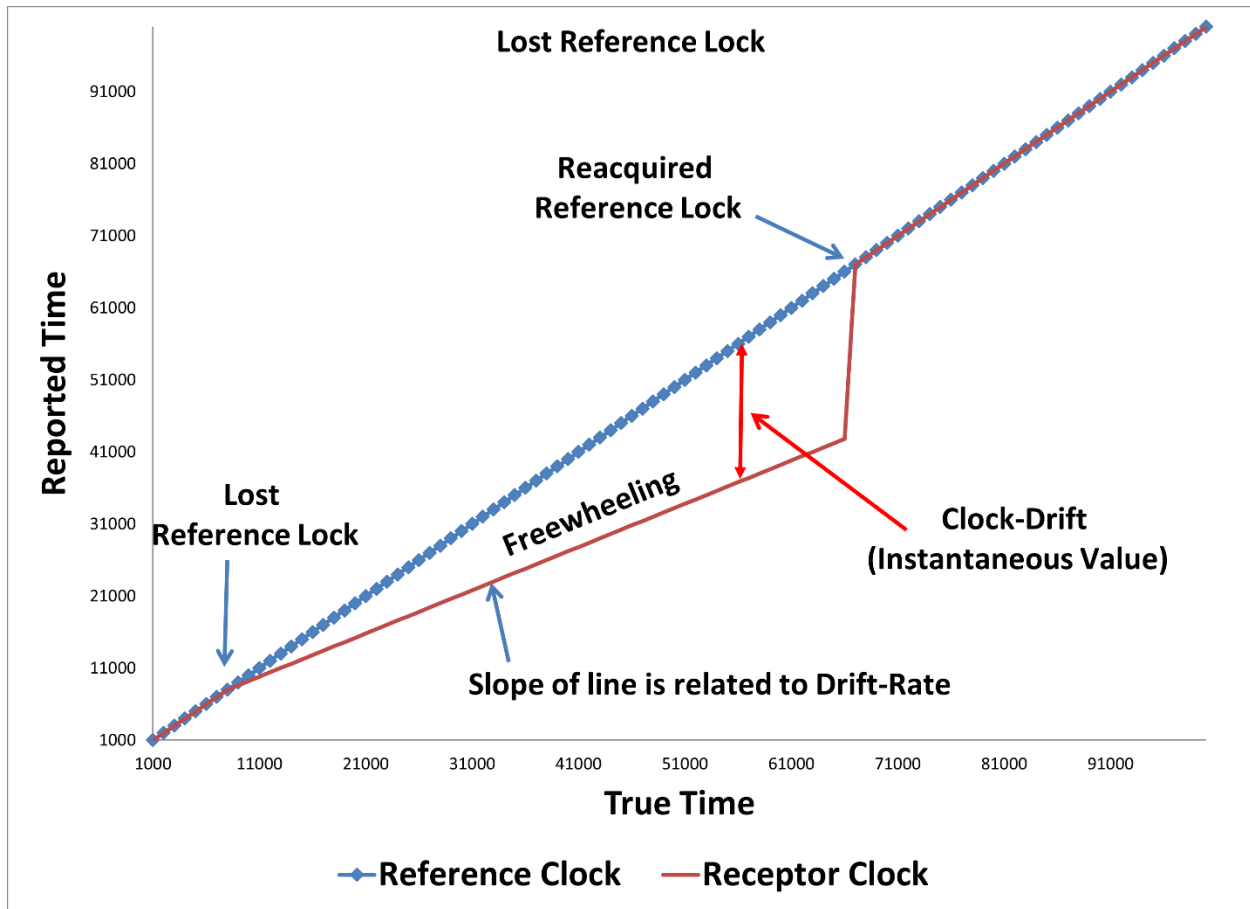


Figure 45: Illustration of Clock Synchronization

The receptor clock is in one of two states relative to the reference clock: locked or unlocked. In the locked state, the receptor time is reporting an equivalent value to the reference time, within some statistical error. In the unlocked state, the receptor clock is reporting its own internal time, which drifts away from the reference time with a device-specific drift-rate. The drift-rate has one of three value ranges: equal-to-one, less-than-one, or greater-than-one. A drift-rate equal-to-one indicates the receptor clock matches the reference clock when freewheeling.

A drift-rate less-than-one indicates the receptor clock is slower than the reference time; this is the condition shown in

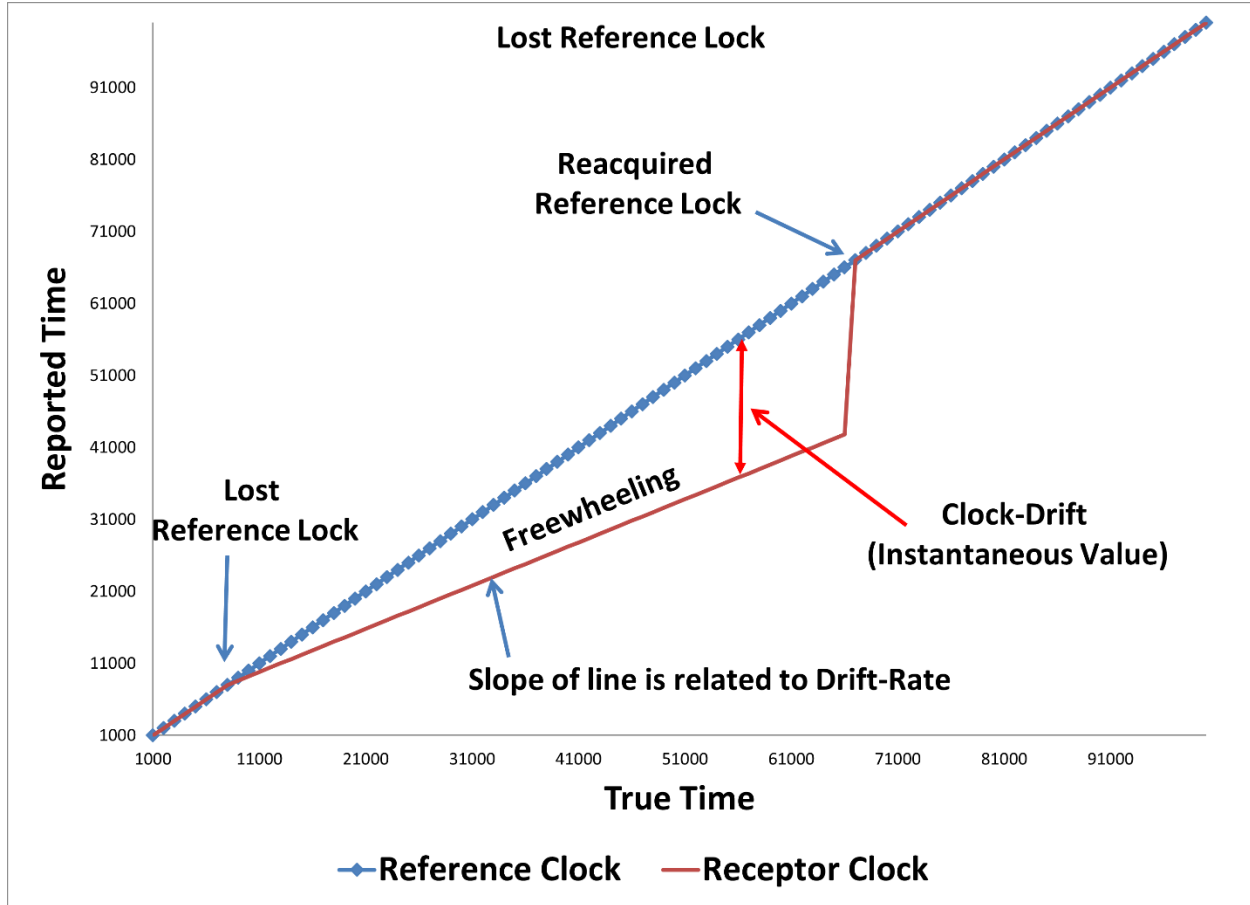


Figure 45. Unless the receptor clock is an atomic clock, when the reference lock is lost the receptor clock changes from a Level 3 to a Level 1 Time System (on the Timing System Capability Levels – see Section 6.2.1); however, adjustments can be made to maintain a Level 3 Time System either in real time or in post processing. When reference lock is reacquired, a slower receptor clock correction occurs with a single time value update, which results in a rapid

change to the time value. Table 18 shows the time values from

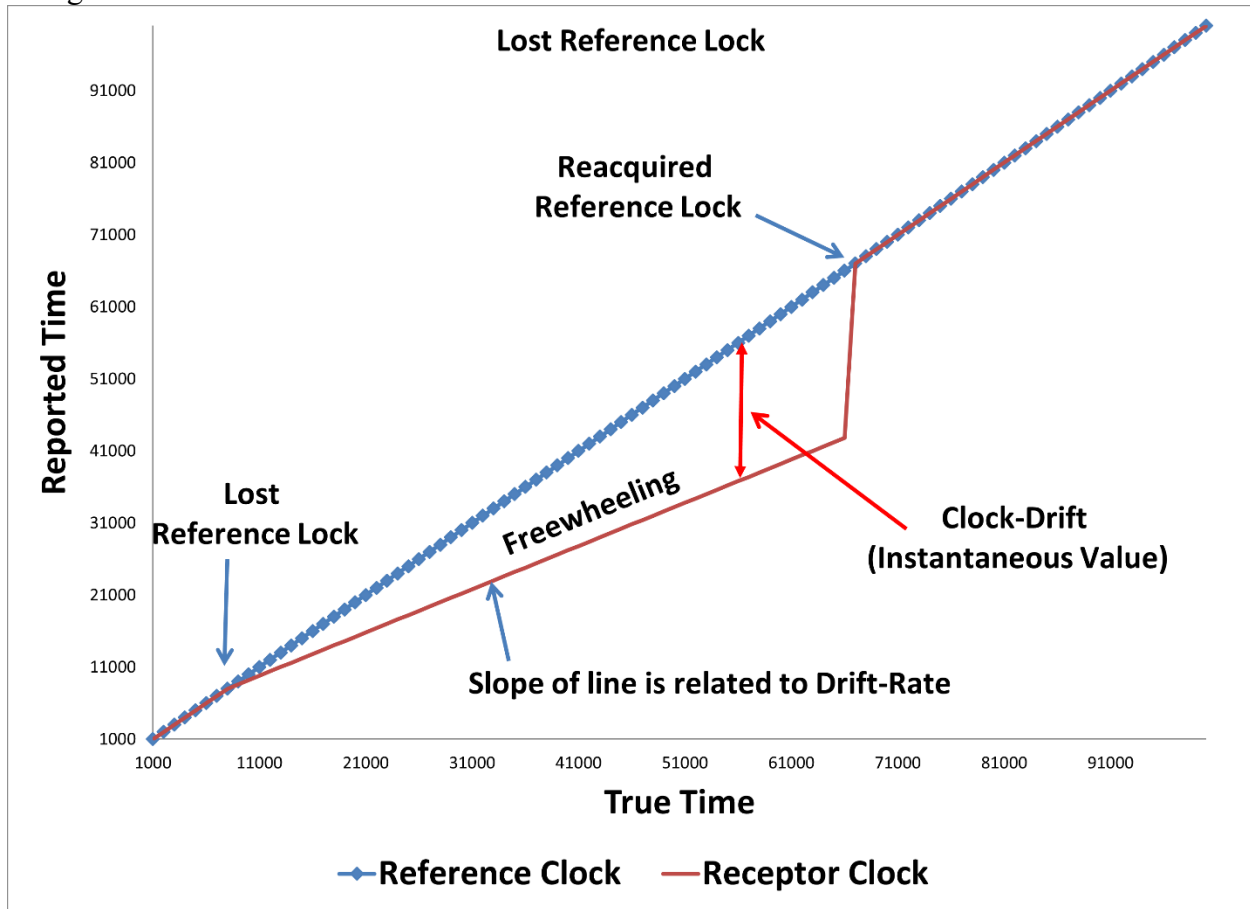


Figure 45 before and during the reference lock being reacquired. Table 18's first column is the value from the Reference Clock; the second column is the difference between the rows of Reference Clock values, one thousand milliseconds or one second; the third column is the Receptor Clock's reported value; the fourth column is the difference between the rows of Receptor Clock's values, which is 600 milliseconds during the unlocked time, which shows the amount of error every second of Reference Clock's time; the final column is the locked/unlocked state of the clocks.

Table 18: Slow receptor clock reacquiring the reference clock. The yellow highlight indicates the large jump forward in the receptors time.

Reference Clock	Reference Difference	Receptor Clock	Receptor Difference	State
62,000		40,400		
63,000	1,000	41,000	600	unlocked
64,000	1,000	41,600	600	unlocked
65,000	1,000	42,200	600	unlocked
66,000	1,000	42,800	600	unlocked
67,000	1,000	67,000	24,200	locked
68,000	1,000	68,000	1,000	locked
69,000	1,000	69,000	1,000	locked

This rapid change is allowable because the receptor's time remains strictly increasing, and therefore, maintaining total order (i.e., Level 1 Time System). After the reference lock is reacquired, the receptor clock changes back to a Level 3. Knowing the unlock time, the re-lock time and the final clock-drift, users can compute the drift-rate, and a correction made in post processing to bring the clock back to Level 3. If the measured drift rate is known a priori, through testing or other means, users can make real time corrections to keep the receptor clock at Level 3.

When the drift-rate is greater-than-one, the receptor clock is counting time faster than the reference time. When the reference is reacquired, the correction for a faster receptor clock is not a simple update. In this case the reported time would jump backwards in time causing a discontinuity, and therefore, the clock would not be a Level 1 clock. This type of correction is a Jam Correction. Table 19 lists example values for a Jam Correction and Figure 46 illustrates the graph which shows the negative jump in time.

Table 19: Fast receptor clock reacquiring the reference clock and adjusting the reported time incorrectly. The yellow highlight indicates a large jump backwards in the receptors time and the red highlights the discontinuity.

Reference Clock	Reference Difference	Receptor Clock	Receptor Difference	State
62,000		83,200		
63,000	1,000	84,600	1,400	unlocked
64,000	1,000	86,000	1,400	unlocked
65,000	1,000	87,400	1,400	unlocked
66,000	1,000	88,800	1,400	unlocked
67,000	1,000	67,000	-21,800	locked
68,000	1,000	68,000	1,000	locked
69,000	1,000	69,000	1,000	locked

To maintain a Level 1 clock the best method is to increment the receptor clock one unit (e.g., one millisecond in these examples) per time “tick” until the reference clock catches up to the receptor clock. The process of slowing the receptor clock down is “Slewing”, so this type of correction is a Slew Correction. When the receptor clock is equal to or less than the reference clock, it can resynchronize to the reference clock as shown in Figure 47.

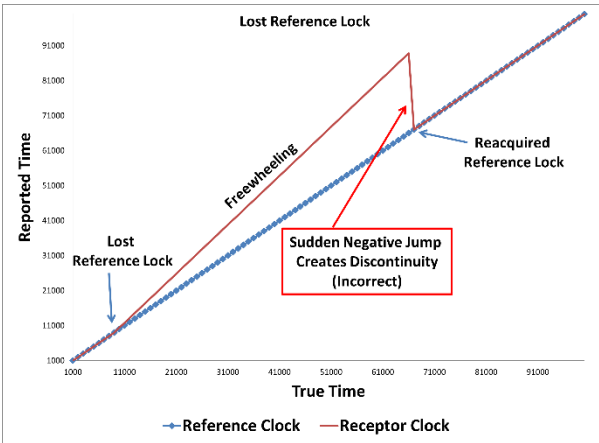


Figure 46: Incorrect method to reacquire lock to Reference Clock

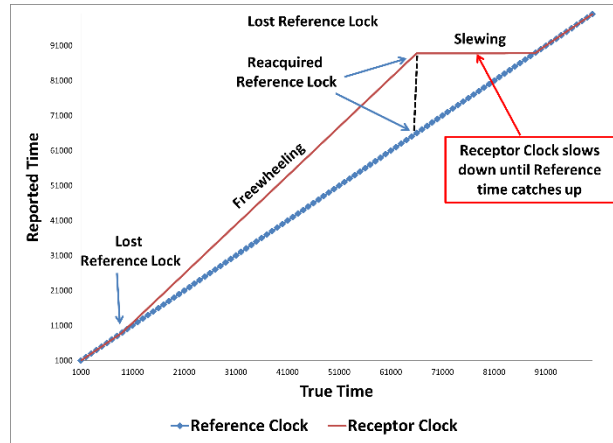


Figure 47: Correct method to reacquire lock to Reference Clock

Table 20 lists the values for the slew correction from the point of reacquiring the lock to resynchronization.

Table 20: Fast receptor clock reacquiring the reference clock and adjusting the reported time for a slew correction. The receptor’s time increments slowly (1 millisecond/tick) while the reference clock catches up. The highlighted yellow shows where the lock is reacquired, and the blue highlight shows where the re-synchronization occurs.

Reference Clock	Reference Difference	Receptor Clock	Receptor Difference	State	Synchronized
62,000		83,200			No
63,000	1,000	84,600	1,400	unlocked	No
64,000	1,000	86,000	1,400	unlocked	No
65,000	1,000	87,400	1,400	unlocked	No
66,000	1,000	88,800	1,400	unlocked	No
67,000	1,000	88,801	1	locked	No
68,000	1,000	88,802	1	locked	No
69,000	1,000	88,803	1	locked	No
...		...	...		
88,000		88,822	1	locked	No
89,000	1,000	88,823	1	locked	No
90,000	1,000	90,000	1,177	locked	Yes
91,000	1,000	91,000	1,000	locked	Yes

92,000	1,000	92,000	1,000	locked	Yes
93,000	1,000	93,000	1,000	locked	Yes

6.4.1 Time Transfer Metadata

To support the ability to correct the freewheeling time value to the reference time requires metadata about the lock and synchronization status of the clocks. Figure 48 depicts a time transfer system that ensures a Level 3 Time System is possible in post processing.

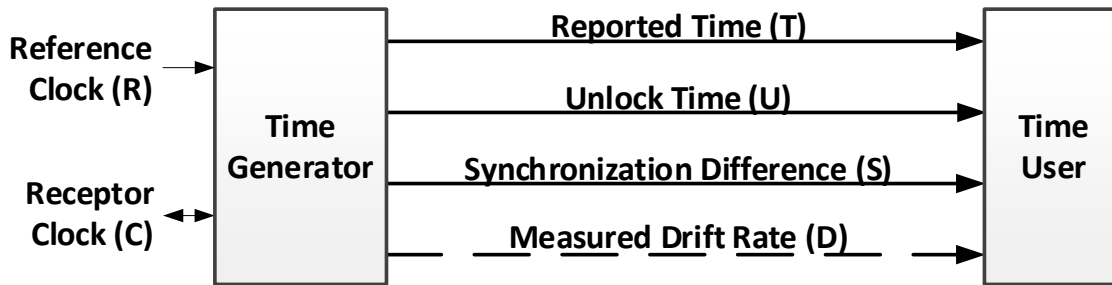


Figure 48: Time Generator

The reference clock (R) generates time values at a well-defined period. Such a clock “pulses” at a continuous, precisely spaced change in voltage, called Pulses Per Second (PPS); for example, GPS produces a one (1) PPS clock period.

R is undefined when unlocked from the reference time. The receptor clock (C) is synchronous with the reference clock when locked (i.e., $C=R$); however, it will freewheel otherwise.

The primary result of the time generator is the reported time (T) which is either locked to R or equal to C. Between reference clock pulses R, the receptor clock C may drift; however, the drift to the reported time T is within required tolerances of the system (otherwise, the system should either use a more frequent (i.e., larger number) PPS, or a better time generator system).

The time generator also produces values used to correct the reported time when the reference clock becomes unlocked. The Unlock Time (U) and Synchronization Difference (S) provide the values for the user to correct the time in post processing. When the Measured Drift Rate (D) is known a priori, users can correct the reported time in real time.

The Unlock Time U is the duration of time measured in the receptor time’s time-frame units since the last locked value; if the reference clock R never loses lock, then U will always be zero. The Synchronization Difference S is the last known time difference between receptor clock C and reference clock R once resynchronization occurs; if $C=R$ (i.e., R locked and synchronized to C), then $S=0$; otherwise, $S \geq 0$.

Given T, U, and S, users can remove the clock drift during post processing using Equation 8. Alternatively, when D is known, users can remove the clock drift in real-time.

Figure 49 shows an example where the time generator loses its reference clock lock twice. In this example, the second lost lock event occurs during slew correction of the first lost lock.

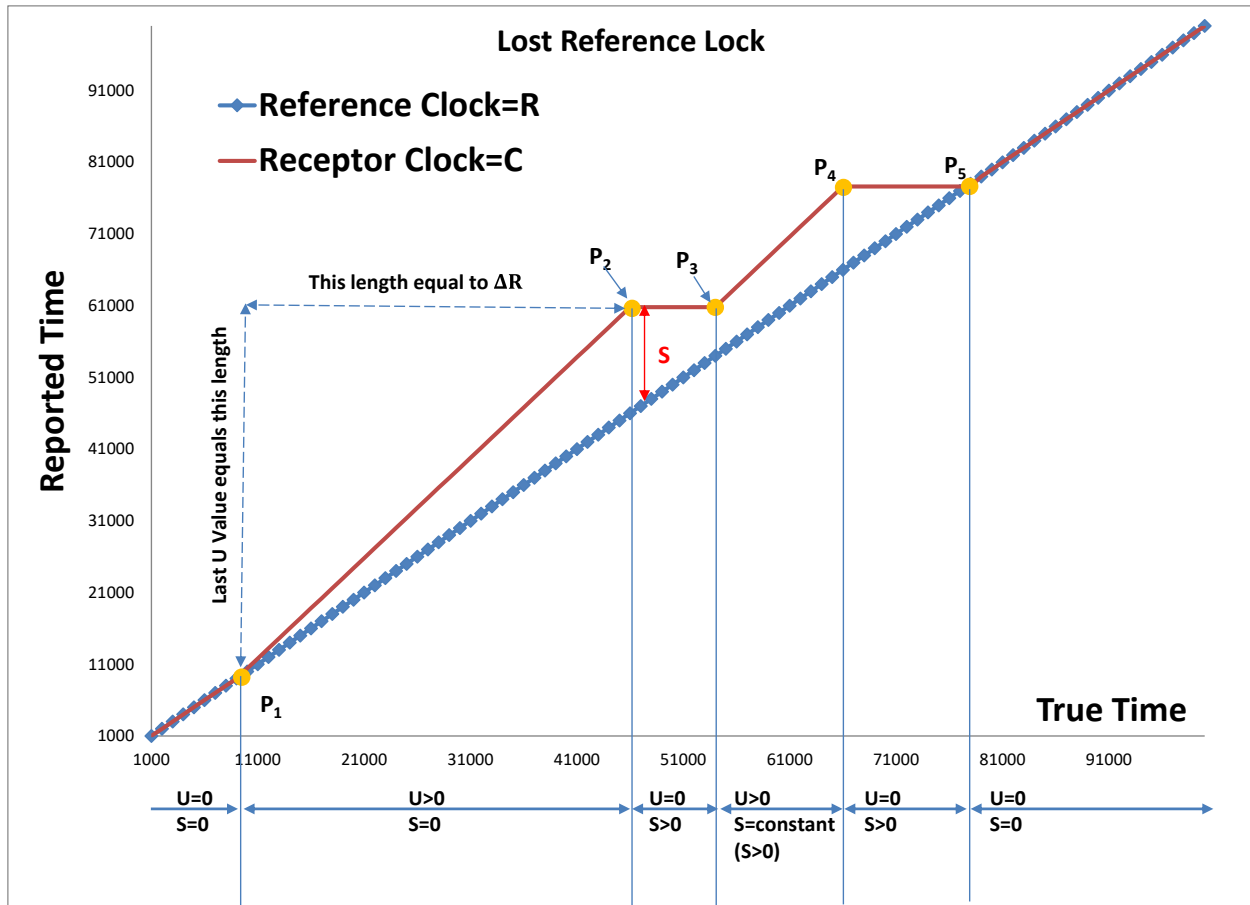


Figure 49: Example of losing reference lock twice, with the second loss during a time-convergence

The figure shows five instances in time, labeled as $P_1 \dots P_5$, where the lock/unlock status changes. Table 21 lists the values for R , C , T , U and S at each of these times along with the preceding time; the subscript “a” denotes the reported time immediately before the change occurs. The value $C_{2a} + 1$ means one time unit (e.g., ms or ns) after C_{2a} , this is a slew correction value.

Table 21: Points of change in Figure 49

	P _{1a} locked	P ₁ unlocked		P _{2a} unlocked	P ₂ locked		P _{3a} locked	P ₃ unlocked		P _{4a} unlocked	P ₄ locked		P _{5a} locked	P ₅ locked
	freewheeling				slewing				freewheeling				slewing	
R	R _{1a}	N/A	...	N/A	R ₂	...	R _{3a}	N/A	...	N/A	R ₄	...	R _{5a}	R ₅
C	C _{1a} =R _{1a}	C ₁	...	C _{2a}	C _{2a} +1	...	C'+1	C ₃	...	C _{4a}	C _{4a} +1	...	C'+1	C ₅ =R ₅
T	C _{1a}	C ₁	...	C _{2a}	C _{2a} +1	...	C'+1	C ₃	...	C _{4a}	C _{4a} +1	...	C'+1	C ₅
U	0	C ₁ -C _{1a}	...	C _{2a} -C _{1a}	0	...	0	C ₃ -C _{3a}	...	C ₄ -C _{3a}	0	...	0	0
S	0	0	...	0	C _{2a} +1-R ₂	...	C'+1-R _{3a}	C'+1-R _{3a}	...	C'+1-R _{3a}	C _{4a} +1-R ₄	...	C'+1-R _{5a}	0
											S ₃ to S _{4a} is constant value			

The area marked as slewing shows where the receptor clock is slowly incrementing (by one unit i.e., micro or nanoseconds) until the reference clock catches up. The value C' is the previous clock's value in the figure during slewing. The shaded region on the "R" row shows where the reference clock is not available. The reference and receptor clocks values are separate because the time user only has access to the T, U and S values.

The following equations compute time corrections made in post processing by monitoring two successive T, U, S sets preceding a re-lock of the clock and a delta difference (see Δ_{2a} below); without the need to maintain other historical data.

To compute a correction to C when it is freewheeling between P₁ and P₂, compute the slope of the line from P₁ to P₂. The slope of the line is the ratio between the difference in receptor clock times ($C_2 - C_{1a}$) to the difference in reference clock ($R_2 - R_{1a}$). The actual value for when reference clock unlocks is unknown (some instant between two pulses), but the values immediately before the unlock are known (i.e., C_{1a} and R_{1a}). The value C_2 is unknown because when the lock is reestablished the receptor clock starts its slow correction; however, an estimate for C_2 is $C_2 \approx C_{2a} + \Delta_{2a}$, where Δ_{2a} is the difference of two preceding reported times, i.e., $\Delta_{2a} = T_{2a} - T_{2a-1}$.

The values for R_{1a} and R_2 are unknown by the time user but the time user can compute these values using the unlock time, U, and synchronization difference, S. Just before relocking, the reported time is $T_{2a} = C_{2a}$ and the unlock time is $U_{2a} = C_{2a} - C_{1a}$. When the relock happens the reported (slow corrected) time is $T_2 = C_{2a} + 1$ and the synchronization difference value is $S_2 = C_{2a} + 1 - R_2$. Since $R_{1a} = C_{1a}$ the value for $R_{1a} = T_{2a} - U_{2a}$. The value of $R_2 = T_2 - S_2$. Equation 3 combines all these equations:

$$slope = m = \frac{C_2 - C_{1a}}{R_2 - R_{1a}} \approx \frac{C_{2a} + \Delta_{2a} - C_{1a}}{R_2 - R_{1a}} = \frac{U_{2a} + \Delta_{2a}}{(T_2 - S_2) - (T_{2a} - U_{2a})} \quad \text{Equation 3}$$

The estimated corrected value for the i^{th} time, R_i , within the freewheeling interval between P₁ and P₂ is linear function of the reported time, T_i , using the slope, m, and the starting point when the clocks became unlocked. Equation 4 and Equation 5 shows this relationship:

$$C_i - C_{1a} \approx m * (R_i - R_{1a}) \quad \text{Equation 4}$$

$$R_i \approx \frac{C_i - C_{1a}}{m} + R_{1a} = \frac{C_i - C_{1a}}{m} + C_{1a} = \frac{T_i - (T_{2a} - U_{2a})}{m} + (T_{2a} - U_{2a}) \quad \text{Equation 5}$$

When C is freewheeling between P₃ and P₄, the correction has an additional offset from the slewing difference. In a similar fashion to Equation 3, Equation 6 defines the slope and Equation 7 compensates for the vertical offset using the S_{4a} value (which is equal to S₃).

$$\text{slope} = m \approx \frac{C_{4a} + \Delta_{4a} - C_{3a}}{R_4 - R_{3a}} = \frac{U_{4a} + \Delta_{4a}}{(T_4 - S_4) - (T_{4a} - U_{4a})} \quad \text{Equation 6}$$

$$C_i - C_{3a} \approx m * (R_i - R_{3a}) + S_{4a} \quad \text{Equation 7}$$

$$R_i \approx \frac{C_i - C_{3a} - S_{4a}}{m} + R_{3a} = \frac{T_i - (T_{4a} - U_{4a}) - S_{4a}}{m} + (T_{4a} - U_{4a}) \quad \text{Equation 8}$$

Equation 6 and Equation 8 are the general case equations used for all cases. In the first example when freewheeling between P₁ and P₂, the value of S_{2a} equals zero, so Equation 7 resolves into Equation 4.

If the time generator knows the receptor clock's drift rate, the time generator will include the Measured Drift Rate, D. The value D is a measure of nanoseconds of drift per second (ns/s) and may be a positive or negative value. The value D is a measure relative to the diagonal line in Figure 49; therefore, the slope, m, is the addition of the diagonal line slope (equal to 1) and the value D. Equation 9 provides the equation for the slope when the Measured Drift Rate, D, is known.

$$\text{slope} = m = D + 1 \quad \text{Equation 9}$$

With this slope value computed from D, Equation 8 can be applied in real time before a relock event.

Chapter 7

Metadata

Metadata is data about data. The Motion Imagery Standards Profile (MISP) uses metadata to characterize both visual and non-visual information about collected Motion Imagery. These characteristics include the environment of Motion Imagery sensors, the kinematics of the sensors, timing, object information, classification and much more.

There are many ways to organize and format metadata, each with advantages and disadvantages. One efficient metadata encoding method used across many MISB standards is the Key-Length-Value (KLV) data encoding standard defined in SMPTE ST 336 [14]. In many government systems, the available delivery bandwidth from a sensor to a user is constrained. This warrants a tradeoff in the quality of the Motion Imagery versus bandwidth allocated for metadata. The MISB extends SMPTE ST 336 with additional rules and structure to further reduce the quantity of bytes needed to represent various metadata elements; this translates directly into reduced bandwidth for data exchange. To this end, the complexity in extending KLV encoding over that prescribed by SMPTE justifies this intent.

7.1 Metadata Theory

Sensors (e.g., cameras) create raw data or *essence* [15]. In a Motion Imagery System, one essence is the visual Motion Imagery; another collaborative essence is audio. Such essence types are far more meaningful with context, which comes in many forms, such as what, where, why, or how an essence is created. The context may be apparent based on the familiarity of the essence; often it is not. To provide users the ability to definitively understand an essence, contextual information is supplied; this contextual information is metadata.

Contextual information about essence can **expand** and **change** as more effort is applied to understanding the data. Expansion can result from adding context from data sources associated with the collection of the essence (e.g., gimbal angles, timing, and GPS position); adding context by performing analysis; or reusing a contextual dataset from other sources. Changes can be attributed to corrections or improvements of measurements, removal of un-needed data, or other functions.

The process of expanding and changing the contextual information forms a life cycle. This life cycle is important to document as it provides traceability to events which occurred to the information; such traceability may be critical in forensic analysis of the data. Therefore, the history of changes needs to be recorded; this history information is called Provenance.

In Figure 50, Item 1 shows an Essence with its Collection Metadata. Item 2 illustrates potential life-cycle changes, where processing operations on the Collection Metadata, Essence or both produces Processed Metadata. Further data exploitation (either human or machine) may produce additional Analysis Metadata.

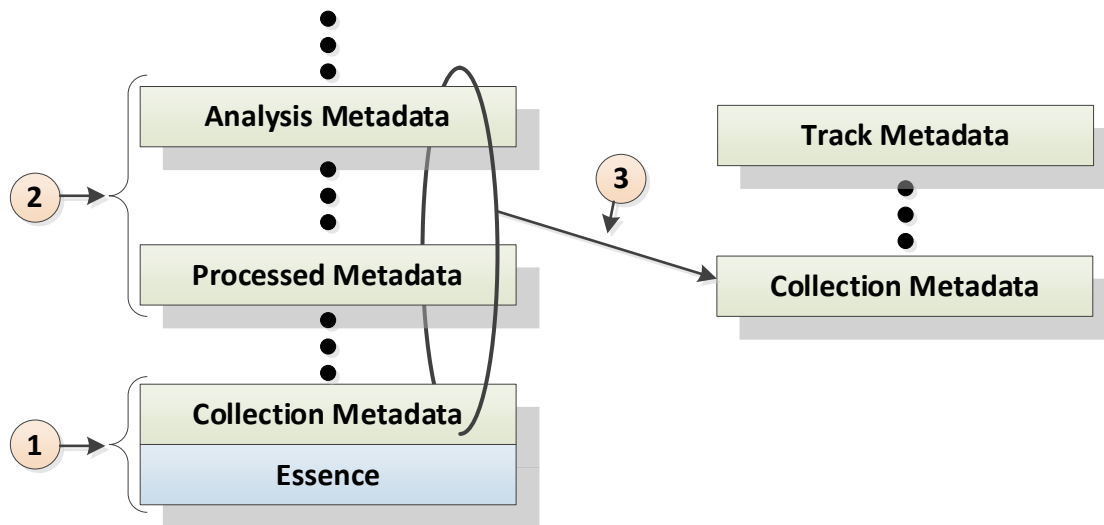


Figure 50: Metadata Creation, Life-Cycle and Spawning

As contextual information evolves in its life cycle, it can create new independent or derived metadata. The goal of any analysis process is to reduce data to a set of suppositions, or ideally, facts. Suppositions rely on supporting information, or a link back to originating data used to create the suppositions; this link is Provenance Metadata as traced back to the data's source.

For example, suppose the goal is to track vehicle movement within a scene. A system collects Motion Imagery along with its associated metadata (such as the Motion Imagery's sensor position, movement, etc.). Tracking software detects and tracks a vehicle producing new track metadata. Extracting and evaluating the track information (suppositions) independent of the Motion Imagery promotes the track information to an independent data source; in this case, the track information links to or references the source i.e., the captured Motion Imagery. Item 3 in Figure 50 illustrates spawning a new data source from existing metadata information, where a subset of the originating metadata forms a new data source independent of the original motion imagery.

7.1.1 Metadata Streams

Since Motion Imagery is an ordered series of images, the supporting contextual information is also an ordered series of metadata packets. Each metadata packet contains a collection of data elements with a pre-defined organization (see Section 7.1.2). There are two methods for ordering metadata packets: data position or index. Data position order means the physical location within the packet series defines which packet is first, second, third, etc. Index order means there is information within the packet that defines which packet is first, second, third, etc. independent of its physical location. Motion Imagery streams typically order packets using time as the index. The series of metadata packets ordered by data position or index defines a Metadata Stream.

Figure 51 illustrates a series of metadata packets and a pipeline or stream representation of the packets. Each packet increments in time so the ordering is time-based.

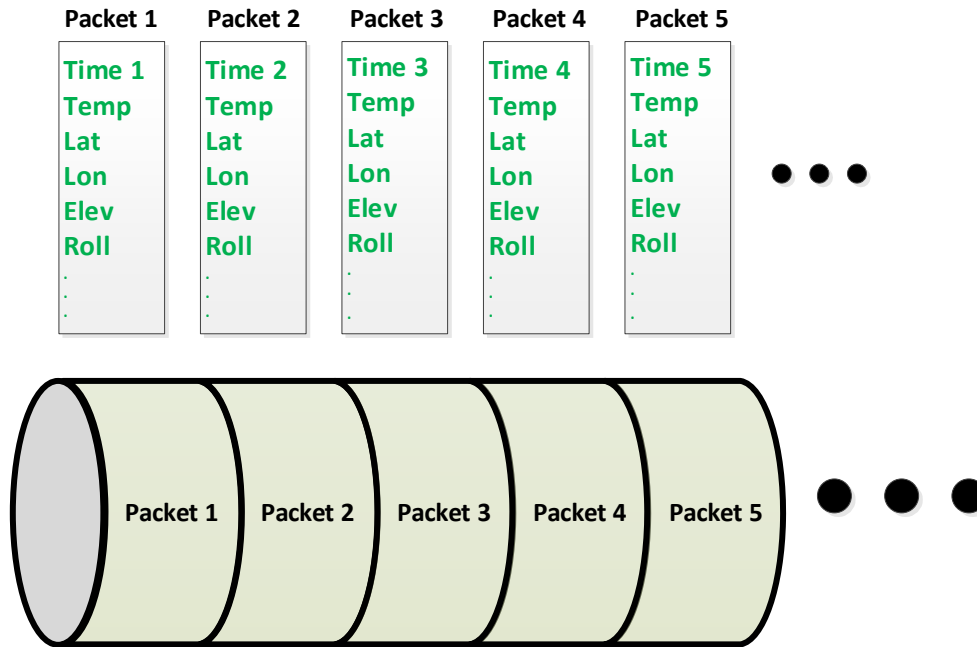


Figure 51: Metadata Stream

Metadata Packet: A collection of data elements.

Data Position Order: A method of ordering Metadata Packets by location within a series of packets.

Index Order: A method of ordering Metadata Packets using a unique index value within the Metadata Packet.

Metadata Stream: A series of Metadata Packets ordered by Data Position Order or Index Order.

7.1.2 Metadata Information

As discussed, metadata is data about essence. Metadata are individual data elements of information providing either context for the essence, or context to the metadata itself. A data element has three parts: **value**, unique **identifier**, and **definition**. The value is the data associated with the essence. The identifier is the name of the data element and links the value to its definition.

A *controlling document* – separate from the essence – defines the meaning of the value in a definition. For example, a data element *identifier* “Sensor Height Above Ellipsoid (HAE)” has a *value* of “100,” and its *definition* is “a value of meters from the WGS-84 ellipsoid to height of the sensor above the ground.” The identifier and its definition for this data element exists in a controlling document, while the identifier and its value exist as metadata for its essence (i.e., Motion Imagery). The controlling document provides the *specification* for the metadata (i.e., meaning, form and rules), while the metadata itself is an *instance* of that specification. The metadata, as an instance of the specification, is binary data.

The example in Figure 52 further clarifies the relationship between a controlling document and an instance of a metadata element. On the left side of Figure 52, a sensor produces Essence per the rules of a chosen standard (i.e., Essence Standard), such as H.264/AVC. Supporting Devices to the sensor provide measurements, which become the sensor's metadata. The meaning, format and rules for constructing the metadata follow the specification as defined in a controlling document (i.e., metadata specification). This creates a Metadata Instance linked to the Essence. The process for creating the metadata “invokes” the controlling document when making the Metadata Instance. The Essence and Metadata Instance are then combined, where now Users, who reference the same Essence Standard and controlling document, can understand how to process and view the sensor essence and understand the meaning of the Supporting Devices metadata.

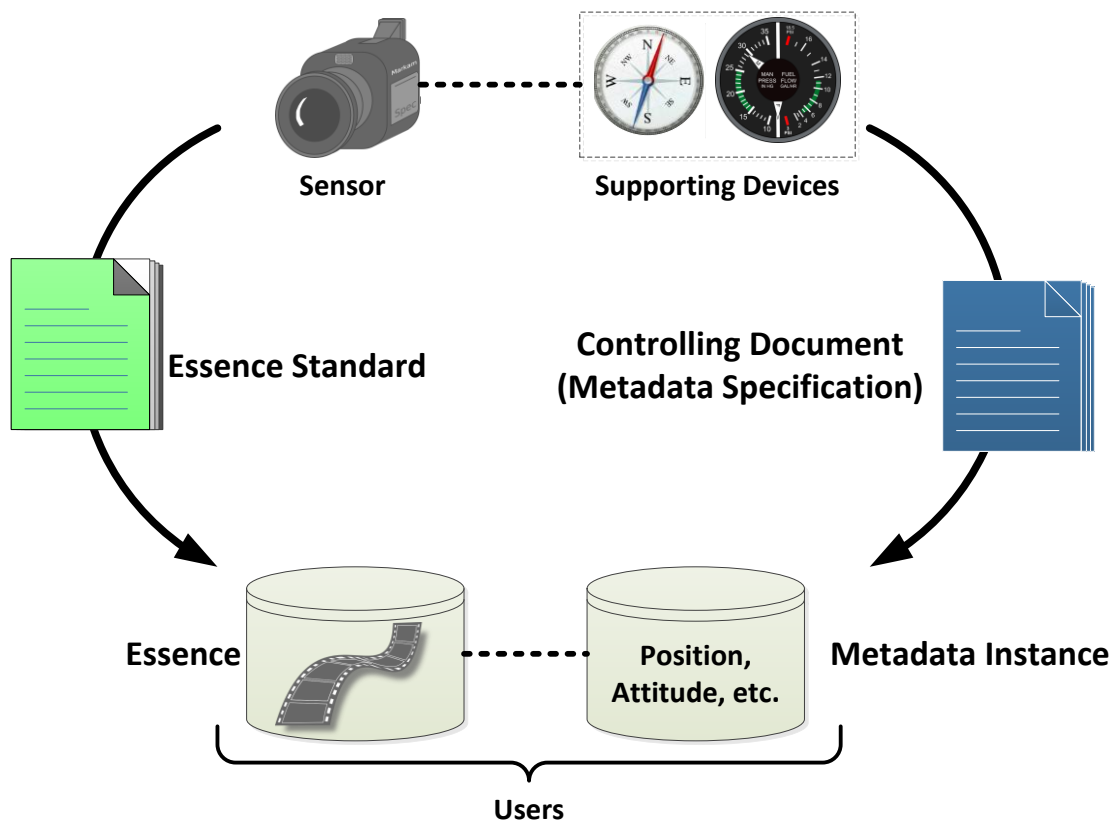


Figure 52: Relationship between a Metadata Specification and its Instance

A controlling document is typically cast into a standard, which provides both a metric for conformance and documented revision control. Metadata standards can use other metadata standards, whereby one standard invokes another standard.

There are multiple ways to group and organize metadata elements. Three common methods for organizing metadata are **Lists**, **Trees** and **Graphs**. The MISP standards directly employ both Lists and Trees; the standards indirectly use Graphs via data references within Lists and Trees.

7.1.2.1 Lists

Lists are the most basic method of organizing a group of data elements. A list is a named group of elements, which is either ordered or unordered. An ordered list requires its data elements arranged in a specific sequence. The first column of Table 22 shows an example of an ordered list; the numbers indicate a required ordering. The second column shows an unordered list of the same information, but with random ordering (thus, with no numbers).

Table 22: Example of Ordered List and Unordered List

Ordered List	Unordered List
1. Temperature	Sensor Longitude
2. Sensor Latitude	Temperature
3. Sensor Longitude	Sensor Latitude
4. Sensor Height Above Ellipsoid (HAE)	Sensor Type
5. Sensor Type	Sensor Height Above Ellipsoid

An ordered list reduces the amount of information needed to convey metadata elements. A controlling document for the list defines the elements and the order of each element in the list. Individual data element identifiers in the list do not need to be represented, thereby saving bytes. In contrast, in an unordered list, where the order is not defined, or known, each data element identifier must be included to understand what each data element means. The advantage of an unordered list is flexibility. Thus, there is a tradeoff in data quantity versus flexibility between the two types of lists.

Figure 53 pictorially illustrates the difference between the two types of lists. The ordered list maintains its identifiers for its data elements in a controlling document along with a prescribed order to the data elements, while the unordered list does not. Because the identifiers are not present in the metadata, the ordered list consumes less bytes. Both types of lists do require a description (or definition) of each data element to understand what its values represent. The definitions are documented in a controlling document, which includes qualifiers such as units (SI measures), type and format – all necessary to properly interpret a data element’s value.

The information in the ordered list of Figure 53 is five data values, while the unordered list has five identifiers *plus* five data values. When using the entire list, an ordered list conserves bytes. For example, when Temperature and Sensor Type are the only two data elements specified, the ordered list will still have five data values (the other values would contain “filler”). The unordered list, on the other hand, would have two identifiers and two data quantities, and thus require fewer overall bytes.

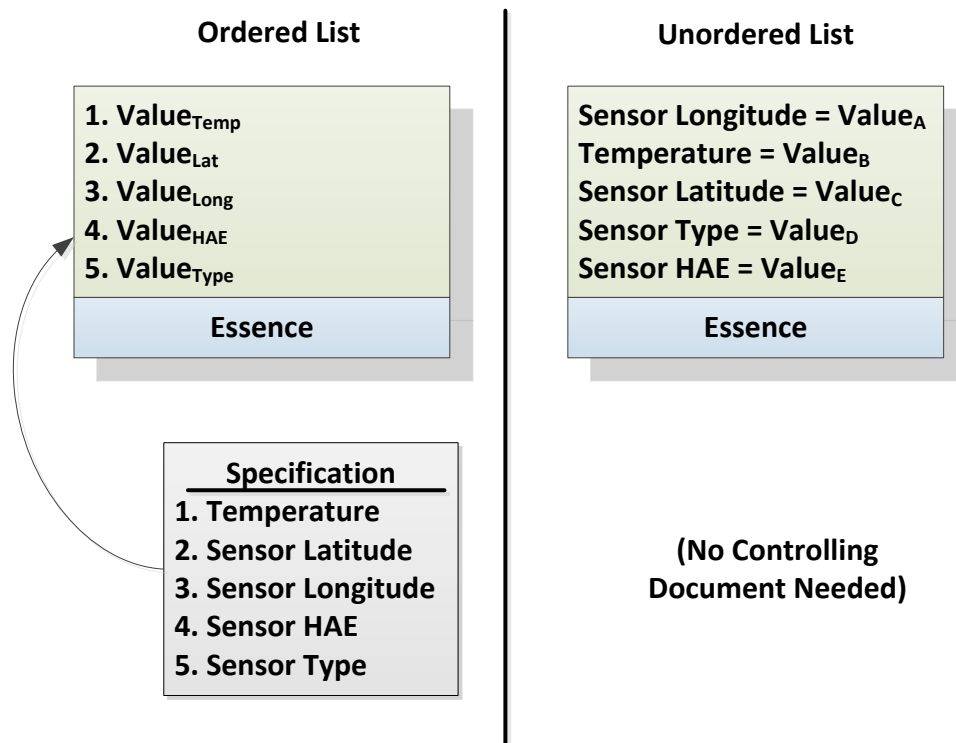


Figure 53: Ordered List and Unordered List Examples

7.1.2.2 Trees

Lists work well in many situations, but when the same data element is needed for different purposes within the same list, another structure is better suited. For example, consider two sensors, with each sensor having a position (Sensor Latitude, Sensor Longitude and Sensor HAE) and Sensor Type. When referencing “Sensor Latitude” in a list, it is unclear to which sensor it refers. Lists could be forced to support these situations by either writing a controlling document to support duplicates or creating names such as “Sensor Longitude for Sensor 1”; however, this is cumbersome and inflexible for a system with many sensors.

A **Tree** structure enables parent/child relationships, where the top-level parent is the “*root*” and a child is a “*branch*” of the tree. In hierarchical fashion, a child can likewise have one or more children, thus making it a parent as well. A Tree structure offers a means to collect a group of information into its own list, and then include that list as a single item in a parent list. For instance, Table 23 shows an example of two different metadata (ordered) trees for the same two sensors.

On the left, at the top of the tree (i.e., parent or root) are the data elements “Temperature,” “Sensor 1,” and “Sensor 2.” This parent list of data elements contains two sub-lists (i.e., children or branches): the data elements “Sensor Latitude” through “Sensor Type” belonging as children to the parent “Sensor 1,” and similarly, the data elements “Sensor Latitude” through “Sensor Type” belonging to the parent “Sensor 2.” Note that each parent contains a sub-list of the same data element identifiers. Although the names are the same, the values for the two sets of identifiers may be different. Using levels to denote relationships, the parent is at *level 1* of the

tree; children beneath the parent are at *level 2*, and so forth. The data elements in the left column thus represents a two-level tree.

The right column of Table 23 shows an example of three-level tree. The root contains the data elements numbered 1, 2, 3; the children at level 2 contain the elements labeled “a”, “b”, and “c”; the children at level 3 contain the elements labeled “i”, “ii”, and “iii”. Parent or children lists can be either ordered or unordered lists.

Table 23: Example of a Tree Structure

Two Sensor Metadata	
Two-Level Tree	Three-Level Tree
1. Temperature 2. Sensor 1 a. Sensor Latitude b. Sensor Longitude c. Sensor HAE d. Sensor Type 3. Sensor 2 a. Sensor Latitude b. Sensor Longitude c. Sensor HAE d. Sensor Type	1. Temperature 2. Sensor 1 a. Position i. Sensor Latitude ii. Sensor Longitude iii. Sensor HAE b. Sensor Type c. Sensor ID1 3. Sensor 2 a. Position i. Sensor Latitude ii. Sensor Longitude iii. Sensor HAE b. Sensor Type c. Sensor ID2

7.1.2.3 Graphs

Graphs enable relationships beyond a top down or bottom-up relationship. Graphs are networks of elements, with any element linked to any other element. The MISP currently does not directly use Graphs but may do so in the future.

7.1.3 Metadata Substreams

Depending on the organization of the Metadata Packet (i.e., a tree structure), a Metadata Stream can contain one or more metadata substreams. The metadata packet’s list of elements is the root of a tree structure, which can include children elements containing their own well-defined sub-list of elements. A sub-list is a named group of elements embedded in another list. A child sub-list can include a unique identifying method to distinguish the child from any other children in the packet. When the identifier for the child is consistent from packet-to-packet, the series of packet children forms a substream within the Metadata Stream.

Table 24 shows an example of two Metadata Packets using an unordered tree structure with two sensor elements. The packets contain information about two sensors, one in green and one blue. Because each sensor’s data is in a well-defined sub-list (i.e., Sensor child) and each sub-list has a unique id (i.e., ID1 and ID2 in red) to link each child together from packet-to-packet, the Sensor

1 child elements define a substream of the Metadata stream, and the Sensor 2 child elements define a second substream of the Metadata stream.

Table 24: Metadata Substream Example

Metadata Packet 1	Metadata Packet 2
Temperature Sensor Position Sensor Latitude Sensor Longitude Sensor HAE Sensor Type Sensor ID1 Sensor Position Sensor Latitude Sensor Longitude Sensor HAE Sensor Type Sensor ID2	Temperature Sensor Position Sensor Latitude Sensor Longitude Sensor HAE Sensor Type Sensor ID2 Sensor Position Sensor Latitude Sensor Longitude Sensor HAE Sensor Type Sensor ID1

Figure 54 illustrates two substreams within a Metadata Stream. Each packet contains a well-defined sub-list and a constant identifier (i.e., ID1 or ID2).

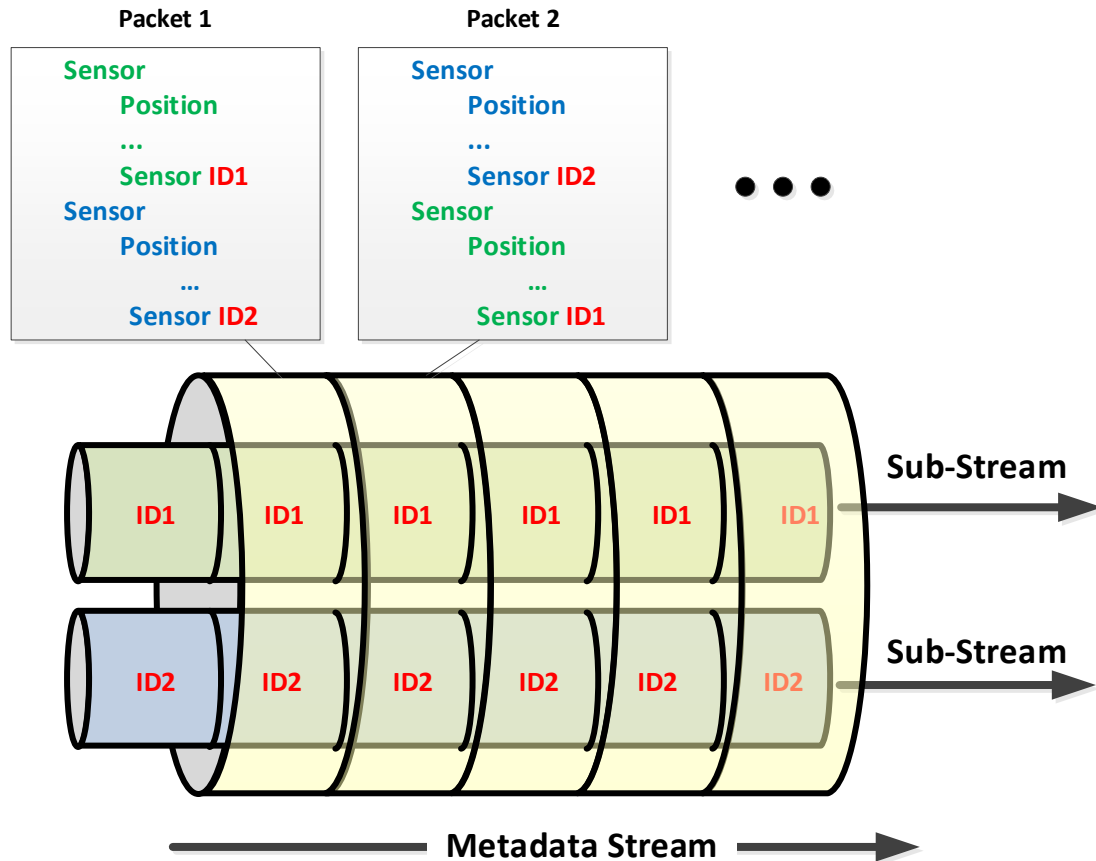


Figure 54: Metadata Substreams

The substream concept is recursive, meaning a substream can also contain substreams. Given a substream as the parent, a child substream of the parent substream follows the same definition as a substream: the child is a well-defined sub-list of the parent's element list and the child's sub-list contains a uniquely identifying method. Every substream must be a well-defined child sub-list of elements, and it must contain a uniquely identifying method.

Metadata Substream: A series of Metadata Packets containing a defined sub-list of elements and the sub-list includes a unique identifying method consistent from metadata packet to metadata packet.

7.1.3.1 Metadata Substream Identifier (MSID)

The purpose of a Metadata Substream Identifier (MSID) is to uniquely “name” a sub-list (or child) within a metadata packet. Each child within a metadata packet has a unique MSID but the MSID is consistent (for that child) from packet to packet, see Section 7.1.3.

There are two MSID forms, a local identifier and universal identifier. The local identifier is a serial number, starting at a value of one (1) for the first child and incrementing by one for the next child, etc. The local identifier is bandwidth efficient but at the expense of requiring systems to deconflict local identifiers if more than one process is adding substreams to a metadata stream. In some cases, deconflicting the MSID can be onerous. To prevent conflicts, the MISB

recommends using local identifiers only in bandwidth constrained situations at the sensor/metadata source, i.e., local identifiers generated at or near the “sensor” in real time.

The universal identifier is a generated Universally Unique Identifier (UUID) of sixteen (16) bytes, e.g., 0xc0365a31-49e1-5055-82c3-7a798b97cfe6. Section 7.5.10.1 describes the different methods for creating UUID values. UUIDs act as very large (16 byte) random numbers with very low probability of conflict with other generated UUIDs. For down-stream and post processing the MISB highly recommends the use of the universal identifier to prevent conflicts with local identifiers (i.e., already in the stream or new local identifiers created upstream after defining the same local identifier value downstream).

MISB standards may dictate one or both forms of MSID.

7.1.4 Metadata Operations

Contextual information about essence is not static; it changes during analysis. For example, days after recording Motion Imagery, an algorithm computes new coordinates with error estimates; the system maintains the original metadata while inserting the new coordinates into the metadata set. Motion Imagery metadata users now have the choice of using the original coordinates or the new coordinates.

The metadata lifecycle is the application of a series of operations that occur to the metadata. In this section, a simple set of operations define how metadata expands and/or changes through the subsequent steps of analysis. The metadata operations are: **create**, **add**, **delete**, and **update**.

The **create** operation produces the initial (first generation) group of data elements. The **add** operation augments or appends new data elements to an existing group. The **delete** operation removes data elements from a data element group. The **update** operation changes or substitutes a new value for an existing data element. The update operation appears as a combination of the delete and add operations but using these two operations on an ordered list would change the order of the list, so the delete-then-add operations are not equivalent to the function provided by the update operation. Figure 55 illustrates these operations on a set of data elements.

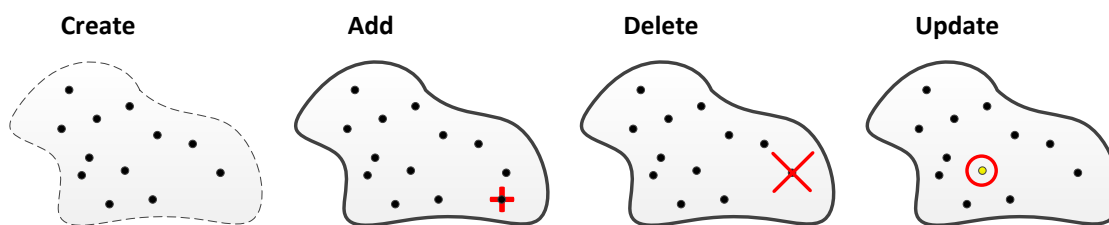


Figure 55: Metadata Operations

7.1.4.1 Data Preservation

In applying the metadata operations, it is critical to not destroy existing metadata. When initially creating metadata, the metadata is “first generation” metadata. Add, delete, and update operations create subsequent generations (second, third, etc.) of metadata. Maintaining all generations of metadata satisfies a legal requirement to “keep all data” (i.e., data preservation) and insures traceability of the essence and metadata back to its first-generation values, if necessary.

Applying delete and update operations compromises data preservation. To perform a delete operation while maintaining data preservation requires keeping the original element and adding a delete “mark”. Likewise, to perform an update operation while maintaining data preservation requires keeping the original element, adding an update “mark”, and adding the replacement value. Operations honoring data preservation are *enhanced* operations. Since an enhanced update operation could happen multiple times, the “marking” methodology must also support updates-of-updates of metadata.

Three methods to “mark” data elements as either deleted or updated include: 1) modifying the data element identifier; 2) adding state information for each data element; 3) or, introducing a list structure called “Amend Metadata”.

1. Modifying the data element identifier presents issues when working at the detailed level of metadata encodings, i.e., changing a Universal Label (Key) to denote a modified data element. This quickly becomes a data management issue, requiring documenting new keys and updating processing equipment. In general, modifying an identifier is not a good solution, especially when updates of prior updates are possible.
2. Supplying state information for each data element requires an additional prefix (or suffix), which systems must process when reading the data value. In an Update or Delete operation, the state information updates to reflect the change. Such an example is assigning a version number for each data element. When updating a data element, the version number for the new data element is the next increment of the current data element’s version number. This requires version numbers for every changed data element. When making groups of changes (i.e., changing several data elements at once), systems must duplicate the state information for each data element, which is inefficient and wastes bytes.
3. MISB uses the “Amend Metadata” list structure for enhanced operations on a metadata group. This structure provides a flexible solution supporting multiple tree levels and adds extra data only when changes occur.

Table 25 provides an example of the enhanced Delete operation, where the “Amend Metadata” list shows the deleted data element (i.e., ‘Delete Sensor Type’). The first column contains the first-generation values; the second column shows a Delete operation with the ‘Sensor Type’ missing from the list; and, the third column shows the enhanced Delete operation, with the Amend Metadata (in red) denoting the deleted element. This method then preserves the deleted data.

Table 25: Example showing the difference of deleting the “Sensor Type” element using the Delete and Enhanced Delete operations for an unordered list

First Generation Values	Delete	Enhanced Delete
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata = ‘Delete Sensor Type’

Table 26 shows an example of how the enhanced Update operation uses Amend Metadata to indicate an element (i.e., ‘HAE’) update. The first column is the first-generation values before the update; the second column shows an Update operation with the ‘HAE’ value changing (in red); and the third column shows the enhanced Update operation, where the Amend Metadata (in red) describes the update. With the enhanced Update operation, the original value of 100 is not changed, but the Amend Metadata indicates the new update of 200 to the ‘HAE’ value.

Table 26: Example showing the difference of updating the “HAE” element using the Update and Enhanced Update operations for an unordered list

First Generation Values	Update	Enhanced Update
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 200 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata = ‘Update HAE = 200’

Using trees, discussed in Section 7.1.2.2, applications store Amend Metadata elements in a child sub-list. Different changes group together into their own Amend Metadata list and each Amend Metadata list is independent. Table 27 is an example of two Amend Metadata sub-lists in red. The first Amend Metadata sub-list shows a deletion of ‘Sensor Type’ and an update of ‘HAE’ from 100 to 200. The second Amend Metadata sub-list shows an update of ‘HAE’ from 100 to 150. In both ‘HAE’ cases, if the child element exists in the parent, then the change is an Update operation; otherwise, it is an Add operation.

Table 27: Example showing an Enhanced Delete-and-Update Amend Metadata as sub-lists of the first-generation metadata

First Generation Values	Enhanced Delete and Update
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata Sensor Type=<null> HAE = 200 Amend Metadata HAE = 150

Per Data Preservation, there is an additional step to consider in the *interpretation* of the resulting metadata during processing. The interpretation must include choices based on the number of Amend Metadata lists included. For example, Table 28 shows several options in interpreting the metadata listed in the right column of Table 27. Note that all data elements listed at the right in Table 27 exist in the group of metadata; that is, no first-generation or subsequent-generation metadata are missing. However, from an end-user perspective, there are three choices in which

data elements to use: First Generation Values, First Generation Values + first Amend Metadata, or First Generation Values + second Amend Metadata.

Table 28: Possible interpretations of metadata by end user

Option 1 (First Generation Values)	Option 2 (First Generation Values + first Amend Metadata)	Option 3 (First Generation Values + second Amend Metadata)
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 200	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 150 Sensor Type = Visible

This interpretation, of children sub-trees, is an “ORing” operation, meaning the end user displays or uses either Option 1 OR Option 2 OR Option 3.

Amend Metadata can be hierarchical, supporting multiple levels of sub-lists to represent updates-of-updates. Table 29 shows two levels of Amend Metadata.

Table 29: Example showing enhanced delete and update Amend Metadata as sub-lists of first-generation metadata

First Generation Values	Enhanced Delete and Update
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata Sensor Type=<null> HAE = 200 Amend Metadata HAE = 150

The interpretation of Table 29 has several options as shown in Table 30:

Table 30: Possible interpretations of metadata by end user

Option 1	Option 2	Option 3
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 200	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 150

Amend Metadata sub-lists can introduce new data as a change occurs. For example, Table 31 illustrates an update of the ‘HAE’ value and the addition of a newly computed standard deviation ‘HAE_SD’ using the Amend Metadata.

Table 31: Example showing the Amend Metadata with an Enhanced Update operation which includes new metadata

First Generation Values	Enhanced Delete and Update
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata HAE = 150 HAE_SD = 2

7.1.4.1.1 Amend Identification

Motion Imagery is a time-based media (i.e., series of images, each associated with a time), therefore, the supporting metadata is also time-based (i.e., series of metadata “packets”, each associated with its own time, see Section 7.5.1). Adding two or more Amend sub-lists to a packet presents a tracking issue between each different packet, i.e., the interpretation of children sub-trees, via the “ORing” operation, could appear different from one packet to the next.

As an example, Table 32 illustrates two packets, each with their own Amend Metadata sub-lists. With Packet 1, some correction process, C_1 , deletes the sensor type and corrects the HAE to 200. A second correction process, C_2 changes the HAE to 150. Packet 2 uses the same correction process C_1 to delete the sensor type and change the HAE to 210; additionally, correction process C_2 changes the HAE to 175. This example illustrates an inability to determine which correction process in the Packet 1 relates to which correction process in Packet 2; i.e., the end user cannot determine if correction process C_1 or C_2 created the first or second Amend sub-list in Packet 2.

Table 32: Example showing two metadata packets each with two Amend sub-lists

Packet 1	Packet 2
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata Sensor Type=<null> HAE = 200 Amend Metadata HAE = 150	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata HAE = 175 Amend Metadata Sensor Type=<null> HAE = 210

To rectify this relational issue, each Amend sub-list includes a Metadata Substream Identifier (MSID), see Section 7.1.3.1. Since the Amend sub-list is a well-define sub-list and it includes the MSID, the Amend sub-list becomes a substream in the metadata stream (see Section 7.1.3).

Table 33 illustrates the example in Table 32 but with designated MSID in each Amend sub-list. Now, with the MSIDs, the Packet 2’s Amend lists unambiguously relate to Packet 1’s Amend

lists. The result is the metadata stream has two substreams, one with MSID=1 and the second with MSID =2.

Table 33: Example showing two metadata packets each with two Amend sub-lists and their respective MSIDs

Packet 1	Packet 2
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata Sensor Type=<null> HAE = 200 MSID =1 Amend Metadata HAE = 150 MSID = 2	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Sensor Type = Visible Amend Metadata HAE = 175 MSID = 2 Amend Metadata Sensor Type=<null> HAE = 210 MSID = 1

Table 34 illustrates the use of MSIDs in a hierarchy of Amend sub-lists. The result is the metadata stream has one substream, i.e., MSID=1, and two sub-substreams, i.e., MSID=2 and MSID=3.

Table 34: Example showing two metadata packets each with a hierarchy of Amend sub-lists and their respective MSIDs

Packet 1	Packet 2
Temperature = 20	Temperature = 20
Latitude = 44.98	Latitude = 44.98
Longitude = -93.26	Longitude = -93.26
HAE = 100	HAE = 100
Sensor Type = Visible	Sensor Type = Visible
Amend Metadata	Amend Metadata
Sensor Type=<null>	Sensor Type=<null>
HAE = 200	HAE = 210
MSID = 1	MSID = 1
Amend Metadata	Amend Metadata
HAE = 150	HAE = 140
MSID = 2	MSID = 3
Amend Metadata	Amend Metadata
HAE = 140	HAE = 120
MSID = 3	MSID = 2

7.1.4.2 Data Sharing

In certain applications, metadata contexts contain similar information. Combining the context (e.g., a tree) into one structure saves bytes; however, applications need an interpretation process separate the trees back into their original contexts. For example, assume a Motion Imagery stream contains a split screen of two sources from the same platform. Some of the metadata accompanying these two sources contains the same information as shown in Figure 56. The red elements indicate the duplicate elements each list; these elements are shareable.

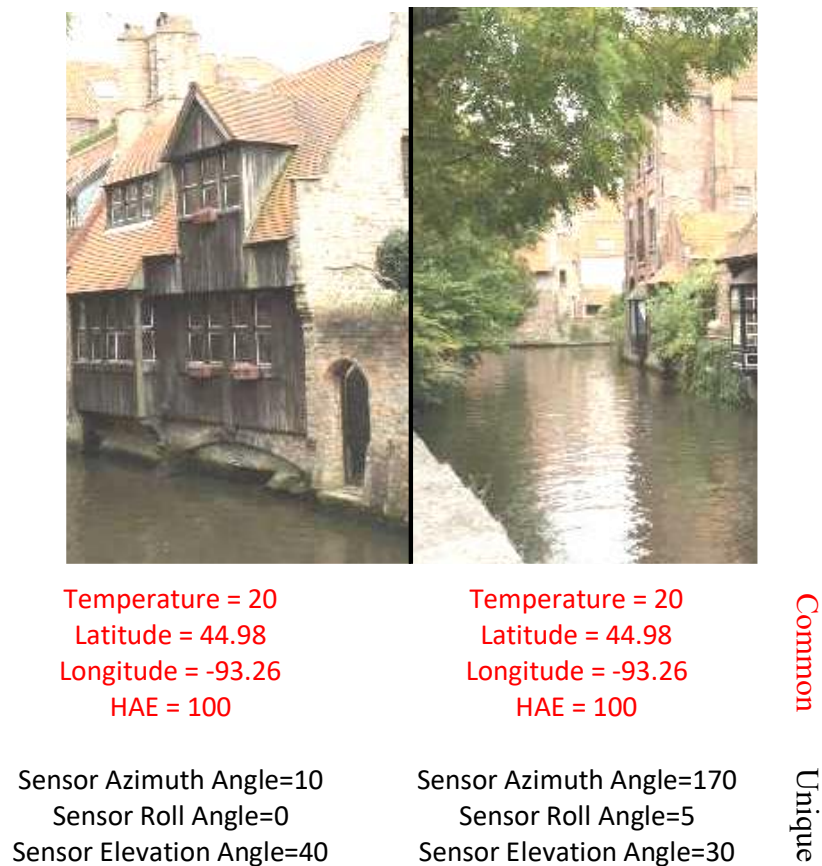


Figure 56: Example of two Motion Imagery sources with common and unique data
(Bruges Belgium 2014 Lat: N51 12'18" Lon: E3 13'48")

Using a tree structure, common elements group together at the parent level, while the unique elements move into their own child groups. A “Segment Metadata” list structure enables this tree structure, where Segment Metadata sub-lists contain child elements to a parent group of metadata. Table 35 shows the combination of the two metadata lists with the common data elements (in red) at the root of the tree, and the unique sensor-specific data elements in two “children” trees, each identified with their own Segment Metadata list.

The reduction in bytes saved by not replicating common elements can be meaningful when using large datasets. Beyond the savings in bytes, however, is a method that affords expressing multiple metadata elements using the same identifier with different values repeatedly without confusion.

With Data Sharing the metadata requires an additional step in its interpretation. The interpretation process must process all “branch” metadata back into their original lists, providing all lists to the end user. For example, a reader of the metadata in Table 35 needs to produce the two lists in Figure 56.

Table 35: Common Metadata Combined

Temperature = 20
Latitude = 44.98
Longitude = -93.26
HAE = 100
Segment Metadata
Sensor Azimuth Angle = 10
Sensor Roll Angle = 0
Sensor Elevation Angle = 40
Segment Metadata
Sensor Azimuth Angle = 170
Sensor Roll Angle = 5
Sensor Elevation Angle = 30

This type of interpretation, of children sub-trees, is an “ANDing” operation; therefore, given several Data Sharing options (e.g., in Figure 56 there are two lists of elements so there are two options), Table 36 shows Option 1 AND Option 2 is displayed to, or used by the end user. Some applications may need to nest branches within branches which is an extension to the single element tree.

Table 36: Options in interpreting the metadata of Figure 56

Option 1	Option 2
Temperature = 20	Temperature = 20
Latitude = 44.98	Latitude = 44.98
Longitude = -93.26	Longitude = -93.26
HAE = 100	HAE = 200
Sensor Azimuth Angle = 10	Sensor Azimuth Angle = 170
Sensor Roll Angle = 0	Sensor Roll Angle = 5
Sensor Elevation Angle = 40	Sensor Elevation Angle = 30

7.1.4.2.1 Segment Identification

Similar to the Amend sub-lists (Section 7.1.4.1.1), adding two or more Segment sub-lists to each packet presents a tracking issue between each packet, i.e., the interpretation of children sub-trees, could appear different from one packet to the next. Table 37 illustrates two packets, each with their own Segment Metadata sub-lists. In Packet 1, the left sensor data (red) of Figure 56 occurred first in the packet, while in Packet 2, the right sensor data occurred first (blue) in the packet. This example shows an inability to determine which segment sub-list in Packet 1 relates to the segment sub-list in Packet 2.

Table 37: Example showing two metadata packets each with two Segment sub-lists

Packet 1	Packet 2
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Segment Metadata Sensor Azimuth Angle = 10 Sensor Roll Angle = 0 Sensor Elevation Angle = 40 Segment Metadata Sensor Azimuth Angle = 170 Sensor Roll Angle = 5 Sensor Elevation Angle = 30	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Segment Metadata Sensor Azimuth Angle = 175 Sensor Roll Angle = 4 Sensor Elevation Angle = 31 Segment Metadata Sensor Azimuth Angle = 11 Sensor Roll Angle = 2 Sensor Elevation Angle = 41

As with the Amend sub-lists, to rectify this relational issue, each Segment sub-list includes a Metadata Substream Identifier (MSID), see Section 7.1.3.1. Since the Segment sub-list is a well-defined sub-list and it includes the MSID, the Segment sub-list becomes a substream in the metadata stream (see Section 7.1.3).

Table 38 illustrates the same example but with a MSID for each Segment sub-list. Now, with the MSID s, the Packet 2's Segment lists unambiguously relate to Packet 1's Segments lists. The result is the metadata stream has two substreams, one with MSID=1 and the second with MSID=2.

Table 38: Example showing two metadata packets each with two Segment sub-lists and their respective MSIDs

Packet 1	Packet 2
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Segment Metadata Sensor Azimuth Angle = 10 Sensor Roll Angle = 0 Sensor Elevation Angle = 40 MSID = 1 Segment Metadata Sensor Azimuth Angle = 170 Sensor Roll Angle = 5 Sensor Elevation Angle = 30 MSID = 2	Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Segment Metadata Sensor Azimuth Angle = 175 Sensor Roll Angle = 4 Sensor Elevation Angle = 31 MSID = 2 Segment Metadata Sensor Azimuth Angle = 11 Sensor Roll Angle = 2 Sensor Elevation Angle = 41 MSID = 1

In a similar fashion to Amend sub-list hierarchies, MSIDs provide the necessary packet-to-packet linkage for Segment sub-list hierarchies.

7.1.4.3 Co-Use of Amend Metadata and Segment Metadata

Utilizing both Data Preservation and Data Sharing within a metadata tree is possible. The Amend Metadata and Segment Metadata are both uniquely identifiable and can be intermixed at the same level of a tree or at different levels. For example, the side-by-side example of Table 38 could include updates for sensor angles in one of the Segment Metadata lists as shown as red text in Table 39.

Table 39: Amend Metadata list in a Segment Metadata list

Packet 1	Packet 2
Temperature = 20 Latitude = 44.98 Longitude = -93.26 HAE = 100 Segment Metadata MSID = 1 Sensor Azimuth Angle = 10 Sensor Roll Angle = 0 Sensor Elevation Angle = 40 Sensor FOV = 20 Amend Metadata MSID = 2 Sensor Azimuth Angle = 15 Sensor Roll Angle = 1 Segment Metadata Sensor Azimuth Angle = 170 Sensor Roll Angle = 5 Sensor Elevation Angle = 30 MSID = 3	Temperature = 20 Latitude = 45.0 Longitude = -93.3 HAE = 101 Segment Metadata MSID = 1 Sensor Azimuth Angle = 11 Sensor Roll Angle = -1 Sensor Elevation Angle = 39 Sensor FOV = 21 Amend Metadata MSID = 2 Sensor Azimuth Angle = 12 Sensor Roll Angle = 0 Segment Metadata Sensor Azimuth Angle = 171 Sensor Roll Angle = 6 Sensor Elevation Angle = 31 MSID = 3

7.2 Metadata Modeling

Metadata Modeling is the process of describing the metadata used within a system or group of systems. Typically, metadata models describe metadata groups (lists, trees, graphs) with a modeling language, and since the MISB uses KLV as a metadata transmission method, the descriptive language is the KLV groups defined in the MISP. The current models are starting to become more complex, so the use of KLV as the modeling language is becoming cumbersome and inflexible. As an example, the use of MISB ST 1010 – Standard Deviation and Correlation Coefficient metadata (SDCC), adds a level of complexity to the metadata structure. The use of the SDCC metadata links disparate metadata elements together in a graph structure, which KLV metadata does not support directly.

In future versions of the Motion Imagery Handbook, the purpose of this section will be to describe or reference a modeling language (e.g., UML) that will be independent of the data format. Users of the models will not need to be MISB KLV experts to understand the metadata structure.

The goal of the MISB is to produce metadata models common across the many varieties of systems both deployed, and in development.

7.3 Metadata Types

Metadata elements are values stored digitally with essence data. The digital values are represented in units of octets or bytes with each octet containing 8 bits. The term “byte” is used in all MISP documents to represent 8-bits of data. Bytes are the building block of many other data formats, such as numeric and character formats. Bytes are used in one of two ways: as a numeric representation, or in table lookup. As a numeric representation, each bit, within one or more designated bytes, relates to each other, thereby providing encoded information used to compute a numeric value. Examples of numerical representations include Integer and Floating-Point formats. With table lookup, the bit values within a byte(s) represent a specific index to a pre-defined list of values. An example of a table lookup representation is ASCII text values.

In the numeric form each byte can be represented as a binary value ranging from zero to 255. Each bit, b_i , represents the inclusion of a power of two in a sum to create a value, V , as shown in Equation 10.

$$V = \sum_{i=0}^7 b_i * 2^i \quad \text{Equation 10}$$

In Equation 10 each bit within the byte has different significance; the first bit signifies a unit value (2^0); the second bit signifies a two value (2^1); the third bit signifies a four value (2^2), etc. up to the eighth bit (2^7). The term *most significant bit* or *msb* indicates the eighth bit in the byte; the term *least significant bit* or *lsb* indicates the first bit in the byte. Bytes can be transmitted with either the msb or lsb sent first. Likewise, bytes can be stored in memory or represented in diagrams with the msb on the left, right, top, or bottom (all byte level diagrams should include indicators of msb and lsb). When multiple bytes are combined into one value, for example as a multi-byte unsigned integer, the byte containing the most significant bit of the whole value is the *Most Significant Byte* or *MSB*; the byte containing the least significant bit is the *Least Significant Byte* or *LSB*. When data is transmitted with the MSB first, the transmission is termed big endian; when data is transmitted with LSB first, the transmission is little endian.

In ST 0107 [16] the MISB requires all metadata transmitted and stored to be big endian.

A data type is the pattern used to transform digital data representations into meaning. There are many data types; international standards define some types, while others are specific to Motion Imagery standards. There are two categories of data types: simple and structured. Simple data types are representations for single values. Structured data types are data representations for groups of values (lists, and trees) as was discussed in Section 7.1.2.

Table 40 lists the simple data types used in the MISB standards. The Data Type column states the full name of the data type. The Abbreviation is the term used in MISP standards for specifying the type. The Section is the section number for more detailed information. The Description provides a brief description of the data type.

Table 40: List of Simple Data Types used by the MISB

Data Type	Abbreviation	Section	Description
BER Length	ber	7.3.1	An integer value represented using ASN.1 Basic Encoding Rules for Length
BER-OID	ber-oid	7.3.2	An integer value represented using ASN.1 Basic Encoding Rules - Object Identifier. Depending on the encoding of this value, the number of bytes is adjusted within the stream itself
Binary	binary	7.3.3	Special binary data which does not match one of the other types listed
Boolean	boolean	7.3.4	A single value representing either true or false (i.e., one or zero)
Character ISO7	iso7	7.3.5	ISO/IEC 646:1991, Information Technology – ISO 7-bit coded Character Set for Information Interchange
Character UTF8	utf8	7.3.6	A character value represented using the 8-bit Unicode value from ISO/IEC 10646:2014
Character UTF16	utf16	7.3.7	A character value represented using the 16-bit Unicode value from ISO/IEC 10646:2014
Enumeration	enum	7.3.8	An unsigned integer used as a lookup in a pre-defined enumeration table
Floating Point	float ¹	7.3.9	A floating-point value represented using IEEE 754 format
IMAP	imapa imapb	7.3.10	An integer value used to represent a floating-point value
Integer	int ¹	7.3.11	A two's complement integer which ranges from a negative min value to a max value based on the number of bytes
Unsigned Integer	uint ¹	7.3.12	An integer value that ranges from 0 to a max value based on its length

¹Each of these types can be defined with an unknown length (as shown) or with a fixed length.

Three of the types in Table 40 i.e., Floating Point, Integer and Unsigned Integer vary in the number of bits used to describe them. A metadata specification can either state a fixed number of bits, or use the non-numeric abbreviation shown in the table. In specifying a fixed number of bits, the number of bits follow the type (e.g., int32, or float64) and the number of bits must be a multiple of eight. Using the non-numeric abbreviation indicates the number of bits is defined at run-time when the data is created.

7.3.1 BER Length

Section 8.1.3.3 of ASN.1 [17] defines the Basic Encoding Rules (BER) for Length. These rules provide a method for embedding the length of an unsigned integer within the integer value itself, thus eliminating the need to define the length in an external specification, or elsewhere within the metadata. The MISP uses only the Short and Long Form BER Length methods.

7.3.1.1 Short Form

For an unsigned integer less than 128, the short form can be used. As shown in Figure 57, the short form is one byte, where the most significant bit (bit 7) in the byte set to zero (0) signaling short form encoding. The last seven bits encode the unsigned integer, ranging from a value of zero (0) to 127.

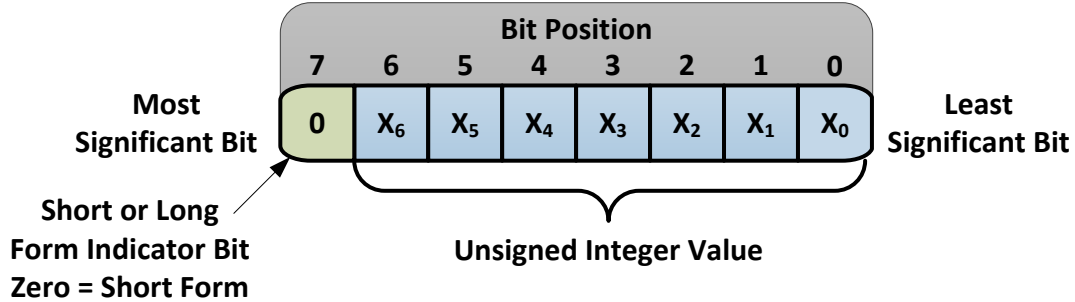


Figure 57: BER Short Form Encoding

The numeric value, V , is computed using Equation 11.

$$V = \sum_{i=0}^7 x_i * 2^i \quad \text{Equation 11}$$

7.3.1.2 Long Form

For an unsigned integer greater than or equal to 128, the long form is used. As shown in Figure 58, the long form uses multiple bytes, where the most significant bit in the most significant byte signals long form when set to one (1). The remaining 7 bits of the first byte ($N_0 \dots N_6$) indicate the number of subsequent bytes used for the unsigned value. The unsigned value begins with the most significant byte (byte N) immediately following the first byte. The bytes that follow the leading byte are the encoding of an unsigned integer and consist of 8 times N number of bits. The maximum number of bytes possible for the value is 127, which would equate to a very large number ($2^{127*8} - 1$)

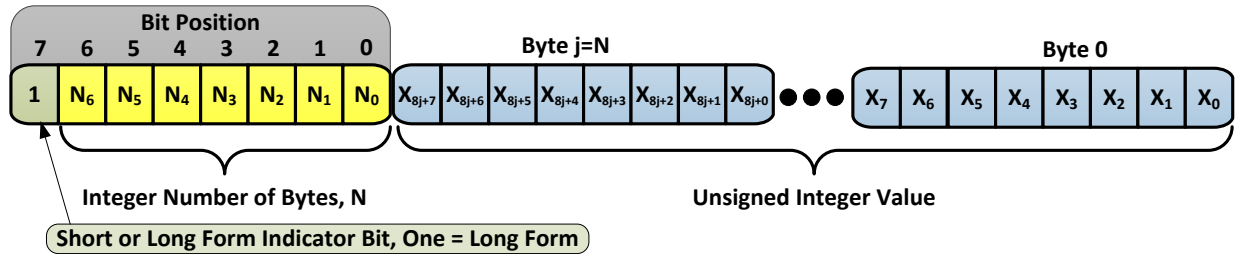


Figure 58: BER Long Form Encoding

The numeric value, V , is computed using Equation 12.

$$V = \sum_{i=0}^{N-1} \sum_{j=0}^7 x_{i*8+j} * 2^{i*8+j} = \sum_{i=0}^{8N-1} x_i * 2^i \quad \text{Equation 12}$$

7.3.2 BER-OID

The Basic Encoding Rules (BER) for an Object Identifier (OID) is defined in Section 8.19.2 of ASN.1 [18]. In a similar fashion to BER Length, these rules provide a method of embedding the length of an unsigned integer within the integer value itself, eliminating the need to have the length defined in an external specification or elsewhere within the metadata.

A BER-OID value is a series of one or more bytes, where the most significant bit (msb) of each byte signals if more bytes follow in defining a value. If the msb is one, a next byte is included; if the msb is zero, the byte is the last byte of the value. The remaining 7 bits of each byte combine into one unsigned integer. Figure 59 illustrates a one-byte BER-OID example, where the most significant bit (i.e., bit 7) is set to zero indicating there are no further bytes needed to represent the value. The single byte BER-OID form is an identical bit pattern to the short form BER encoding from section 7.3.1.1, including the method of computing the value; however, the meaning of bit 7 is different. Care must be taken to understand which type of BER encoding is being used.

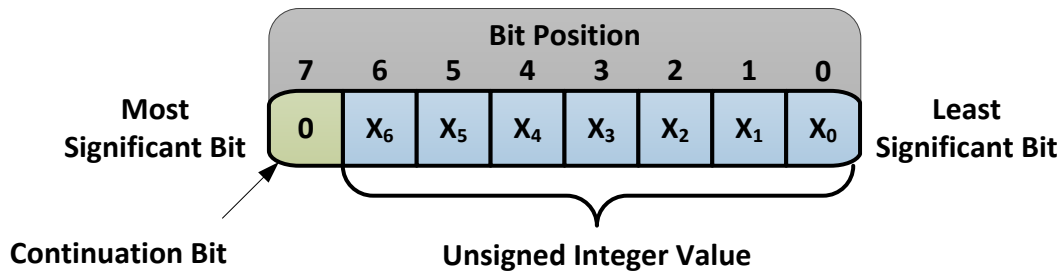


Figure 59: One-byte BER-OID Example

With multi-byte BER-OID values, bit 7 of each byte is the continuation bit. As shown in Figure 60, when the continuation bit is one, there is at least one more byte of data. A continuation bit value of zero (shown in red) signals the end of the unsigned integer. After all the bytes are identified, the remaining 7 bits of each byte are combined to construct the value.

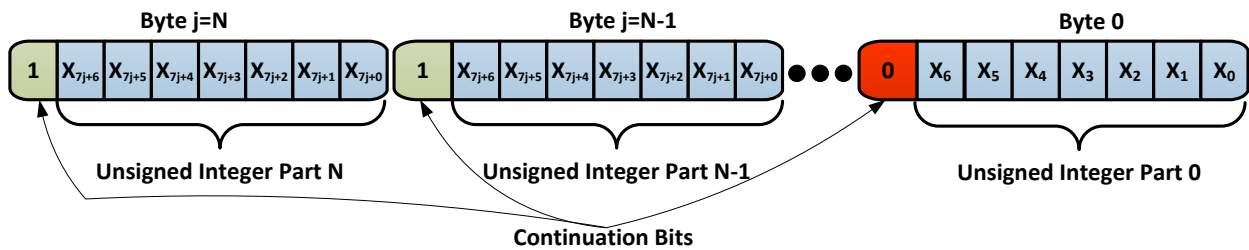


Figure 60: Multi-byte BER-OID Example

The numeric value, V , is computed using Equation 13.

$$V = \sum_{i=0}^{N-1} \sum_{j=0}^6 x_{i*7+j} * 2^{i*7+j} = \sum_{i=0}^{7N-1} x_i * 2^i \quad \text{Equation 13}$$

7.3.3 Binary

A Binary data type compacts a set of information into one or more bytes. The binary type can contain flags, small enumerations and other bit-specified controls. When MISP metadata standards use binary types, the standard describes how to interpret each bit of the binary value.

7.3.4 Boolean

The Boolean data type is a single value representing either true or false (i.e., one or zero). The data representation of a Boolean value may be with a single bit or byte. With a single bit, when the bit value is zero (0), the Boolean meaning is false; when the bit value is one (1), the Boolean meaning is true.

With a byte, when the byte value equals zero (0x00), the Boolean meaning is false; when the byte value is one (0x01) the Boolean meaning is true; all other values are not valid.

7.3.5 Character ISO7

The ISO7 data type is one or more characters from the ISO/IEC 646:1991 [19] character set. ISO7 uses the lower seven bits of a byte; the eighth bit is always zero.

ISO7 supports the Latin character set for English only. For this reason, the MISP is discontinuing ISO7 in document updates and new publications. Some existing MISP standards use ISO7 for historical and backward compatibility reasons. The replacement data type for ISO7 is UTF8.

7.3.6 Character UTF8

The UTF8 data type is one or more characters from the ISO/IEC 10646:2014 Universal Character Set (UCS) [20]. UTF8 is one of several character encoding schemes within ISO/IEC 10646:2014. UTF8 is a variable length value, which operates in a similar fashion to BER-OID encoding discussed in Section 7.3.2. With UTF8, the upper bit of the first byte determines if the character requires additional bytes. UTF8 characters use a minimum of one byte and can grow up to four bytes, which leaves a large address space for non-English character sets, such as dialects of Chinese. The first group of 127 UTF8 characters, with bit 8 equal to zero, is equivalent to ISO7.

MISP standards use UTF8 for character strings because it expands as necessary to support alternate languages in support of NATO countries.

7.3.7 Character UTF16

The UTF16 data type is one or more characters from the ISO/IEC 10646:2014 Universal Character Set (UCS) [20]. UTF16 is one of several character encoding schemes within ISO/IEC 10646:2014. UTF16 is a variable length value, which operates in a similar fashion to BER-OID encoding discussed in Section 7.3.2. With UTF16, the upper bits of the first byte determine if the character uses more bytes. UTF16 characters use either a minimum of two bytes or a maximum of four bytes, leaving a large address space to support the full Unicode character set. The first group of 127 UTF16 characters, with all bits of byte one zero, and bit 8 of byte two equal to zero is equivalent to UTF8 and ISO7.

Older MISP standards have used UTF16, but all document updates and new publications will utilize UTF8 instead of UTF16.

7.3.8 Enumeration

An Enumeration type is an unsigned integer whose value maps to a predefined table of choices. A controlling document (e.g., standard) defines the range of allowed unsigned integer values beginning at zero to some maximum value along with the meanings for each value. An enumeration compacts a list of choices into a single unsigned integer value, thereby saving bytes.

7.3.9 Floating Point

The Floating-Point type is a subset of the IEEE 754 Standard for Floating-Point Arithmetic and Floating-Point formats [21]. IEEE 754 Floating-point numbers support decimal and binary interchange of values. The MISP standards only use binary interchange formats.

The basic form of the binary interchange format includes three values: sign bit, biased exponent and significand. IEEE 754 defines different binary interchange formats by specifying variations of the length – or number of bits – of a value, and the length for the biased exponent and significand. The MISP standards supports IEEE 754 binary16 (two-byte), binary32 (four-byte Single Precision), and binary64 (eight-byte Double Precision) floating-point values. Future MISB standards may support binary128 (sixteen-byte Quadruple Precision) values.

IEEE 754 also defines special bit patterns for representing $\pm\infty$, Not a Number (NaN), and other values. The MISP standards do not allow these special values unless a controlling document specifies how to use the special value.

In a MISP standard, the type designator “float” indicates that an embedded method determines the length. For example, the length within a KLV triplet supplies the byte count used for the floating-point value. A MISP standard can also define a specific length for a floating-point value by appending the number of bits to use at the end of the “float” designator; for example, “float32” indicates a single precision floating-point value, and “float64” indicates a double precision floating-point value.

7.3.10 IMAP

The IMAP type is an unsigned integer, which is a mapping to a floating-point value as specified by MISB ST 1201 [22]. Knowing certain parameters (min, max, resolution) about the value enables this representation to use fewer bytes than an equivalent IEEE 754 floating-point value.

7.3.11 Integer

The Integer type is a two’s complement integer, where the most significant bit (msb) is a sign bit and all remaining bits specify the integer value. The binary representation of a two’s complement value is not obvious. The integer value for a two’s complement binary number with bits a_{N-1} (msb) to a_0 (least significant bit) can be computed using Equation 14. In this equation a_{N-1} is the sign bit.

$$value = -a_{N-1} * 2^{N-1} + \sum_{i=0}^{N-2} a_i 2^i \quad \text{Equation 14}$$

Integer values have different bit lengths; the MISP Integer type defines the values in groups of eight bits, or one or more bytes. In a MISP standard, the type specifier of “int” indicates that another embedded method determines the length. For example, the length within a KLV triplet

supplies the byte count for the integer value. A MISP standard can also define a specific length for an integer value by appending the number of bits to use at the end of the “int” designator; for example, “int8,” “int16,” “int32,” etc.

7.3.12 Unsigned Integer

The Unsigned Integer type is a standard binary representation of a non-negative integer. Values range from zero to $2^N - 1$, where N is the number of bits used. The MISB defines the Unsigned Integer type in groups of eight bits, or one or more bytes. In a MISP standard, the type specifier of “uint” indicates an embedded method determines the length. For example, the length within a KLV triplet supplies the byte count for the unsigned integer value. A MISP standard can also define a specified length for an unsigned integer value by appending the number of bits to use at the end of the “uint” designator; for example, “uint8,” “uint16,” “uint32,” etc.

7.4 SMPTE – Key, Length, Value (KLV) Metadata Formatting

The MISB has adopted KLV developed by SMPTE for transmitting and storing metadata in Motion Imagery Systems. This section briefly describes the SMPTE KLV standard as a basis for how the MISB has extended its use of KLV. For complete details about SMPTE KLV see [14].

KLV is a data encoding structure for transmission or storage. The basic concept of the KLV triplet is to encode a data value, along with a count of bytes needed to support the data value, and an identifier, into one block of binary data. The identifier links the value to its definition or meaning, and the value can be used for computation, display, etc. The identifier is the Key; the number of bytes needed to describe the data is the Length, and the data being transmitted is the Value. These three pieces of information are combined to make a single Key-Length-Value (KLV) item as illustrated in Figure 61.

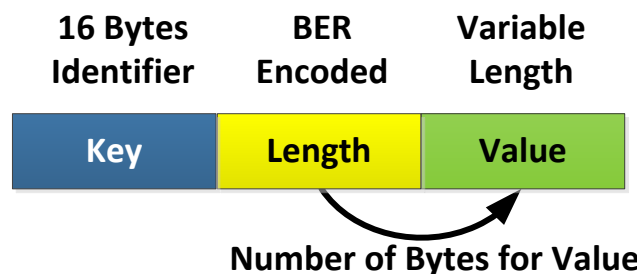


Figure 61: Illustration of Key Length Value Item

The Key is a reserved set of 16 bytes used as an index into a dictionary of registered Keys. The Key determines how to parse the value and provides insight into how to use the value. The byte(s) immediately following the key indicates the Length. The Length is the number of bytes needed to describe the value. For example, if a value is specified as a 32-bit integer (i.e., four bytes), the Length is four, representing the four bytes for the integer value. Immediately following the Length are the bytes for the Value. The Value is a block of bytes encoded using the rules specified for the given key. The Value can be a single numeric value (i.e., integer or float), string, a binary value, or a group of items (such as a list) of any length.

Once a Key, Length and Value combine, they form a KLV data block for transmission or storage as a singular unit. Other media embed KLV blocks as their metadata, such as Motion Imagery streams or files; or KLV blocks are used independently of any media.

7.4.1 KLV Dictionary

To ensure each key is unique and properly defined, SMPTE defines a KLV Dictionary that lists all defined keys with their attributes e.g., name, data type and other information. This list provides the meaning of the value for a given key. For example, as a decoder parses a KLV block, the first item read is the KLV Key. This key is the index in the dictionary to look up the meaning of the value to determine how to process or display it. As an illustration, the key could reference the altitude of the camera as its imaging the scene, so the decoder would know how to display that information to the user (e.g., graphically or units used).

7.4.2 Key

A KLV key is a 16-byte identifier that has three components, **SMPTE KLV Identifier**, **Parsing Identifier** and **Table Lookup Identifier**, as illustrated in Figure 62.

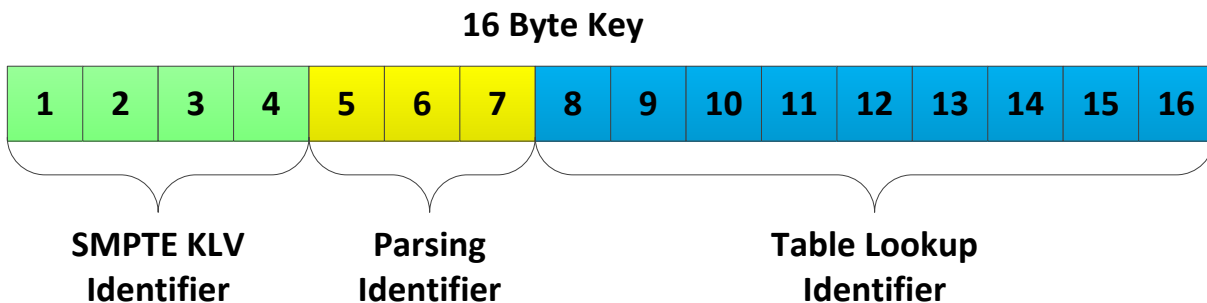


Figure 62: Key's Components

The **SMPTE KLV Identifier** indicates the bytes are a KLV key; these bytes are registered with ISO and are always 0x06-0E-2B-34. The **Parsing Identifier** of three bytes indicates the type the KLV value uses, either a single value, group of values or label (each discussed in Section 7.4.4). The parsing identifier can have several options with the details described in SMPTE 336 [14]. See Appendix D below for a quick reference of these options. The **Table Look-Up Identifier** is a set of bytes that reference a dictionary, which provides both syntactical and semantic information about the value.

7.4.3 Length

The length is one or more bytes encoded using BER Short or Long form encoding (see Section 7.3.1). The Length can grow as needed to specify the number of bytes for the value. A length of zero indicates a value is not included in the KLV triplet.

7.4.4 Value

The Value is a block of bytes interpreted using the information from the parsing identifier of the Key.

7.4.4.1 Single Item

A Single Item value is block of bytes convertible into one of the types listed in Table 40.

7.4.4.2 Group

A Group is a block of bytes containing a list of single items and potentially another group, so each group can recursively contain other groups. There are five types of groups listed in Table 41. The first column is the full name of the group; the second column is the short name of the group used as a type in the KLV dictionary; the final column is a short description of the group. In all cases, each group is transformable into a universal set and the items are referenceable in the KLV dictionary. See Appendix D for the list of KLV bytes to signal these different groups in the key.

Table 41: Summary of KLV Groups

KLV Group Name	Short Name	Description
Universal Set	uset	Each element in the group uses a full 16-byte key, length and value. This type of group does not need a controlling document to understand how to parse the values. Parsers can extract data from these groups without any special software. <u>Universal sets are not in use by the MISB anymore because a key is inefficient, consuming too many bytes to specify.</u>
Global Set	gset	Same as the universal set except each element uses a subset of a KLV key. This type of group does not need a controlling document to understand how to parse the values. Parsers can extract data from these groups without any special software. <u>Global sets are not in use by the MISB because a key is inefficient, consuming too many bytes to specify.</u>
Local Set	ls	A Local Set requires a controlling document to define a list of predefined identifiers for each element contained in the Local Set. Local sets can specify elements in any order. The identifiers are Tags, instead of Keys. Tags are a list of serially numbered unsigned integers starting with zero. Tags can use different binary representations; however, the MISB uses BER-OID tags where possible. BER-OID tags allow sets to grow as needed without limits on a specific unsigned integer size.
Variable Length Pack	vlp	A Variable Length Pack requires a controlling document to define a list of items in a specific order. Because of the pre-defined order, the Variable Length Pack parses into a known list of items. Since the list is pre-defined, there is no need for a key or tag, thereby reducing the number of bytes needed to specify a value. A Variable Length Pack only specifies the length of each value followed by the value.
Defined Length Pack	dlp	A Defined Length Pack requires a controlling document to define a list of items in a specific order and with predefined or computable lengths. Because of the pre-defined order, a Defined Length Pack parses into a known list of items. Since the list is pre-defined, there is no need for a key or tag, thereby reducing the number of bytes needed to specify a value. Since the length is pre-defined or computable, there is no need for the length, thereby reducing the number of bytes needed. A Defined Length Pack only

		specifies the values themselves without the key or length; thus, they are the most bit efficient method for specifying a group.
--	--	---

7.4.4.2.1 Local Set (LS)

A Local Set is a KLV construct consisting of a 16-byte key, Local Set length and a list of Local Set triplets. Each Local Set triplet consists of a Tag, value Length and Value as shown in Figure 63.

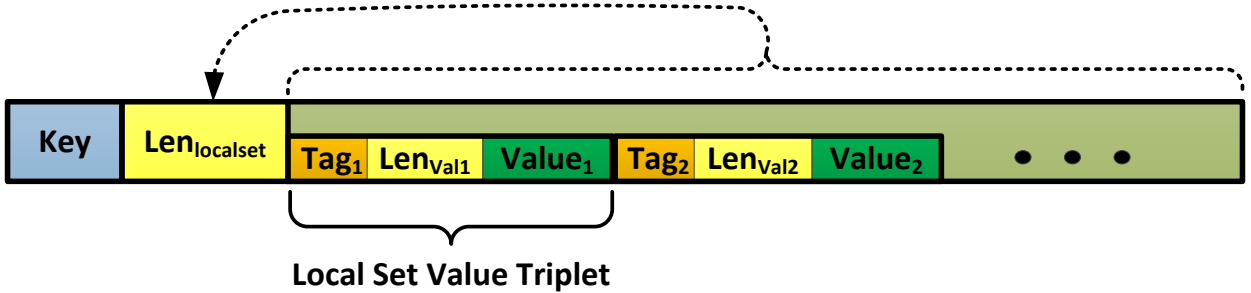


Figure 63: Illustration of a Local Set

7.4.4.2.2 Variable Length Pack (VLP)

A Variable Length Pack is a construct containing a 16-byte key, pack length and a list of VLP Value Pairs as shown in Figure 64.

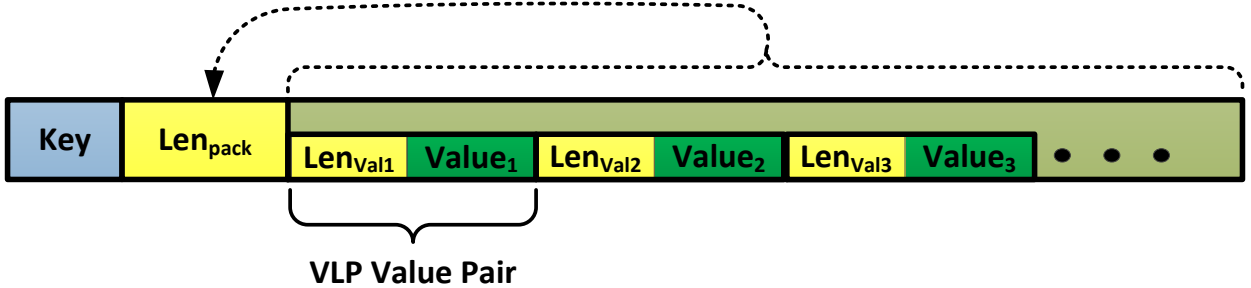


Figure 64: Illustration of a Variable Length Pack

Variable Length Packs may include zero length values by setting the value’s length to zero.

7.4.4.2.3 Defined Length Pack (DLP)

A Defined Length Pack is a construct containing a 16-byte key, pack length and a list of values as shown in Figure 65.

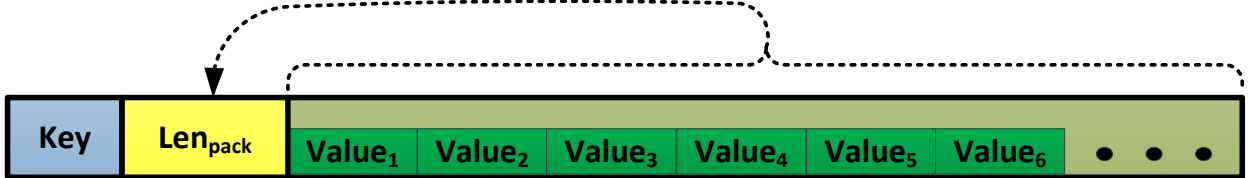


Figure 65: Illustration of a Defined Length Pack

Each value in a Defined Length Pack has either a pre-defined length or the length is computable from information within the value or the whole pack. For example, if the value list is a series of BER-OID values, each value's length is computable while reading the value. Additionally, one value in a pack can have its length "float", meaning the length is variable. The floating-length value's length is computed by adding up the lengths of all other items and subtracting those lengths from the pack length; the remaining length is the length of the floating-length item. The document defining the Defined Length Pack must specify, for each item, either the item's fixed length, the item's method of parsing (if the item provides its own length), or if the item is the floating-length item.

7.4.5 Parsing Rules

KLV parsing rules are simple; there are three items to parse: key, length and value. When parsing the key, the SMPTE KLV Identifier validates the Key is a proper KLV key. The Parsing Identifier determines how to parse the value, either a single item or group. The Table Lookup Identifier provides a lookup value into the dictionary, which supplies the specifics about the value within the dictionary. If the parsing software does not recognize the key (i.e., it is not in the dictionary), the parser is expected to read and use the length to determine how many bytes to skip over the value, i.e., the value is ignored. The capability of skipping over unknown items enables backwards compatibility for future changes and removes issues with fixed record lengths.

7.5 MISB KLV Extensions

The MISP standards extend and refine KLV with the overall goal of reducing the number of bits needed to convey metadata (i.e., reduced bandwidth) and improving the quality of transmission and interpretation of the metadata. These extensions/refinements are specific to the types of metadata Motion Imagery Systems produce.

7.5.1 MISB KLV Terms and Definitions

SMPTE KLV allows KLV groups within other groups forming a tree with parent/child relationships as discussed in Section 7.1.2.2. The top-most parent of a KLV metadata tree is the **KLV Root**. The combination of a 16-byte KLV key, length of root data, and the root data itself (including all children of the root) defines a **KLV Packet**. The KLV Root data is also known as the **Packet Payload**. A series of KLV Packets is a **KLV Metadata Stream**. Each KLV Packet is an independent element in a KLV Metadata stream, so one packet does not rely on another packet for decoding.

KLV Root: The top-most parent of a KLV metadata hierarchy.

KLV Packet: The combination of the KLV 16-byte key, length of KLV Root and KLV Root data (i.e., parent data and children data).

Packet Payload: The KLV “value” portion of the KLV Packet, also known as the KLV Root data, or just “payload”.

KLV Metadata Stream: A series of KLV Packets.

7.5.2 MISB KLV Dictionary

MISB KLV uses definitions from two different KLV Dictionaries: the SMPTE KLV Dictionary discussed in Section 7.4.1, and the MISB KLV Metadata Dictionary MISB ST 0807 [23]. Most of the MISB KLV keys used in the MISB standards are registered in the MISB KLV Dictionary; however, to maintain backwards compatibility and for historical reasons some keys are registered in the SMPTE dictionary.

The MISB maintains its own dictionary for two reasons. First, additions and updates to the SMPTE dictionary can take a minimum of six months to a year with the attributes of the keys under scrutiny by SMPTE members, who are outside of the Motion Imagery community. In some cases, SMPTE has changed the name, type, or meaning of a proposed value. Second, the MISB applies different rules to what is registered, and the use of data types as discussed below.

7.5.2.1 KLV Value Registration

One SMPTE KLV rule is all values are to be registered within a KLV dictionary. This rule is not practical in some MISB KLV documents because of the sheer volume in registering such values.

For instance, Motion Imagery Systems rely on measurements to extract useful information about the Motion Imagery, such as geographical positions from sensor location and orientation. With any measurement, there exists some uncertainty along with cross-correlation between measurements. The MISP registers the primary measurement value (i.e., latitude, longitude, sensor depression angle, etc.) in the KLV dictionary, but not secondary statistics, such as standard deviations and cross-correlations between the primary values. There is far too much overhead in registering the volume of these statistics. Cross-correlation alone could produce an N^2 increase in KLV dictionary items; that is, ten primary elements would produce 100 statistic elements. Instead of registering these statistical elements, the MISP reports standard deviation and cross-correlation values using MISB ST 1010 [24].

7.5.2.2 Data Type Processing

The MISB Data Types are normative, but MISB standards using Local Sets or Packs may override the format used to conserve bytes. For example, MISB standards referencing keys in the MISB dictionary which use the floating-point type may encode the floating point as an integer with techniques from Section 7.5.5. Another example is a MISB standard which references keys in the MISB dictionary using the unsigned integer type but may encode the unsigned integer as a BER-OID encoded value. With MISB standards and dictionary definitions, the standard describing how the bytes are encoded into KLV defines the format used. Where a MISB standard does not specify a special or adjusted format, the format indicated in the KLV dictionary is used.

MISB standards utilizing Universal Sets do not override the formats defined in the MISB dictionary.

7.5.3 Value Sizing

One method to reduce the number of bytes in a Local Set is to adjust the number of bytes used to represent a value based on the value. For example, if a Local Set defines a uint value and the value to encode is the number 12, only one byte is needed. The Local Set would define the length of this value to be one, resulting in three bytes to send the value 12; one byte for the tag, one byte for the length and one byte for the value.

7.5.4 MISP KLV Groups

MISP metadata is principally specified as one or more Local Sets, variable length packs and defined length packs for grouping items. Universal and global sets require many more bytes consuming unnecessary bandwidth.

The MISP imposes an additional rule to Local Sets beyond SMPTE rules. When forming a data block in a SMPTE Local Set, the element order can be random, so when parsing and using the data it is an unordered list. With MISP Local Sets, element order may be important so when parsing and using the data the order must be retained. For example, MISB ST 1010 - Standard Deviation and Cross-Correlation metadata (SDCC) requires the items in the parent Local Set preceding the SDCC to be in a specific order. Defined order in a Local Set improves the efficiency in processing the metadata. For example, knowing the time value is the first element in a Local Set reduces the need to search for that element amongst the Local Set items, thereby reducing processing time.

The defined length pack used in MISP standards have three refinements: Flexible Length Values, Floating Length Packs and Truncation Packs.

7.5.4.1 Flexible Length Values

The MISP allows flexible length values within a pack. “Flexible length” values are values where the length can be determined by the value itself. BER-OID values, see Section 7.3.2, are flexible length, because the value includes a continuation bit in each byte. As discussed previously, BER-OID is the Basic Encoding Rules for Object Identification. The MISP uses BER-OID as a method for optionally typecasting any unsigned integer, not just Identifiers. BER-OID values are very useful within defined length packs to determine the length of a value within a pack without directly specifying the length.

7.5.4.2 Floating Length Pack

A “floating length” pack is identical to a defined length pack except the one of the elements’ length can change or “float,” enabling the pack to contain a variable length item, or group within it. To compute the length of the variable element, subtract the length of the other $n - 1$ known-length elements from the total pack length. Figure 66 illustrates a “floating length” pack, with the last element having variable length.

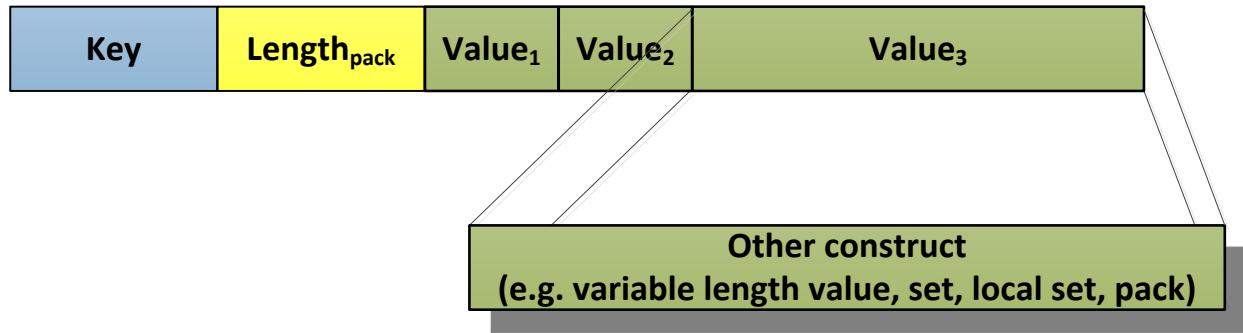


Figure 66: Floating Length Pack Illustration

The Key (in blue) is a pack key (16 bytes); the Length_{pack} (in yellow) is the total number of bytes used by Value₁, Value₂ and Value₃; Value₁ and Value₂ are defined with specific lengths in the controlling document. Value₃ is variable in length and can contain other KLV groups (sets, packs, etc.). The length of Value₃ is computed by subtracting the lengths of value₁ and value₂ from the length of the pack (Length_{pack}): $\text{Length}_3 = \text{Length}_{\text{pack}} - (\text{Length}_1 + \text{Length}_2)$.

There can be only one floating length value in a Floating Length Pack, all other elements are either fixed length or flexible length values.

7.5.4.3 Truncation Packs

A “truncation” pack is a KLV defined length pack, where its length value changes based on a subset of values used. For example, Figure 67 shows four values in a pack; the length of this pack is $8+12+8+16 = 44$ bytes.

Key	Length _{pack}	Value ₁	Value ₂	Value ₃	Value ₄
	8+12+8+16=44	(8 bytes)	(12 bytes)	(8 bytes)	(16 bytes)

Figure 67: Defined Length Pack Example

If an application decides Values 3 and 4 are not necessary, the pack can be truncated by setting the length to: $8+12 = 20$ bytes as shown in Figure 68.

Key	Length _{pack}	Value ₁	Value ₂	Value ₃	Value ₄
	8+12=20	(8 bytes)	(12 bytes)	(8 bytes)	(16 bytes)
				Truncated from pack Not sent	

Figure 68: Truncation Pack Example

There are two considerations in using a truncated pack: element order and incomplete values. First, element order is important when defining a truncation pack: the most important items come first – if truncation occurs, the lower priority items are removed. Secondly, applications need to remove complete values and not partial values. If a truncation results in a partial value, decoders must skip over decoding the partial value.

An example of truncation pack order, shown in Figure 69, is a pack containing Lat, Lon, Elev, Sig Lat, Sig Lon, Sig Elev, Comp Method and Time offset. By locating the latitude, longitude

and elevation items first, the remainder of the pack can be truncated leaving the most important items (the Lat, Long and Elev). Adjusting the $\text{Length}_{\text{pack}}$ for a smaller pack and truncating unneeded values conserves bandwidth.

Key	$\text{Length}_{\text{pack}}$	Lat	Lon	Elev	Sig Lat	Sig Lon	Sig Elev	Comp Method	Time offset
					Truncated from pack – not sent				

Figure 69: Example of Truncation Pack

All values in a pack must be supplied prior to truncating the pack - values cannot be skipped or dropped. Sometimes it is necessary to supply filler values when a pack value is unknown. Each value listed in a truncation pack's controlling document must define a filler value; an example is to use IEEE NaN for floating-point values. Figure 70 shows an example where the Lat/Lon, Sig Lat and Sig Lon are valid values, but the Elev value is unknown so it must use a filler value of NaN.

Key	$\text{Length}_{\text{pack}}$	Lat	Lon	NaN	Sig Lat	Sig Lon	Sig Elev	Comp Method	Time offset
							Truncated from pack – not sent		

Figure 70: Example of Truncation Pack with Filler Value

7.5.5 Float to Integer Mapping

The IEEE 754 Floating-Point Standard [21] supports large ranges of values along with fractional values. In most Motion Imagery, measurement values do not need to use the full floating-point range or full fractional resolution. Any measurement value has a known min/max range and resolution. For example, an angle measure has a range from 0° to 360° and, depending on the system, a fixed measurement resolution. The min, max and resolution enable a mapping between a floating-point value and an integer representation, where the integer representation may use fewer bytes. MISB ST 1201 Floating Point to Integer Mapping [22] defines the methods and rules for performing the mapping. Before ST 1201, some standards, such as MISB ST 0601, defined their own float-to-integer mapping for each value.

7.5.6 Data Structures

The MISP has defined standards to ensure implementers consistently work with certain common data structures: Standard Deviation and Cross-Correlation (SDCC) and Multi-Dimensional Arrays.

7.5.6.1 Standard Deviation and Cross-Correlation (SDCC-FLP)

The SDCC-FLP is a data structure which enables the efficient encoding of Standard Deviation and Cross-Correlation information. The data structure encoded is a triangular matrix of values, where the main diagonal is the standard deviation values, and the upper triangular area is the cross-correlation values. The standard deviation and cross-correlation values are encoded as

either IEEE floating-point values or using MISB ST 1201 Floating Point to Integer Mapping. The standard deviation values are not required to be encoded with the same method as the cross-correlation values. The cross-correlation values can be sparsely encoded, saving bytes with large cross-correlation matrixes. Refer to MISB ST 1010 [24] for details on using and implementing the SDCC-FLP.

7.5.6.2 Multi-Dimensional Array Pack (MDARRAY)

The MDARRAY structure defines the efficient encoding of vectors and arrays of consistent type and consistent size data elements. The MDARRAY can encode any dimension of data elements (e.g., 1, 2, 3 (or more) dimensional data). Typical MDARRAY elements are integers, floating-point values or ST1201 Floating Point to Integer mapped values. Other types can be grouped including whole sets or packs as long as they are all the same length. Refer to MISB ST 1303 [25] for details on using an implementing the MDARRAY.

7.5.7 MISP Baseline Packet (Previously MISP Baseline Local Set) Information

The MISP Baseline Packet Information defines basic data components needed in MISP Packet to enable accurate time, versioning, and data reliability.

The MISP Baseline Packet information typically includes three values: **Timestamp**, **Version** and **Check Value**. MISB ST 0603 [26] defines the **Timestamp** and provides the base time for all temporal values in a packet. Temporal values are measurements at a given time. Sub-groups within the packet may refine or override a timestamp as needed.

The **Version** value is typically the minor version number of the standard used to define the packet. The minor version of a document is the “decimal” value after the four-digit standard number. For example, with ST 1201.1, the minor version is the value “1” after the period. This value links a specific version of a controlling document to the metadata generated. This value informs a receiver on how to interpret the values and rules for the parsed metadata or detect when data conforming to an older standard need’s corrective processing. Some standards may manage the version number differently than using the minor version number of the standard.

The **Check Value** is a value-code used to validate metadata delivery from a source to a receiver; also known as a **CRC** (Cyclic Redundancy Check). In most cases, when metadata becomes corrupted or lost, a check value computation detects an issue. Check value’s do not correct data; they only detect a problem, so applications become aware of a problem and can take corrective actions, such as skipping the potentially corrupted data. A check value applies to the KLV root data or packet payload. KLV root data is not contained in any other group; it is parent of all other sets for a given KLV packet of metadata. Some standards use the term “checksum”, which is a specific type of CRC.

7.5.7.1 Check Value Implementation

A check value is the result of an error-detecting algorithm processed over a block of bytes. Systems that build KLV packets compute the check value and include it into the packet. Users of the packet perform the same computation over the data block and compare the result to the packet’s check value. If the check value matches the computed value, the metadata was likely

transmitted without error. If the check value fails to match the computed value, the whole data block is suspicious, and an error recovery process begins.

The data error(s) could be anywhere in the data block, including the keys and lengths. If a packet length is corrupted, skipping the bad data block may skip more elements than desired, so applications should use a more extensive evaluation process. Since the location of an error is unknown, a scanning process searches for the next occurrence of the SMPTE KLV Identifier (0x06-0E-2B-34) starting after the first four bytes of the key. After finding the next SMPTE KLV Identifier, normal KLV decoding continues with key validation, etc.

There are different methods for implementing check values depending on the KLV packet's group type, i.e., Local Set, Variable Length Pack, Defined Length Pack.

7.5.7.1.1 Local Set Check Value Implementation

When implementing a check value with a Local Set Packet, the check value is an item (i.e., tag, length, value triplet) in the Local Set. For ease of implementation the check value triplet is the last item in the Local Set, so the last bytes of the packet contain the check value. The check value algorithm input includes all preceding bytes in the packet, starting with the 16-byte key, length, and all the preceding tags, lengths, and values, up to the tag and length of the check value itself.

Figure 71 illustrates which data (bytes) the check value algorithm uses as input to compute the check value and the location of where the check value is within the packet.

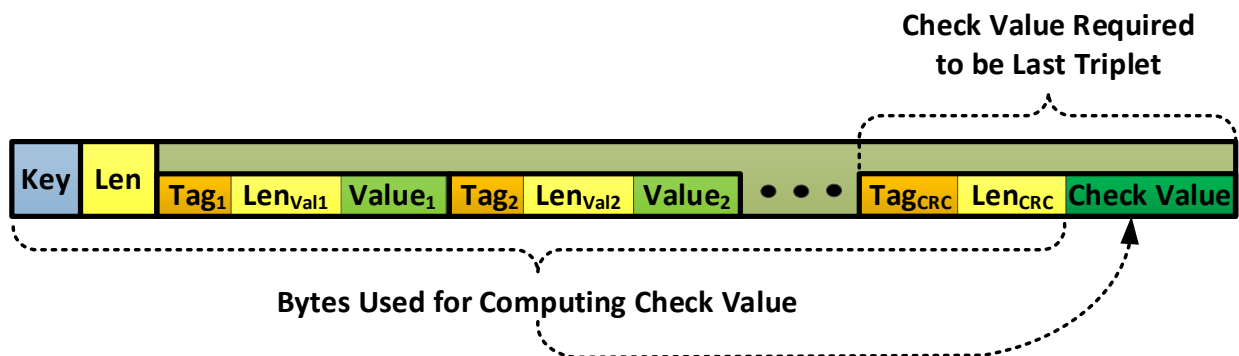


Figure 71: Illustration of Local Set CRC Computation

This method requires all the items to be in the Local Set before the CRC algorithm can begin its processing because the packet length must be known before processing the first tag.

The check value depends on which CRC algorithm the MISB standard requires, i.e., different standards will require different CRC polynomials and CRC lengths.

See Appendix C for how to access reference code.

7.5.7.1.2 Pack Check Value Implementation

When implementing a check value for a KLV Pack, the check value is an item in the Pack definition. For ease of implementation the check value is the final item in the Pack. There are two forms of KLV Packs: Defined Length and Variable Length, both of which utilize the same check value methodology. Different MISB standards (and different versions of the same

standard) can use different parts of the Pack for input into the check value algorithm. There are two methods for Pack check value implementation: method 1 and method 2.

Method 1 combines the pack key and all values from the start of the packet payload to the CRC value itself; the Packet length is not a part of the CRC computation. By not including the length the check value algorithm can compute the check value while assembling the packet data. If the transmission corrupts the length value the check value algorithm will operate on the wrong set of bytes, and therefore, the algorithm should compute an incorrect check value thus flagging the packet as invalid. Figure 72 illustrates method 1 for a Defined Length Pack.

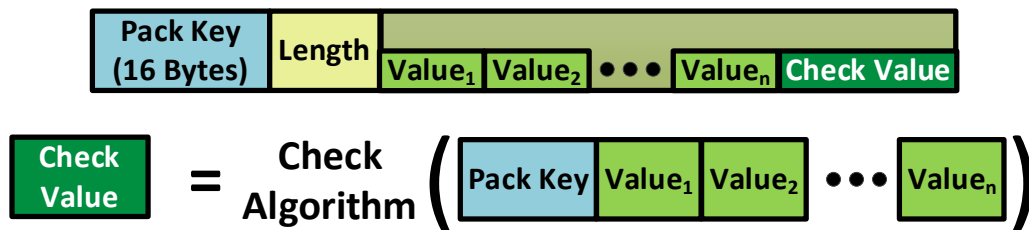


Figure 72: Method 1 – Check Value for a Defined Length Pack

Figure 73 illustrates method 1 for a Variable Length Pack, where L_i is the length for Value_i . The check value algorithm input includes the length of the check value (L_{cv}) itself.

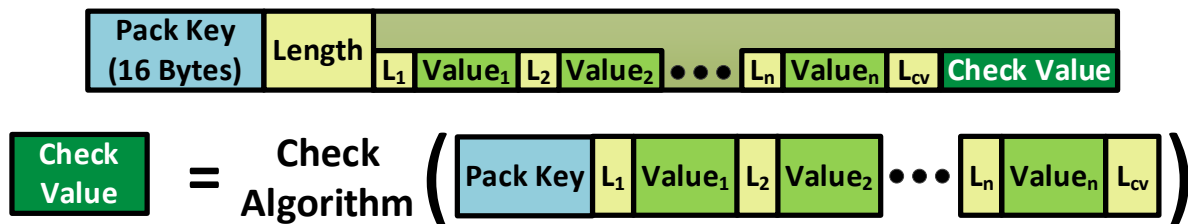


Figure 73: Method 1 – Check Value for a Variable Length Pack

Method 2 uses two check values to reduce the probability of a packet length error which may result in a truncated or exceedingly long read operation. A correct packet length is critical when parsing KLV packets. Given a corrupted packet length there are two outcomes when reading the packet payload: either a shorter length or a longer length. A shorter length means the packet payload read from the KLV stream is not complete; that is, not all bytes are read. In this case, the check value should fail, and recovery should start. A longer length means the packet payload read from the KLV stream overruns the end of the packet. In this case, there is essentially no limit on the length of bytes read, and therefore, the length of a subsequent read operation. A length corruption can cause a very large read of data potentially affecting a systems performance.

Method 2 addresses this by using two check values. A Check value 1 item immediately follows the packet length. With a Defined Length Pack, Check value 1 validates both the packet key and payload length. With Variable Length Packs, Check value 1 validates the packet key, payload length, and length of the check value. Check value 2 is at the end of the payload, it validates the remaining pack items. Figure 74 illustrates method 2 for a Defined Length Pack.

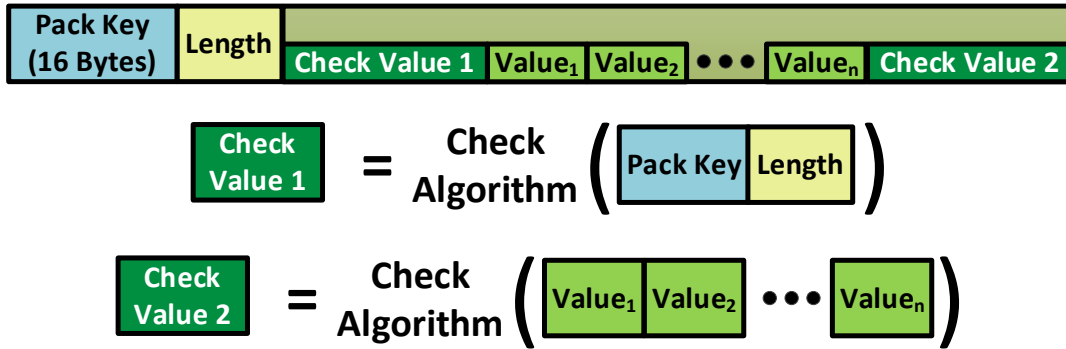


Figure 74: Method 2 – Check Value for a Defined Length Pack

Figure 75 illustrates Method 2 for a Variable Length Pack. The first check value input for Method 2 with a Variable Length pack includes the length of the check value item (L_{cv1}).

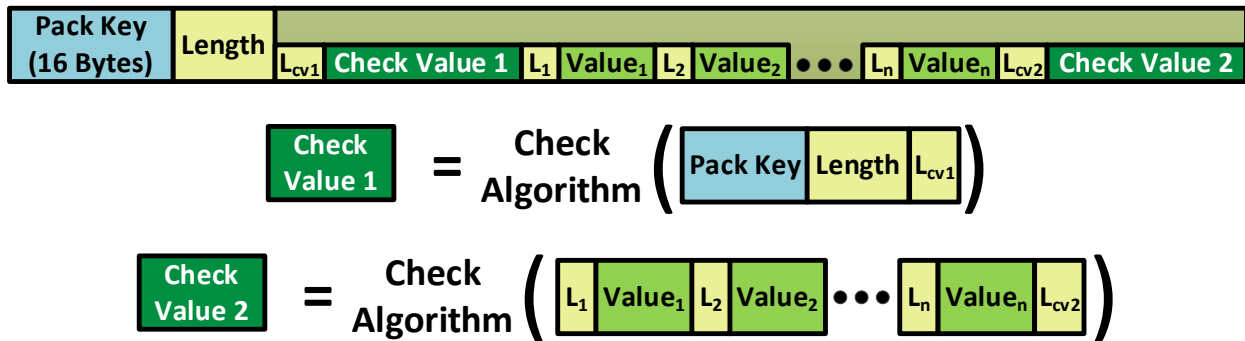


Figure 75: Method 2 – Check Value for a Variable Length Pack

7.5.8 Implementation of Data Preservation and Data Sharing

The MISP supports Data Preservation and Data Sharing by using the Amend Metadata and Segment Metadata list structures discussed in Section 7.1.3. The implementation of these lists requires an additional step of processing beyond a typical SMPTE Local Set. MISB ST 1607 [27] defines the Amend Metadata Local Set and Segment Metadata Local Set with its governing rules for use.

7.5.9 Report-on-Change

Reducing the quantity of data is always an implementation goal as bandwidth-constrained channels present challenges in the delivery of Motion Imagery and metadata. Adding more metadata to provide additional information for exploitation comes at a tradeoff in visual quality of the Motion Imagery. Eliminating redundancy in metadata reporting is a good technique to reduce the overall metadata bandwidth. This is the goal of Report-on-Change.

The Report-on-Change technique reduces bandwidth with KLV Local Sets by including data items only when they change or at a defined refresh rate. Additionally, Report-on-Change allows for data item expiration and enables transmitters to signal expired data items.

The following example describes the development of the Report-On-Change technique. Table 42 illustrates a series of 100 packet transmissions (from a producer to receiver), each with data items X, Y, and ID. Assuming 1 byte each for X and Y, and 4 bytes for the ID, each packet will be 6 bytes producing a total of 600 bytes for the 100 packets.

Table 42: Series of 100 Metadata Packets

1	2	3	4	5	...	37	38	39	...	100
X=10	X=15	X=14	X=12	X=11	...	X=13	X=17	X=12	...	X=21
Y=16	Y=23	Y=22	Y=25	Y=25	...	Y=25	Y=26	Y=24	...	Y=22
ID=Cam1	ID=Cam1	ID=Cam1	ID=Cam1	ID=Cam1	...	ID=Cam1	ID=Cam1	ID=Cam1	...	ID=Cam1

Clearly, the ID is redundant from packet to packet since it's the same value for all 100 packets. Table 43 shows reporting the ID value once in packet 1, then not reporting it for the remaining 99 packets. Removing this redundancy saves 396 bytes (99 packets x 4 bytes) for a reduction of 66% for a new total of 204 bytes = 34% (204/600).

Table 43: Removing ID Redundancy

1	2	3	4	5	...	37	38	39	...	100
X=10	X=15	X=14	X=12	X=11	...	X=13	X=17	X=12	...	X=21
Y=16	Y=23	Y=22	Y=25	Y=25	...	Y=25	Y=26	Y=24	...	Y=22
ID=Cam1										

Measured numeric values may stabilize and not change over a series of packets, as shown with the Y value in packet 4 through packet 37. Table 44 shows removing the Y value redundancy in packets 5 through 37 saving another 37 – 5 + 1 = 33 bytes for a revised byte count of 171 bytes (204 - 33) or 171/600 = 28.5% of the original 600 bytes.

Table 44: Removing Y Value Redundancy

1	2	3	4	5	...	37	38	39	...	100
X=10	X=15	X=14	X=12	X=11	...	X=13	X=17	X=12	...	X=21
Y=16	Y=23	Y=22	Y=25				Y=26	Y=24	...	Y=22
ID=Cam1										

The producer makes the decisions on which items are redundant and removes those items. The receiver keeps track of the history of each item and should a packet be missing an item the historical value becomes the current value. However, if a receiver misses the last update to a value, the receiver will not have appropriate metadata for the item. For example, if the receiver starts “listening” to (or “joins”) the metadata stream at packet 5 in Table 44, the receiver will never know the ID and would not discover the value of Y until packet 38. The solution is to periodically repeat redundant values so receivers may in time discover all values sent. Table 45 shows an example with redundant items re-sent every 10 packets (i.e., 11, 21, etc.). When a receiver joins this stream at packet 5, at packet 11 the receiver discovers all the metadata. Adding

redundant data back into the packets increases the data transmitted only slightly; in this example $171 + 5 \times 3 + 6 \times 4 = 210$ bytes for a total of $210/600 = 35\%$ of the original data rate.

Table 45: Data Reduction with Static Data Updates

1	2	...	10	11	12	...	20	21	22	...	37	38	39	...
X=10 Y=16 ID=Cam1	X=15 Y=23	...	X=52	X=41 Y=25 ID=Cam1	X=29	...	X=13	X=75 Y=25 ID=Cam1	X=12	...	X=13	X=17 Y=26	X=12 Y=24	...

These examples illustrate three data item “states” a receiver will need to account for: **Unknown**, **Known-Changing**, and **Known-Static**. When a data item has not had a value defined, the state of the data item is **Unknown**. A data item may never have a value so the state of the data item will be Unknown for the entire stream. An example of an Unknown data item (Table 45) is where a receiver joins the stream at packet 5; here both the Y and ID values are unknown until packet 11. When a data item, with a value, is in a packet, the item state is **Known-Changing** (note, the value is unimportant – it may be varying or constant). For example, data item X for all packets, and data item Y for packets 1 through 4 and 38 through 100 (Table 44), are in each packet, therefore Known-Changing for those packets. Data items which are not in a packet after being in the Known-Changing state are **Known-Static** (note, the receiver assumes the value is constant). For example, data item ID for packets 2 through 10, 12 through, 20, etc., and data item Y for packets 6 through 36, where in the Known-Changing state in a previous packet but are not a part of these packets.

The re-sending or “Updating” of redundant data occurs over a time period, instead of the packet count used in the above examples. The time between re-send updates is the **Metadata Refresh Period**, measured in seconds. If an item is not resent within the Metadata Refresh Period, the receiver changes the item to the Unknown state because it cannot trust a past value is correct. An Unknown state indicates the source has discontinued sending this item, potentially caused by a system failure or other reasons. Senders may also force an Unknown state by the method described in the next section.

Motion Imagery System packet rates are usually short compared to the Metadata Refresh Rate; however, it is possible to have a packet rate at or longer than the Metadata Refresh Rate. In this case all data refreshes at the Metadata Refresh Rate.

Metadata Refresh Period: The amount time for the re-transmission of any single Known-Static data item before it becomes Unknown. Unless defined within a MISB standard, the default Metadata Refresh Period is 30 seconds.

Known-Changing: A data item with a value that is in a packet. Typically, a constantly changing value; for example, the position of an aircraft is constantly changing and in every packet. If the time between packets is at or larger than the Metadata Refresh Period, re-send Known-Changing items at least once every Metadata Refresh Period.

Known-Static: A data item not in a packet but was previously Known-Changing. For example, the Mission ID was Known-Changing in the previous packet but not part of the current packet (since its value does not change throughout the mission). Producers resend Known-Static items at least once every Metadata Refresh Period or sooner to keep them in one of the “Known” states. A Known-Static data item becomes Unknown if not resent by the end of the Metadata Refresh Period.

Unknown: Where a data item’s value is undefined. The data item could be undefined because the platform/system/sensor does not support the item or there is no need to send the item’s data for some reason. For example, powering down a sensor in flight, thus there is no information until the sensor is back on.

Figure 76 is a state diagram showing the three states and the transitions between the states. An item is in the Known-Changing whenever a data update occurs, from any state. An item enters the Known-Static state when there is not an update to the data item. An item enters the Unknown state either when the item times out (Metadata Refresh Period elapses) from the Known-Static state or the item is intentionally “Deleted”, while in one of the other two states.

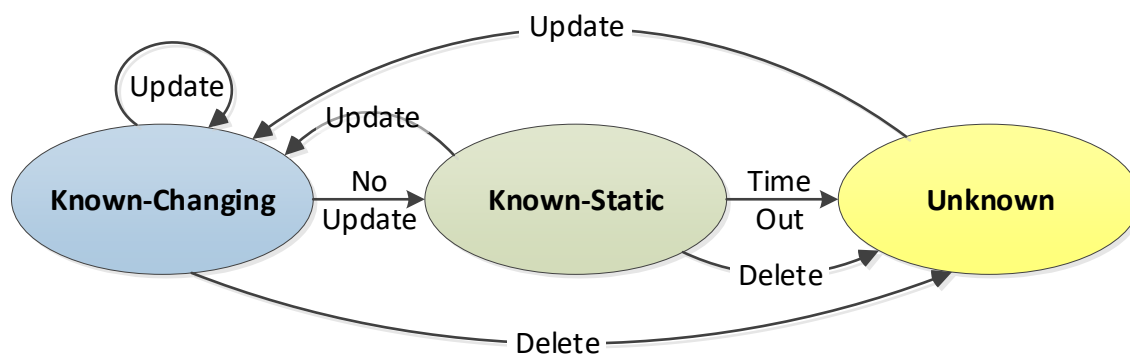


Figure 76: Report-on-Change State Diagram

7.5.9.1 Signaling Items not transmitted

A receiver assumes metadata is valid when items meet the Metadata Refresh Period constraint. Should a sender not report an item within this timeframe the receiver sets the data item to Unknown. Waiting for the Metadata Refresh Period to expire may not be the best method for a source to inform the receiver that the item has become Unknown. As an illustrative example, Figure 77 depicts a situation where the source transmits metadata from a platform at least as

frequently as the Metadata Refresh Period, but a failure in the sensor renders the Sensor Metadata in error or Unknown, starting in packet 2.

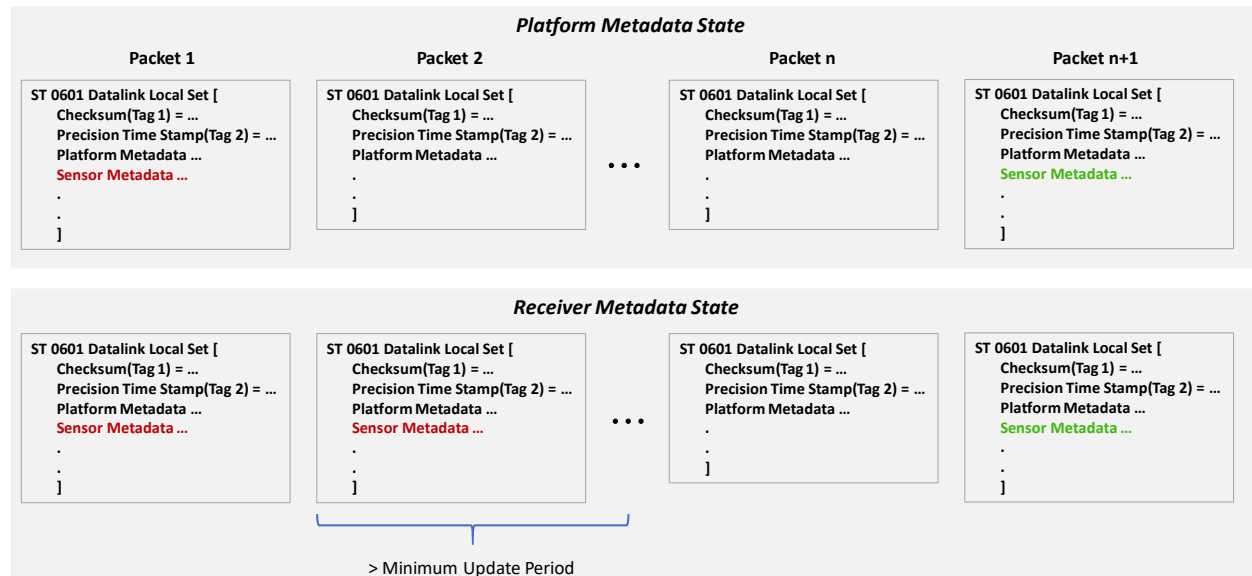


Figure 77: Example of metadata lost or Unknown metadata

In Packet 1 the source transmits a complete set of metadata (top) to the receiver (bottom). In Packet 2 the source does not report the Sensor Metadata (red in Packet 1). With Report-on-Change the receiver continues to assume the Sensor Metadata in Packet 2 (bottom) is valid until the Metadata Refresh Period expires. Thereafter, the receiver does not obtain any Sensor Metadata information as shown through Packet n and the Sensor Metadata item is set to Unknown. Once the platform reestablishes the Sensor Metadata to send (green) at Packet n+1 (top) the receiver likewise receives the Sensor Metadata (bottom).

This example illustrates metadata item(s) moving from the states of Known-Changing or Known-Static to Unknown, in which case the sender stops transmitting one or more items. After the Metadata Refresh Period the receiver determines the items are Unknown. Ideally, alerting a receiver immediately when data has become Unknown provides better sender/receiver correlation. An immediate alert provides information useful for backend processing like searches and it could signal something in the system is potentially wrong; thus, it alerts the receiver to the issue.

The Report-on-Change technique provides for the immediate alert using a Zero-Length-Item (ZLI) (see Figure 78), which is a Local Set item with no value (i.e., tag followed by a length of zero, with no value). The receiver interprets a ZLI as the value becoming immediately Unknown. The use of a ZLI comes only after a value has been in the Known state (either Known-Changing or Known-Static). MISB standards indicate which Local Set items are permitted to use a ZLI. The MISB recommends only using a ZLI for an item where the item's description allows it.

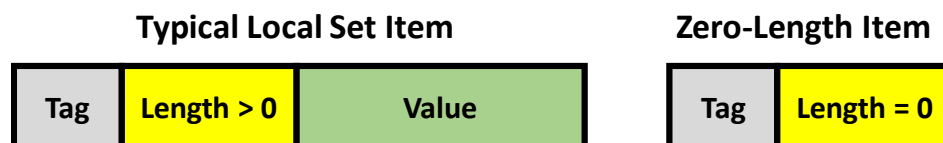


Figure 78: Local Set item vs. Zero-Length Item

MISB ST 0107 states the Report-on-Change rules and requirements, including requirements for ZLI. These requirements apply across all MISB standards. The rules do not allow a producer to circumvent the minimum metadata requirements set forth by MISB ST 0902.

7.5.9.2 Impulse Items

In addition to the three data item states, it is possible to have an “Impulse” data item whose value transitions from Unknown to Known-Changing then back to Unknown in the span of three KLV packets.⁸ An example of an Impulse item is the “Weapon Fired” item in MISB ST 0601. Local Set packets do not include the Weapon Fired item (i.e., Unknown) until releasing a weapon. The first KLV packet after releasing the weapon defines the Weapon Fired item (i.e., Known-Changing) with an appropriate value. The next KLV packet uses a ZLI to set the Weapon Fired item back to Unknown. In the span of three packets the Weapon Fired items goes from Unknown → Known-Changing → Unknown.

7.5.9.3 Metadata Distribution

Within a constrained bandwidth channel, shared by both Motion Imagery and metadata, transmitting a large quantity of metadata at one time can impact the received quality of the Motion Imagery. To prevent this, the MISB recommends distributing the Known-Static metadata items over a 30-second refresh period. For example, instead of sending items A, B and C every 30 seconds at time 00:00:00, send item A every 30 seconds starting at time 00:00:00; send item B every thirty seconds starting at time 00:00:05 (five seconds later); send item C every thirty seconds starting at time 00:00:10 (10 seconds after the first item); etc. This distributes the metadata over time. Further, some list items can be sent as individual list items (or small groups of items) distributed over multiple packets instead of all at once. Metadata items which allow this are noted in the standard document describing those items.

7.5.10 Metadata Identifiers

Several MISB standards need to include identification within a data stream or data set. The Universal Unique Identifier (UUID) is one method for providing an identifier.

7.5.10.1 Universally Unique Identifier

The Universal Unique Identifier (UUID) standards ([28] and [29]) define multiple techniques for generating UUIDs. NGA has written a Recommended Practice NGA.RP.0001 [30] that discusses allowed UUID versions.

UUID Version 1 uses the Ethernet MAC address and system clock in implementation; version 4 relies on a random number generator; version 5 uses SHA hash coding of some textual information.

This standard recommends, when possible, to construct UUID’s from unique information through the version 5 UUID coding process, for example choose a name-space UUID then

⁸ This type of value behavior is like a unit impulse function in signal processing.

combine manufacturer name, model number, serial number, and use the SHA hash function to generate the UUID.

If using version 4, ensure to provide a good cryptographic random number generator; refer to [31] for information on random number generation.

MISB does not recommend Version 1 UUIDs, but, if necessary, generate them only if the device has a MAC address.

Per NGA.RP.0001, generate UUIDs using only UUID versions 1, 4 and 5. Additionally, do not use the null MAC address for generating UUIDs.

7.5.10.1.1 Combining Multiple UUIDs

Sometimes it is necessary to form a new UUID from two or more existing UUID's. The following technique uses the version 5 UUID construction algorithm but without a name space UUID. This technique enables the possibility of reconstructing a UUID from the source UUIDs.

Some computer languages may supply a version 5 UUID construction algorithm that does not require a name space UUID. However, verify these algorithms do not use a default UUID or nil (all zeros) UUID. The following algorithm manually builds the version 5 UUID without using a name space UUID. The use of this algorithm provides backwards compatibility with historical versions of the MISB ST1204 standard.

To combine two or more UUIDs use the following algorithm (Table 46 provides an example of combining two UUIDs using this technique):

- 1) Convert each of the UUIDs into the hexadecimal format defined in Section 6.4 of [28] in uppercase and remove the separator characters (i.e., dashes "-"),
- 2) Append UUIDs together into one hex character string,
- 3) Perform SHA1 hash code algorithm on the hex character UTF8 string producing a 20-byte hash code,
- 4) Truncate the last four bytes of the SHA1 hash code,
- 5) Convert to a UUID by setting the UUID version number to 5 (SHA1 UUID) and variant (fixed value of 2) information.

To emphasize the process of step (3) – SHA1 encode the UTF8 hex string and not the byte sequence it represents.

Table 46 shows an example of combining two UUIDs into one UUID. The first column matches the step numbers in the algorithm list above. The **red** text in step 3 shows the truncated bytes and the yellow highlighted characters show the updates to those bytes for inserting the version and variant information.

Table 46: UUID Combination Example

Step	UUID 1	UUID 2
Start	f81d4fae-7dec-11d0-a765-00a0c91e6bf6	1ea5de30-e1d3-40fa-b501-2ca5cb58ca25
1	F81D4FAE7DEC11D0A76500A0C91E6BF6	1EA5DE30E1D340FAB5012CA5CB58CA25
2	"F81D4FAE7DEC11D0A76500A0C91E6BF61EA5DE30E1D340FAB5012CA5CB58CA25" (see Note)	

3	CD9A0EF224A6C3436A213CD6BE881B52 10F2282E
4	CD9A0EF224A6C3436A213CD6BE881B52
5	CD9A0EF224A6 5 343 A A213CD6BE881B52
Final	cd9a0ef2-24a6-5343-aa21-3cd6be881b52

Note: The input for the SHA1 algorithm is a string (in the example the 64 characters in the quotes) converted to a byte sequence. When inputting the value into the SHA1 hash algorithm, each string character converts to its UTF8 numeric value. For example, the first character in the string, “F”, converts to a value of 70 or a hex value of 46. The following byte sequence (in hex) is the input into the SHA1 encoding for step (3) (not including the commas, braces, or preceding “0x”):

0x{46,38,31,44,34,46,41,45,37,44,45,43,31,31,44,30,41,37,36,35,30,30,41,30,43,39,31,45,36,42,46,36,31,45,41,35,44,45,33,30,45,31,44,33,34,30,46,41,42,35,30,31,32,43,41,35,43,42,35,38,43,41,32,35}

Appendix A References

- [1] MISB, *MISP-2025.1: Motion Imagery Standards Profile*, 2025-06-11.
- [2] Chris McGlone, *Manual of Photogrammetry*, Sixth Edition, Bethesda, Maryland: American Society for Photogrammetry and Remote Sensing, 2013.
- [3] National Research Council, *Seeing Photons: Progress and Limits of Visible and Infrared Sensor Arrays*, Washington, D.C.: The National Academies Press, 2010.
- [4] SMPTE, *EG 28:1993 - Annotated Glossary of Essential Terms for Electronic Production*.
- [5] IETF, *RFC 768 - User Datagram Protocol*, 1980-08.
- [6] IETF, *RFC 793 - Transmission Control Protocol*, 1981-09.
- [7] ISO/IEC, *13818-1:2023 - Information Technology - Generic Coding of Moving Pictures and Associated Audio Information - Part 1: Systems*.
- [8] MISB, *ST 1402.2 - MPEG-2 Transport Stream for Class 1/Class 2 Motion Imagery, Audio and Metadata*, 2016-10-27.
- [9] IETF, *RFC 3550 - RTP: A Transport Protocol for Real-Time Applications*, 2003-07.
- [10] NIST, *Special Publication 330 - The International System of Units (SI), 2019 Edition*. (<https://doi.org/10.6028/NIST.SP.330-2019>)
- [11] ITU-R, *TF.460-6 - Standard-frequency and Time-signal Emissions*, 2002-02.
- [12] IEEE, *1003.1-2024 - Standard for Information Technology - Portable Operating System Interface (POSIX™) Base Specifications, Issue 8*.
- [13] ISO, *8601-1:2019 - Date and Time - Representations for Information Interchange - Part 1: Basic Rules*.
- [14] SMPTE, *ST 336:2017 - Data Encoding Protocol Using Key-Length-Value*.
- [15] SMPTE, *ST 330:2004 - Unique Material Identifier (UMID)*.
- [16] MISB, *ST 0107.5 - KLV Metadata in Motion Imagery*, 2021-10-21.
- [17] ISO/IEC, *8825-1:2021 - Information Technology - ASN.1 Encoding Rules - Part 1: Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER)*.
- [18] ITU-T, *X.690 - Information Technology - ASN.1 Encoding Rules - Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER)*, 2021-02.
- [19] ISO/IEC, *646:1991 - Information Technology - ISO 7-bit Coded Character Set for Information Interchange*.
- [20] ISO/IEC, *10646:2020 - Information Technology - Universal Coded Character Set (UCS)*.
- [21] IEEE, *754-2019 - Standard for Floating-Point Arithmetic*.
- [22] MISB, *ST 1201.5 - Floating Point to Integer Mapping*, 2021-06-24.
- [23] MISB, *ST 0807.27 - MISB KLV Metadata Registry*, 2025-06-11.
- [24] MISB, *ST 1010.3 - Generalized Standard Deviation and Correlation Coefficient Metadata*, 2016-10-27.
- [25] MISB, *ST 1303.2 - Multi-Dimensional Array Pack*, 2020-06-25.

- [26] MISB, *ST 0603.5 - MISP Time System and Timestamps*, 2017-10-05.
- [27] MISB, *ST 1607.2 - Constructs to Amend/Segment KLV Metadata*, 2025-06-11.
- [28] ITU-T, *X.667 - Information Technology - Procedures for the Operation of Object Identifier Registration Authorities - Generation of Universally Unique Identifiers and Their Use in Object Identifiers*, 2012-10.
- [29] IETF, *RFC 4122 - A Universally Unique Identifier (UUID) URN Namespace*, 2005-07.
- [30] ASM, *NGA.RP.0001_1.0.0 - NGA Recommended Practice for Universally Unique Identifiers*, 2013-01-03.
- [31] NIST, *Special Publication 800-90A Rev. 1 - Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, 2015-06.
(<https://doi.org/10.6028/NIST.SP.800-90Ar1>)

Appendix B Acronyms

CCD	Charge-Coupled Device
CMOS	Complementary Metal Oxide Semiconductor
EG	Engineering Guideline
EMR	Electro-Magnetic Radiation
FMV	Full Motion Video
HF	High Frequency
HSI	Hyper-Spectral Imagery
IP	Internet Protocol
IR	Infrared
ISO	International Standards Organization
KLV	Key Length Value
LIDAR	Light Detection and Ranging
MI	Motion Imagery
MISB	Motion Imagery Standards Board
MISP	Motion Imagery Standards Profile
MPEG	Moving Pictures Expert Group
MSI	Multi-Spectral Imagery
MSID	Metadata Substream Identifier
MTU	Maximum Transmission Unit
RADAR	Radio Detection and Ranging
RFC	Request for Comment
RGB	Red Green Blue
RP	Recommended Practice
RTCP	Real Time Control Protocol
RTP	Real Time Protocol
SAR	Synthetic Aperture Radar
SDI	Serial Digital Interface
SMPTE	Society of Motion Picture and Television Engineers
ST	Standard
TCP	Transmission Control Protocol
TS	MPEG-2 Transport Stream
UDP	User Data Protocol
UHF	Ultra-High Frequency
UTC	Coordinated Universal Time
VHF	Very High Frequency
ZLI	Zero-Length Item

Appendix C Reference Algorithms

This appendix provides information on the MISB reference algorithms hosted on the MISB PiX website (<https://pixtoday.net/article/article/223008>). For access to the MISB PiX site, please send email to MISBChair@nga.mil.

From the home page, click on the “Reference Implementations” link on the left (red circle in Figure 79) or scroll to the “Reference Implementations” section. All MISB Handbook reference software zip files may be found in this section.

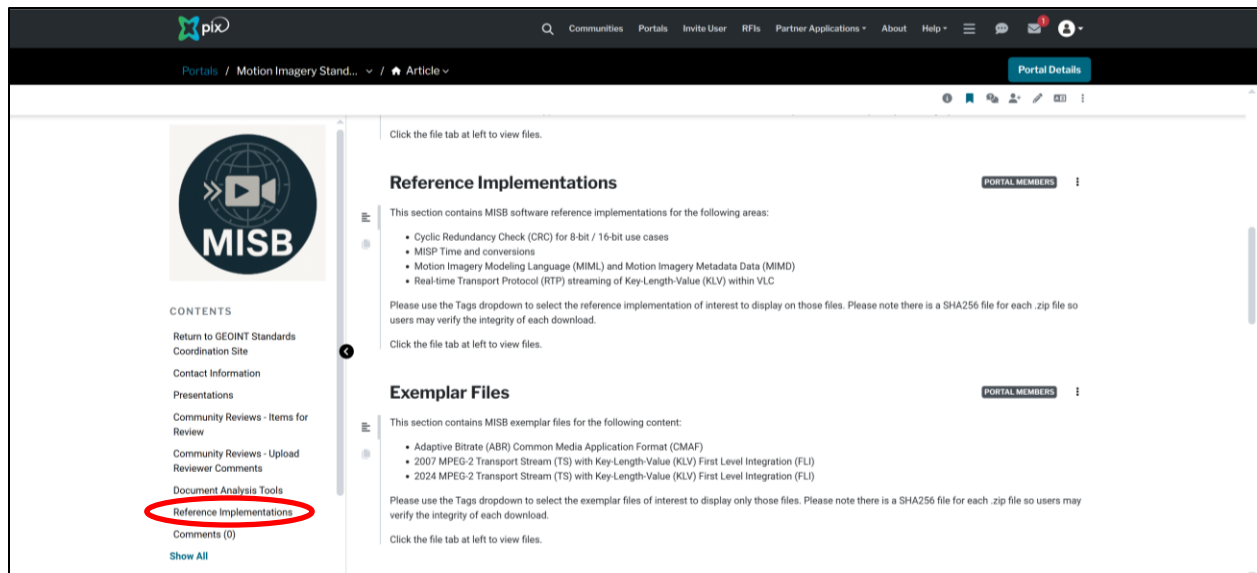


Figure 79: Motion Imagery Standards Board PiX Home Page

The zip files can be downloaded, compiled and executed without using external classes or libraries. All reference code is java and has the following copyright statement:

Copyright (c) 2022 National Geospatial Agency (NGA) This software is made available as is with no liability on NGA's part. Users can make changes as needed with sole responsibility to provide NGA a copy if enhancements are made or issues are corrected.

C.1 Reference Software

The following sections describe the contents of the available zip files.

C.1.1 Time Algorithms

The handbook-time-2022-09-08.zip file contains reference code for the MISB Time algorithms.

This zip file contains source code for defining TAI, UTC, and MISP time java classes, along with conversions between the different time bases. These files are expected to be in the mil.nga.gwg.misb.standards.handbook.time java package but can be relocated as needed.

The TimeConversionExample produces the following result:

1970-01-01T00:00:00.000000000 (MISP) = 0

1970-01-01T00:00:08.000082000 (TAI) = 378691208000082000

1970-01-01T00:00:00.000000000 (MISP) = 0

1970-01-01T00:00:08.000082000 (TAI) = 378691208000082000

2022-07-12T09:05:32.000618000 (MISP) = 1657616732000618000

2022-07-12T09:05:40.000700000 (TAI) = 2036307940000700000

Leap second offset = 37

2022-07-12T09:05:03.000700000 (UTC)

C.1.2 CRC Algorithms

The handbook-crc-2022-09-08.zip file contains reference code for the MISB CRC algorithms.

The MISB uses two different CRC algorithms: CRC-16-CCITT and CRC-8-CCITT. The CRC-16-CCITT algorithm computes a two-byte CRC value computed using the $x^{16} + x^{12} + x^5 + 1$ polynomial over the defined data block. The CRC-8-CCITT algorithm computes a one-byte CRC value computed using the $x^8 + x^2 + x + 1$ polynomial over the defined data block.

The zip file contains two java classes CRC_16_CCITT and CRC_8_CCITT, both of which contain a main method containing test values.

Appendix D KLV-Key Quick Reference Chart

The following page contains a printable KLV-Key Quick Reference Chart to aid in understanding the meaning behind the first six bytes of KLV Keys.

KLV-Key Quick Reference Chart

↱	Used by MISB (Y=Yes, H=Historical or Deprecating (not used anymore), Blank=Not Used)						
	06	OID					
	06	0E					
	06	0E	UL Size				
	06	0E	2B	UL Code			
	06	0E	2B	34	SMPTE Designator		
	06	0E	2B	34	01	Dictionaries	
Y	06	0E	2B	34	01	01	Metadata
	06	0E	2B	34	01	02	Essence
	06	0E	2B	34	01	03	Control
	06	0E	2B	34	01	04	Types
	06	0E	2B	34	02	Groups	
H	06	0E	2B	34	02	01	Universal Sets
	06	0E	2B	34	02	02	Global Sets - Length ASN.1 BER short or long
	06	0E	2B	34	02	22	Global Sets. Length = 1 Byte
	06	0E	2B	34	02	42	Global Sets. Length = 2 Bytes
	06	0E	2B	34	02	62	Global Sets. Length = 4 Bytes
Y	06	0E	2B	34	02	03	Local Sets. Length = ASN.1 BER Short or long, Tag = 1 Byte
Y	06	0E	2B	34	02	0B	Local Sets. Length = ASN.1 BER Short or long, Tag = ASN.1 OID BER
	06	0E	2B	34	02	13	Local Sets. Length = ASN.1 BER Short or long, Tag = 2
	06	0E	2B	34	02	1B	Local Sets. Length = ASN.1 BER Short or long, Tag = 4
	06	0E	2B	34	02	23	Local Sets. Length = 1, Tag = 1
H	06	0E	2B	34	02	2B	Local Sets. Length = 1, Tag = ASN.1 OID BER
	06	0E	2B	34	02	33	Local Sets. Length = 1, Tag = 2
	06	0E	2B	34	02	3B	Local Sets. Length = 1, Tag = 4
	06	0E	2B	34	02	43	Local Sets. Length = 2, Tag = 1 Byte
	06	0E	2B	34	02	4B	Local Sets. Length = 2, Tag = ASN.1 OID BER
	06	0E	2B	34	02	53	Local Sets. Length = 2, Tag = 2
	06	0E	2B	34	02	5B	Local Sets. Length = 2, Tag = 4
	06	0E	2B	34	02	63	Local Sets. Length = 4, Tag = 1
	06	0E	2B	34	02	6B	Local Sets. Length = 4, Tag = ASN.1 OID BER
	06	0E	2B	34	02	73	Local Sets. Length = 4, Tag = 2
	06	0E	2B	34	02	7B	Local Sets. Length = 4, Tag = 4
Y	06	0E	2B	34	02	04	Variable Length Packs. Length = ASN.1 BER short or long
	06	0E	2B	34	02	24	Variable Length Packs. Length = 1 Byte
	06	0E	2B	34	02	44	Variable Length Packs. Length = 2 Byte
	06	0E	2B	34	02	64	Variable Length Packs. Length = 4 Byte
Y	06	0E	2B	34	02	05	Defined Length Packs
	06	0E	2B	34	03	Wrappers and Containers	
	06	0E	2B	34	03	01	Simple Wrappers and Containers
	06	0E	2B	34	03	02	Complex Wrappers and Containers
	06	0E	2B	34	04	Labels	
	06	0E	2B	34	04	01	Labels Register